



Dynamic Software Licensing

Best Practices

International Association of MediaTech (IAMT)

Status: Published Best Practices

07 September 2026

Version Information

- Version: 1.0.0
- Status: Published Best Practices
- Publication date: 07 September 2026

This PDF is an assembled publication view generated from the Markdown sources in the repository.

Management Summary

Dynamic Software Licensing (DSL) defines an interoperable framework for licensing software in dynamic broadcast and media environments. It is aimed at operating models in which applications, media functions, and supporting services may run across on-premises, cloud, hybrid, and resilience-oriented topologies. In those environments, licensing must behave as a governed runtime capability rather than as a fixed activation step tied to a single host or appliance.

The main problem DSL addresses is the gap between traditional licensing assumptions and current deployment practice. Many established licensing systems were built around static infrastructure, manual fulfillment, long-lived instances, and limited workload mobility. Modern operations depend on automated provisioning, short-lived runtime contexts, controlled failover, API-based control planes, and policy-driven service continuity. When the licensing layer does not match those conditions, it introduces avoidable operational risk, slows deployment, complicates recovery, and reduces portability across environments and vendors.

DSL responds to that problem by separating commercial entitlement from runtime authorization. Entitlements define the durable customer rights for an application or capability. Leases represent the time-bounded runtime grants issued to specific workloads within those rights. Policies define renewal timing, grace behavior, and enforcement actions. Identity evidence and binding rules define how the control plane establishes trust in the requesting workload. Usage and metering events provide the basis for audit, analytics, and commercial reconciliation. This gives DSL a complete lifecycle for issue, renew, reissue, revoke, inspect, and observe operations without coupling the model to a single vendor architecture.

From a management perspective, the benefit of DSL is control with fewer structural dependencies. The framework improves clarity over who is entitled to use what, under which conditions, for how long, and with which audit trail. That supports governance, service continuity, tenant separation, security review, and operational accountability across vendors, customers, and integrators. It also reduces the risk that licensing becomes the constraint that blocks cloud transition, disaster- recovery planning, cross-site mobility, or multi-vendor deployment choices.

DSL is intentionally specified as an open best-practice model, not as a proprietary product design. It does not require a single commercial modality, a single token format for all vendor-specific payloads, or a single deployment technology. Instead, it defines a common vocabulary and a stable set of architectural expectations that different implementations can adopt while preserving local product differentiation. This is relevant to software vendors designing licensing platforms, customer organizations evaluating portability and governance, system integrators assembling multi-vendor solutions, and industry groups seeking a shared reference model.

This repository provides that model in a form that can be reviewed and implemented. The document set includes terminology, design principles, a reference model, conformance framing, normative requirements, canonical API definitions, canonical data-model definitions, security and trust guidance, operational profiles, and migration considerations. Its purpose is to provide a practical basis for decision-making, architecture review, implementation planning, and further industry alignment, rather than a purely conceptual position paper.

Change Log

All notable changes to the Dynamic Software Licensing repository should be documented in this file.

This changelog is used to track the evolution of the DSL best practices as an IAMT initiative, including structural, editorial, and content-related changes.

The format is intentionally lightweight at this stage and is primarily intended to support transparency across published baselines and ongoing repository evolution.

[1.0.0] - 2026-09-07

Changed

- Added a management summary to the repository README and reused it as dedicated front matter content in the generated PDF
- Separated version information, management summary, change log, and table of contents more clearly in the PDF front matter
- Added lifecycle and sequence diagrams to strengthen visual parity in Annex D and Annex G
- Improved PDF rendering for Markdown lists so bullet lists no longer collapse into inline dash paragraphs
- Improved PDF rendering for inline code inside Markdown tables so long endpoint and path values wrap without overlapping text
- Added consistent PDF header and footer metadata with document title, version, date, and page number on content pages
- Clarified API companion documentation around schema-backed and intentionally Markdown-only model elements
- Aligned the OpenAPI companion and related wording with the current published baseline
- Updated PDF publishing so main tracks the current published release and keeps the latest PDF aligned with official tagged versions
- Integrated the current review-feedback fixes collected on the governance release-readiness branch ahead of the next baseline
- Expanded the FAQ with implementation, migration, security, and onboarding questions for first-time readers
- Removed repeated per-chapter status blocks and centralized status handling in the repository-level guidance and release surfaces
- Promoted the repository baseline from reviewed draft to published best practices version 1.0.0

Contents

[1.0.0] - 2026-09-07	4
00. Status of this Document	10
01. Introduction	13
01.01. Industry Context	15
01.02. Current Licensing Challenges	17
01.03. The DSL Initiative	19
01.04. Objectives and Vision	21
02. Scope	24
02.01. In Scope	26
02.02. Out of Scope	28
02.03. Intended Audience	29
03. Conventions and Normative Language	32
03.01. Normative Keywords and Statement Strength	33
03.02. Content Classification	35
03.03. Platform-Agnostic Boundaries and Annex Use	37
04. Terminology and Abbreviations	39
04.01. Core Terms	41
04.02. Architecture Terms	42
04.03. License and Policy Terms	43
04.04. Operational Terms	44
04.05. Abbreviations	45
05. Stakeholders, Use Cases and Requirements	48
05.01. Stakeholder Groups	49
05.02. Personas and Perspectives	53
05.03. Use Case Classes	63
05.04. Derived Requirement Themes	72
05.05. Workflow Definitions	80
06. Design Principles	95
06.01. Design Principles	96
07. Reference Model	100
07.01. Conceptual Model	101
07.02. Functional Components	105
07.03. Control Plane	109
07.04. Workspace Plane	113
07.05. End User Portal	118
07.06. Trust Model	120
07.07. Responsibility Model	127

08. Conformance and Profiles	136
08.01. Conformance Targets	138
08.02. Conformance Levels	139
08.03. Profile A - Foundational	142
08.04. Profile B - Cloud Native	144
08.05. Profile C - Advanced Interoperable	145
09. Normative Requirements	146
09.01. General Requirements	148
09.02. Identity and Binding Requirements	150
09.03. Lease and Policy Requirements	152
09.04. API Requirements	153
09.05. Metering Requirements	155
09.06. License Modality Requirements	156
09.07. Security Requirements	158
09.08. Operational Requirements	159
09.09. Integration Requirements	160
10. Canonical API Model	163
10.01. Overview	164
10.02. Issue	169
10.03. Renew	173
10.04. Move or Reissue	177
10.05. Revoke	181
10.06. Usage Record	185
10.07. JWKS and Public Keys	189
10.08. Error Model	192
11. Canonical Data Model	199
11.01. Lease Object	200
11.02. Entitlement Object	204
11.03. Policy Object	206
11.04. Usage Event Object	208
11.05. Token Claims Object	210
11.06. Identity Evidence Object	213
11.07. License Envelope Object	215
12. Security and Trust Framework	218
12.01. Security Objectives	219
12.02. Threat Model	223
12.03. Trust Assumptions	227
12.04. Security Controls	230
12.05. Audit and Compliance	236
12.06. Offline and Grace Behavior	238
13. Operational and Deployment Profiles	243

13.01. 24x7 Broadcast Profile	244
13.02. Event Production Profile	246
13.03. Test and Validation Profile	248
13.04. AI Outcome Services Profile	250
13.05. Hybrid and DR Profile	252
14. Migration and Transition	255
14.01. Assessment and Preparation	256
14.02. Pilot and Parallel Operation	258
14.03. Technical Transformation	259
14.04. Business Logic Expansion	260
14.05. Full Migration and Decommissioning	261
14.06. Scaling and Industry Establishment	263
15. Governance and Future Work	265
Annex A. Conformance Checklist	268
Purpose	268
How to Use This Annex	268
Foundational Checklist	268
Cloud Native Checklist	269
Advanced Interoperable Checklist	269
Annex Positioning	270
Annex B. API Contracts	270
Purpose	270
Contract Scope	270
Baseline Endpoints	271
Common Request Expectations	271
Common Response Expectations	271
Error Model	272
Contract Notes	272
Machine-Readable OpenAPI Companion	272
Annex Positioning	272
Annex C. Token Claims	273
Purpose	273
Claim Set Overview	273
Baseline Claims	273
Claim Semantics	274
Relationship to Other Artifacts	274
Annex Positioning	274
Annex D. Lifecycle and State Model	274
Purpose	274
Canonical Lease States	274
Lifecycle Overview Diagram	275

Primary Lifecycle Operations	275
Transition Semantics	275
Operational Notes	276
Annex Positioning	276
Annex E. Terminology Reference	276
Purpose	276
Preferred Terms	276
Terminology Guidance	277
Annex Positioning	278
Annex J. FAQ	278
Purpose	278
Frequently Asked Questions	278
Annex Positioning	281
Annex F. AWS Reference Architecture	281
Purpose	281
Scope and Status	281
AWS-Specific Design Priorities	282
Reference Architecture Zones	282
Example Mapping	285
AWS Account and Organization Structure	285
Identity and Access Interpretation	285
Example Interaction Pattern	286
Design Considerations	286
Relationship to the Main Document	286
Annex Positioning	287
Annex G. Example Sequences	287
Purpose	287
Sequence 1. Initial Issue	287
Sequence 2. Renewal with Temporary Connectivity Loss	288
Sequence 3. Reissue for Failover	289
Relationship to the Main Document	289
Annex Positioning	289
Annex H. Persona Mapping	289
Purpose	289
Persona-to-Concern Mapping	289
Interpretation Notes	290
Relationship to the Main Document	290
Annex Positioning	290
Annex I. Use Case Mapping	291
Purpose	291
Use Case Mapping Summary	291

Interpretation Notes	291
Relationship to the Main Document	291
Annex Positioning	292

00. Status of this Document

Purpose

This chapter defines the status, intended role, maturity, and interpretation of the Dynamic Software Licensing best practices.

It explains what this document is, what it aims to achieve, and how it is understood as a published baseline and through later revision cycles.

Initiative Context

This document is part of **Dynamic Software Licensing (DSL)**, an **IAMT initiative** focused on modern software licensing approaches for broadcast and media workflows.

The best-practices document is maintained as a structured industry document that supports collaboration across manufacturers, broadcasters, media organizations, system integrators, and other stakeholders.

Document Type

This document is **industry best practices**.

It is not currently a formal standard, certification framework, or legal specification. However, it is intentionally organized in a standard-like way to improve clarity, consistency, interoperability, and implementation guidance.

Intended Role of the Best Practices

The best-practices document:

- establish a common industry vocabulary,
- define a shared conceptual and architectural model,
- describe a practical target model for licensing, metering, trust, and policy handling,
- support migration away from fragmented or legacy licensing approaches,
- provide a basis for future alignment across the industry.

Current Maturity

Current maturity: **Published Best Practices**

At this stage, this means:

- the document structure is stable and publicly publishable,
- the main chapters, annexes, API companion, and PDF outputs form a coherent baseline,
- terminology and cross-chapter framing have been aligned to a common baseline,

- future updates should be handled as explicit versioned revisions rather than as draft-stage restructuring,
- the repository serves as the authoritative published baseline for ongoing adoption and feedback.

Unified Status Model

This best practices uses a centralized status model defined in `GOVERNANCE.md`.

The model is expressed at the repository and publication level rather than repeated in every chapter file.

Chapter-Level Interpretation Chapter-specific purpose, normative weight, and editorial role are described in the chapter prose itself instead of through repeated status blocks.

How This Publication Should Be Interpreted

This publication should currently be interpreted as:

- the official published baseline of the DSL best practices,
- a structured and coherent reference for architectural, operational, and implementation alignment,
- a versioned release that can be cited while remaining open to later revision through new tagged baselines.

Unless explicitly stated otherwise, examples, implementation references, and platform-specific content should not be interpreted as mandatory requirements.

Versioning Approach

The best-practices document is expected to evolve through versioned revisions.

A version may include:

- structural changes,
- chapter refinements,
- terminology updates,
- clarified requirement language,
- additional annex content,
- reclassification of content between normative and informative sections.

Significant changes should be reflected in:

- this chapter,
- `CHANGELOG.md`,
- the affected chapter or annex files.

Normative Development Status

Normative keywords such as **MUST**, **SHOULD**, and **MAY** appear where appropriate to express disciplined requirement language and interpretation strength.

Publication Status

The document is now published as best practices of the Dynamic Software Licensing initiative.

That publication baseline is supported by:

- structural completeness,
- terminology consistency,
- stable chapter ownership,
- reviewed requirement language,
- coherent annex structure,
- clear interpretation guidance.

Published Baseline

This chapter defines the status interpretation for the published baseline issued as version 1.0.0.

Future changes should focus on controlled revision, wording precision, traceability, and compatibility across later tagged releases rather than on reintroducing repository-wide draft status handling.

01. Introduction

Purpose

This chapter introduces the business and technical context behind the Dynamic Software Licensing best practices.

It explains why a new licensing approach is needed, why existing models are increasingly misaligned with modern broadcast and media workflows, and why the DSL initiative is relevant for the industry.

Initiative Context

Dynamic Software Licensing (DSL) is an IAMT initiative focused on modern software licensing for broadcast and media workflows.

This best-practices document supports a more consistent and future-ready licensing approach across manufacturers, broadcasters, media organizations, system integrators, and other stakeholders.

Industry Background

The broadcast and media industry is transitioning from hardware-bound and static deployment models toward software-defined, cloud-enabled, and hybrid operating environments.

As workflows become more dynamic, distributed, and automated, licensing can no longer remain a separate, manual, or infrastructure-external function. Licensing must evolve together with the services, platforms, and orchestration models it governs.

Problem Context

Current software licensing approaches often create friction in modern environments.

Recurring challenges include:

- fragmented product and licensing models,
- poor portability between on-premises and cloud environments,
- manual setup, assignment, and renewal processes,
- limited alignment with DevOps, Infrastructure-as-Code, and CI/CD workflows,
- insufficient transparency for usage, cost, and forecasting,
- limited auditability and incomplete security integration.

These issues increase complexity for both vendors and customers and slow down the transition toward more agile and service-oriented operating models.

Why These Best Practices Exist

The DSL best practices exist to provide a shared framework for a more modern and interoperable licensing approach.

The intention is to support a licensing model that is:

- implementation-oriented,
- secure,
- observable,
- portable,
- automation-friendly,
- suitable for both continuous and event-driven production scenarios.

Role of This Chapter

This chapter does not define detailed requirements.

Its role is to:

- establish the industry context,
- summarize the problem space,
- explain the motivation for the DSL initiative,
- create a common starting point for the remainder of the best practices.

Subchapters

This chapter is organized into the following subchapters:

01.01. Industry Context File: 01-industry-context.md

Describes the broader industry transformation from hardware-bound, CapEx-driven models toward software-defined, cloud-enabled, and hybrid production environments.

01.02. Current Licensing Challenges File: 02-current-challenges.md

Identifies the concrete friction points that customers, system integrators, and vendors experience with current licensing approaches, organized by impact and structural nature.

01.03. The DSL Initiative File: 03-dsl-initiative.md

Explains the origin, purpose, and collaborative nature of the IAMT Dynamic Software Licensing initiative and who is involved.

01.04. Objectives and Vision File: 04-objectives-and-vision.md

Defines what the DSL best practices aim to achieve and describes the target vision and core principles underpinning the framework.

Chapter Positioning

This chapter provides the common entry point for readers and reviewers by framing the industry problem, the motivation for DSL, and the overall objective of the best practices.

It serves as the rationale layer for the document rather than as a requirement chapter. Its role in the published baseline is to ensure that later technical and normative chapters are interpreted against the right industry context.

01.01. Industry Context

From Hardware to Software-Defined Operations

The broadcast and media industry is in the midst of a fundamental structural transition.

For decades, production infrastructure was built around dedicated hardware: purpose-built appliances and servers, fixed configurations, and long capital investment cycles. Licensing followed the same logic — tied to physical units, serial numbers, or hardware identifiers such as MAC addresses and hostnames. Both infrastructure and licensing were treated as long-lived, static assets.

That model is no longer fit for purpose.

Production environments are increasingly moving to software-defined workflows running on commodity servers, virtualized infrastructure, container orchestration platforms, and public, private, or hybrid cloud environments. Workloads scale up and down dynamically. Systems are deployed, migrated, and decommissioned through automation rather than manual intervention. Applications run on infrastructure that may have no fixed identity, no permanent hostname, and no physical presence at any single location.

This transition is not marginal. It affects playout, graphics, transcoding, media processing, quality control, contribution, and virtually every other function across the production chain.

The CapEx-to-OpEx Shift

The move to software-defined infrastructure was partly driven by a financial argument.

Chief Financial Officers across the industry recognized that hardware-centric investment models carried inherent risk: capital expenditures are committed years in advance, based on capacity projections that may not match actual production demand. Over-investment ties up capital in underutilized assets. Under-investment creates capacity gaps at critical moments.

Software-defined, cloud-enabled models offered an alternative: a pay-as-you-go approach where costs align more closely with actual usage, where capacity can be adjusted in near real time, and where organizations can adopt best-of-breed multi-vendor solutions without being locked into a single supplier's hardware stack.

The financial logic was compelling. But the licensing models that came with the software did not make the same transition.

The Licensing Gap

As infrastructure became more dynamic, licensing remained static.

When an organization moves a workload from a physical server to a virtual machine, or spins up a cloud instance for a live event, or deploys a container as part of an automated pipeline, the software running in that environment still requires a valid license. But the mechanisms used to issue, bind, validate, and manage that license were designed for a world where the underlying infrastructure did not change.

Hardware-bound licenses break when instances are replaced, migrated, or scaled. Manual activation processes block automation. Perpetual, node-locked models do not map cleanly to elastic, ephemeral deployments. The result is operational friction at the exact points where modern workflows need to move fastest.

This gap — between the pace of infrastructure modernization and the rigidity of existing licensing models — is the central problem the DSL best practices are designed to address.

Broadcast Operations in a Hybrid World

The transition is not a binary switch from on-premises to cloud.

Most broadcast and media organizations operate in hybrid environments, where on-premises infrastructure coexists with cloud deployments, where live productions mix permanent installations with burst capacity, and where the same software products may need to run across multiple environments under consistent licensing governance.

Disaster recovery adds further complexity. Redundant systems, hot-standby configurations, and blue-green deployment patterns — all common in broadcast operations — require licenses that can follow workloads across environments without manual intervention at the point of failover.

The expectation placed on licensing has changed: it must be as dynamic, portable, and automatable as the infrastructure it governs.

01.02. Current Licensing Challenges

Overview

Organizations across the broadcast and media industry — customers, system integrators, and independent software vendors alike — encounter a consistent set of problems with current software licensing approaches.

These challenges are not isolated technical issues. They form a pattern of structural misalignment between how licensing works today and what modern software-defined, cloud-enabled production environments require.

The following sections organize the most significant challenges by their nature and impact.

Operational and Automation Barriers

Current licensing models create direct friction in day-to-day operations and block automation at multiple points.

Infrastructure as Code integration gap. Licensing is not part of IaC workflows. When infrastructure is provisioned using Terraform, CloudFormation, or similar tools, license activation typically requires separate manual steps outside the automated pipeline. This breaks the reproducibility and repeatability that modern DevOps practices depend on.

Manual setup and management. License assignment, activation, and renewal are frequently manual processes. In environments where hundreds of software instances may be deployed, migrated, or decommissioned across a production cycle, manual licensing is a operational bottleneck that does not scale.

No headless activation path. Many licensing systems require interactive user sessions or graphical interfaces to activate. This blocks CI/CD integration and prevents licensing from being embedded in automated deployment workflows.

No CLI or API access. The absence of programmatic interfaces prevents licensing from being managed as part of scripted, infrastructure-level operations.

Technical and Architectural Constraints

Hardware identity binding. Licenses bound to hostnames, MAC addresses, or other hardware identifiers are incompatible with dynamic infrastructure. Cloud instances, containers, and virtual machines may have no stable identity across restarts, redeployments, or failover events. Blue-green deployment patterns and autoscaling groups break hardware-bound license bindings entirely.

Lack of license mobility. Licenses cannot follow workloads across instances, availability zones, or environments. Disaster recovery scenarios, where traffic must move from a primary to a standby system, require manual ISV intervention rather than automated handover.

Missing standard lifecycle APIs. Most vendors lack a consistent, documented API for the core licensing operations: issue, renew, move, and revoke. Without a standard interface, every integration is proprietary, and multi-vendor deployments require separate licensing processes for each product.

No support for air-gapped or offline environments. Many broadcast operations run in environments with restricted or no external network access. Current licensing models provide no defined behavior for offline operation, leaving organizations exposed to unexpected service interruptions.

Visibility and Cost Challenges

Cost opacity. Organizations frequently cannot differentiate between platform costs and licensing costs, or allocate licensing expenditure to specific projects, productions, or events. Finance teams struggle to forecast, audit, and report on software licensing as a distinct cost category.

Missing usage type differentiation. Current approaches do not distinguish between production, test, training, staging, or disaster recovery usage. Customers are charged at the same rate regardless of the operational context, which discourages the use of non-production environments and limits testing and experimentation.

No centralized management across vendors. Organizations managing software from multiple vendors have no unified view of their license inventory, utilization, or upcoming renewals. License management is fragmented across vendor portals, spreadsheets, and support processes.

Security and Compliance Gaps

Licensing mechanisms are not tamper-proof or auditable. Many existing licensing implementations provide no cryptographic guarantees, no audit trail, and no way to verify that a license has not been transferred, duplicated, or otherwise misused. In regulated broadcast environments, this represents a compliance and governance risk.

Vendors require access to customer infrastructure. Some licensing models require the vendor's systems to reach into the customer's environment for validation, creating security exposure that conflicts with zero-trust architectures and internal compliance requirements.

Undefined behavior during outages. When a license server is unreachable, the behavior of the licensed application is often undefined or undocumented. Organizations cannot plan operational continuity around a licensing model that provides no clear policy for network interruptions.

Ecosystem Fragmentation

The aggregate effect of these challenges is fragmentation. Each vendor implements licensing differently, each activation process is proprietary, and each integration is bespoke. System integrators building multi-vendor solutions must navigate a different licensing model for every product in the stack.

This fragmentation increases complexity, raises support costs, slows deployment, and makes it difficult to realize the operational and financial benefits that software-defined workflows were intended to deliver.

01.03. The DSL Initiative

Origin

A review of IAMT member organizations in the vendor community identified a clear gap: there is no common approach to software licensing within the broadcast and media industry.

Each manufacturer implements licensing differently. Each activation process is proprietary. System integrators building multi-vendor solutions must navigate a different licensing model for every product in the stack, and customers bear the accumulated cost of that fragmentation in operational overhead, audit risk, and deployment friction.

The Dynamic Software Licensing (DSL) initiative was started in response to this finding. Rather than leaving the industry to develop further siloed, vendor-specific solutions in parallel, the initiative brings key stakeholders together to develop a shared framework of best practices.

Purpose

The DSL initiative is a collaborative effort to define how broadcast and media software should be licensed in cloud-enabled, hybrid, and software-defined production environments.

Its output is not a formal technical standard with mandatory compliance obligations. It is a framework of best practices: a set of agreed principles, patterns, and reference approaches that manufacturers, customers, and system integrators can adopt to achieve more consistent, interoperable, and operationally sound licensing outcomes.

The initiative is deliberately implementation-oriented. The goal is practical adoption, not theoretical completeness.

Participants

The DSL initiative operates under the IAMT — the International Association of MediaTech — and involves collaboration across multiple stakeholder groups:

- **Cloud platform providers**, who contributed architectural patterns and reference implementations
- **System integrators**, who provided operational requirements from real-world multi-vendor deployments
- **Independent software vendors (ISVs)**, who contributed requirements from the perspective of licensing system implementors
- **Broadcast customers and media organizations**, who articulated deployment, automation, and operational continuity requirements

This multi-stakeholder composition is intentional. Licensing challenges manifest differently depending on where in the ecosystem an organization sits, and a framework that only reflects the priorities of one constituency will not achieve broad adoption.

Scope of the Initiative

The DSL initiative focuses on the licensing lifecycle for software applications running in broadcast and media production environments.

This includes how licenses are issued, renewed, moved across infrastructure, revoked, and metered. It covers the interfaces between licensing systems and the surrounding operational infrastructure — identity providers, orchestration platforms, infrastructure-as-code tooling, and observability systems.

It does not prescribe vendor pricing models, commercial terms, or customer billing arrangements. Those remain entirely within the scope of individual vendor-customer relationships.

Why Collaboration Is the Right Approach

Individual vendors solving this problem in isolation will produce more fragmentation, not less.

A system integrator deploying ten different broadcast applications from ten different vendors cannot realistically maintain ten incompatible licensing integration paths in a single automated pipeline. A broadcaster operating a hybrid environment cannot govern licensing consistently if each product responds differently to failover events, network interruptions, or infrastructure changes.

Common best practices reduce the integration surface, lower the operational burden on all parties, and give the entire ecosystem a shared foundation to build on. The value of the framework increases as more of the industry adopts it.

IAMT members are driving this initiative, and its success depends on continued industry collaboration and a shared commitment to developing and accepting a

common approach to licensing.

01.04. Objectives and Vision

What These Best Practices Are Intended to Achieve

The DSL best practices have a single overarching objective: to provide the broadcast and media industry with a shared, implementation-oriented framework for software licensing in modern production environments.

This means defining how licenses should be issued, renewed, moved, revoked, and metered in ways that are consistent across vendors, compatible with automated infrastructure, and operationally reliable in cloud, hybrid, and on-premises deployments.

The framework is not a formal standard with mandatory compliance obligations. Its value comes from voluntary adoption: the more of the industry builds toward a common model, the lower the integration burden for everyone — customers, system integrators, and software vendors alike.

The Target Vision

The target vision is a licensing model that follows the same principles as the infrastructure it governs.

Modern broadcast production infrastructure is elastic, automated, and managed as code. Licensing must be the same. A license should be as easy to issue as a cloud instance, as easy to revoke as a terminated container, and as auditable as any other infrastructure event. It should integrate into the same pipelines, respond to the same automation, and behave predictably in the same failure scenarios.

Licensing must shift from being a separate, manual, vendor-specific function — often bolted on after the fact — to being an integrated, automated, and observable component of the production environment.

This shift is not optional. As production workflows become more dynamic, any licensing mechanism that cannot keep pace becomes an operational constraint that limits the value of the infrastructure investment it sits on.

Core Principles

The DSL framework is built on five principles that apply across all aspects of the architecture, API model, and operational guidance.

API-First All licensing functions — issue, renew, move, revoke, inspect, and usage recording — must be fully accessible through open, documented interfaces.

This is a prerequisite for automation, integration with orchestration platforms, and interoperability across vendors.

An API-first approach is not only a technical requirement. It is a statement of intent: licensing should be a programmable service, not a manual process.

Security by Design Licenses are control mechanisms. Every license token must be cryptographically signed and tamper-proof. Every licensing operation must be auditable. Access must be governed by least-privilege principles, and license binding must be based on workload identity rather than hardware identifiers.

Security is not an add-on. It is a structural requirement of any licensing model operating in distributed, multi-tenant, or regulated environments.

Observability Both manufacturers and customers require complete transparency into license usage, active allocations, consumption trends, and costs. Observability is what makes usage-based and flexible licensing models trustworthy: without it, neither party can verify that billing and entitlement are aligned.

Platform Independence The framework must function across cloud, hybrid, and on-premises environments without requiring platform-specific implementations. Organizations should not need to choose a licensing model based on their infrastructure provider.

Licensing as Code Licenses should be managed with the same tools and practices as infrastructure. A license should be declarable, versionable, and provisionable through infrastructure-as-code tooling. This is how licensing becomes a first-class participant in CI/CD pipelines rather than an external dependency that breaks automation.

What a Successful Outcome Looks Like

The DSL best practices will have achieved their purpose when:

- a system integrator can deploy a multi-vendor broadcast solution through a single automated pipeline, with licensing handled as part of the deployment rather than as a separate manual step,
- a broadcaster can move a workload from one environment to another — including in a failover scenario — without manual intervention from any vendor,
- a finance team can attribute licensing costs accurately to productions, events, or projects, with full visibility into consumption,
- a software vendor can implement a conformant licensing interface once and have it work consistently across the range of customer environments and deployment patterns in the industry,

- and the licensing layer is invisible to end users — handled automatically, reliably, and without friction.

02. Scope

Purpose

This chapter defines the scope boundaries of the Dynamic Software Licensing best practices.

It clarifies what the best practices covers, what it intentionally excludes, and who it is written for. Its role is to keep the document focused and to help contributors decide whether a topic belongs in the main best practices, in an annex, or outside the best practices altogether.

Scope of the DSL Best Practices

The Dynamic Software Licensing best practices address modern software licensing approaches for broadcast and media workflows.

The main focus is on reusable and transferable concepts that can support industry alignment across different vendors, deployment models, and operational environments.

In Scope

Topics that are expected to be in scope include:

- software licensing approaches for broadcast and media workflows,
- software-based, hybrid, and software-defined licensing concepts,
- entitlement, lease, renewal, move, revoke, and usage-recording concepts,
- API-oriented licensing interactions,
- workload identity-related binding concepts,
- metering, usage tracking, and policy handling,
- observability, auditability, and operational transparency,
- migration from legacy licensing approaches toward more modern target models.

Out of Scope

Topics that are expected to remain out of scope include:

- the media services themselves,
- detailed ERP, CRM, or finance and billing system integration models,
- legal contract wording,
- commercial price lists,
- vendor-specific product packaging models,
- formal certification procedures,
- platform-specific implementation detail unless explicitly included as informative reference material.

These topics may be referenced where useful, but they are not the primary subject of the best practices.

Intended Audience

This best-practices document is intended for:

- manufacturers / ISVs,
- broadcasters and media organizations,
- system integrators,
- architects and platform designers,
- operations and engineering teams,
- finance and procurement stakeholders,
- security and compliance stakeholders.

Main Document Boundary

The main best practices remain focused on platform-agnostic concepts wherever possible.

Where material becomes:

- platform-specific,
- deployment-specific,
- vendor-specific,
- example-driven rather than generally applicable,

it should normally be placed in an annex rather than the core best practices.

Role of This Chapter

This chapter helps contributors answer the following questions:

- Does this topic belong in the main DSL best practices?
- Should this content be moved into a normative annex?
- Should this content be moved into an informative annex?
- Is this topic outside the intended scope of the best practices?

Subchapters

This chapter is divided into three subchapters:

- 02.01. In Scope — defines the topics and areas that the best practices address, including the core licensing operations, API model, identity binding, metering, and migration guidance.
- 02.02. Out of Scope — defines what is explicitly excluded, including existing proprietary licensing models, ERP integration, legal contract wording, and vendor-specific product packaging.
- 02.03. Intended Audience — describes the stakeholder groups this document is written for, from software vendors and system integrators to operations teams and finance stakeholders.

02.01. In Scope

What These Best Practices Cover

The Dynamic Software Licensing best practices are intended to define a coherent, implementable framework for software licensing in broadcast and media environments. The scope is deliberately focused: the document addresses the licensing layer itself, not the products or workflows that the licensing layer governs.

The following areas are within the scope of these best practices.

Software Licensing Approaches for Broadcast and Media Workflows

The primary focus is on how software licenses are managed in the context of broadcast, media production, and related media technology workflows. This includes both traditional broadcast environments and modern cloud-based or hybrid production platforms.

The best practices address how licenses should behave in these environments — not how the media workflows themselves should be built.

Software-Based, Hybrid, and Software-Defined Licensing Concepts

The best practices apply to licensing models that are suited to software-defined, containerized, and cloud-native infrastructure. This includes fully software-based deployments, hybrid models that combine on-premises and cloud resources, and environments where workloads move dynamically between infrastructure tiers.

Hardware-based licensing mechanisms are not the target model, though the best practices acknowledge that migration from hardware-bound approaches is a relevant transition scenario.

Core Licensing Operations

The following licensing operations are in scope and are defined normatively in this document:

- **Entitlement:** the right to use software, as granted by the software publisher to the customer organization.
- **Lease:** the active, time-bounded allocation of an entitlement to a specific workload or deployment context.
- **Renewal:** the extension of an active lease before it expires.
- **Move:** the transfer of an active lease from one workload or environment to another.
- **Revoke:** the explicit termination of an active lease.
- **Usage recording:** the capture of metered consumption data for billing, auditing, or reporting purposes.

These operations form the operational vocabulary of the DSL framework. All normative content in later chapters is grounded in these concepts.

API-Oriented Licensing Interactions

The best practices define how licensing functions are exposed and consumed through APIs. This includes the structure of licensing requests, the format of license tokens, the behavior of licensing endpoints, and the patterns for integration with orchestration and automation tooling.

API-oriented interaction is a foundational requirement of the DSL framework, not an optional feature.

Workload Identity-Based Binding

License binding based on workload identity is in scope. This means that licenses are associated with verifiable software or infrastructure identities rather than with hardware properties such as MAC addresses, hostnames, or hardware dongles.

The best practices define the expectations for identity-based binding, including the requirements that apply to the identity tokens used in licensing requests.

Metering, Usage Tracking, and Policy Handling

In scope are the mechanism and data models for metering software usage, tracking active license allocations, and enforcing license policies. This includes both real-time enforcement and post-hoc reporting.

Usage-based and consumption-based licensing models are explicitly in scope and are treated as first-class patterns alongside traditional entitlement-based models.

Observability, Auditability, and Operational Transparency

The best practices define requirements for visibility into the licensing layer. This includes the ability to inspect active leases, monitor consumption against entitlements, produce audit trails of licensing events, and provide cost attribution data to operators and finance stakeholders.

Observability is treated as a structural requirement, not as a monitoring add-on.

Migration from Legacy Licensing Approaches

Guidance on migration from existing hardware-bound or manual licensing mechanisms toward the DSL target model is in scope. This is addressed as a practical concern: many organizations will need to operate both models simultaneously during a transition period, and the best practices should support that reality without endorsing the continuation of legacy approaches.

Migration guidance is primarily informative. It does not constitute a normative endorsement of legacy licensing patterns.

02.02. Out of Scope

What These Best Practices Do Not Cover

The following areas are explicitly outside the scope of the DSL best practices. They may be referenced where useful for context, but they are not the subject of normative guidance in this document.

Maintaining clear out-of-scope boundaries is important for the integrity of the best practices. Without them, the document risks becoming a general-purpose integration guide rather than a focused framework for the licensing layer itself.

A New Licensing Model, Not a Wrapper Around Existing Ones

The DSL best practices define a new approach to software licensing. They do not describe how to integrate, extend, or federate existing proprietary licensing systems.

This distinction is fundamental. The goal is not to build a compatibility layer on top of current vendor-specific licensing mechanisms. It is to define what a modern, interoperable licensing model should look like when built from the ground up for software-defined and cloud-native environments.

Organizations implementing the DSL framework are expected to adopt the target model. Migration guidance is provided to support the transition from legacy approaches, but the legacy approaches themselves are not in scope as reference models or design inputs.

The Media Functions Themselves

The best practices address the licensing layer. The software products and media functions (sometimes referred to as media services in deployment documentation) that consume licenses — transcoders, graphics engines, ingest systems, quality control tools, playout systems, and similar applications — are considered only as consumers of the licensing API. Their internal design, configuration, and operation are outside the scope of this document.

ERP, CRM, and Financial System Integration

The DSL framework defines how licensing data can be made available for billing and cost attribution purposes. It does not define how that data integrates with customer or vendor ERP systems, CRM platforms, or financial management tools.

These integrations are organization-specific and commercially sensitive. They fall outside the boundary of a vendor-neutral best practices document.

Legal Contract Wording

The best practices do not contain or define legal license agreements, terms of use, or contractual language. The framework operates at the technical and operational level. Commercial and legal terms remain the responsibility of the contracting parties.

Commercial Price Lists and Licensing Economics

Pricing models, rate cards, and commercial licensing structures are outside the scope of this document. The best practices define the mechanisms by which usage is tracked and entitlements are enforced. How that usage is priced is a commercial decision, not a technical one.

Vendor-Specific Product Packaging

The DSL framework is designed to be vendor-neutral. It does not define how any specific vendor should package, version, or bundle their software products for licensing purposes. Product packaging remains a vendor-internal concern.

Formal Certification Procedures

The best practices do not establish a formal certification scheme, test suite, or accreditation process. Conformance guidance is provided to help implementors understand what the framework requires, but this is not a certification program.

Platform-Specific Implementation Detail

Where implementation detail is necessary to illustrate a concept, it is provided in informative annexes rather than in the main best practices. The core document is intentionally platform-agnostic and implementation-neutral.

AWS-specific, Azure-specific, GCP-specific, or other cloud-provider-specific implementation patterns belong in informative annexes. They do not belong in the normative body of the best practices.

Likewise, on-premises and hybrid deployment examples may be used for illustration, but they must not alter or narrow the normative requirements of the platform-agnostic DSL model.

02.03. Intended Audience

Who This Document Is Written For

The DSL best practices are written for a broad technical and operational audience across the broadcast and media technology industry. Different parts of the document will be more directly relevant to some groups than others, but the

framework as a whole is intended to be readable and useful across the following stakeholder groups.

The audience framing in this section is intentionally aligned with the stakeholder groups and persona lenses formalized later in Section 05.01 and Section 05.02. This keeps scope language and role expectations consistent across the document.

Manufacturers and Independent Software Vendors

Software vendors who develop products used in broadcast and media production environments are a primary audience. For manufacturers, the best practices define what a conformant licensing interface looks like, what operations it must support, and how it should integrate with customer infrastructure.

Vendors who implement the DSL framework can expect their licensing interfaces to work consistently across the range of customer environments that have adopted the same framework — reducing the per-customer integration burden significantly.

Broadcasters and Media Organizations

Operators who license software from multiple vendors are a central beneficiary of the DSL framework. For broadcasters and media organizations, the best practices define what they should be able to expect from a conformant licensing implementation: automation support, observability, portability, and operational predictability.

This audience includes both technical operations teams and the procurement and finance stakeholders who need visibility into software costs and consumption.

System Integrators

System integrators who design and deploy multi-vendor production environments need a shared foundation for how licensing works across the systems they build. The DSL best practices provide that foundation. Integrators can use the framework to define consistent integration patterns, avoid vendor-specific workarounds, and deliver solutions where the licensing layer does not become a bottleneck.

Architects and Platform Designers

Platform architects who design cloud-based, hybrid, or on-premises production infrastructure need to understand how licensing fits into the infrastructure model. The best practices define licensing as a platform component — one that must be designed for, not bolted on. This audience will find the reference model, API contracts, and conformance guidance most directly relevant.

Operations and Engineering Teams

Engineers responsible for deploying, operating, and maintaining production environments are a practical audience for much of the normative content. Concepts such as lease lifecycle management, workload identity binding, usage recording, and observability are directly applicable to day-to-day operational work.

Finance and Procurement Stakeholders

Finance and procurement teams who manage software vendor relationships and operational cost attribution benefit from the visibility that a conformant DSL implementation provides. The observability and usage-recording requirements in the best practices are designed to produce the data these teams need — cost attribution by workload, event, or project; consumption trend visibility; and alignment between contracted entitlements and actual usage.

Security and Compliance Stakeholders

Security architects and compliance teams have a direct interest in how licenses are bound, how access to licensing APIs is controlled, and how audit trails are produced. The best practices treat security as a structural requirement rather than an add-on, and the trust and security framework chapters are written with this audience in mind.

03. Conventions and Normative Language

Purpose

This chapter defines the writing, interpretation, and classification conventions used throughout the Dynamic Software Licensing best practices.

Its purpose is to ensure that contributors and readers understand how the document is read, how different kinds of statements are interpreted, and how normative and informative material are separated.

Role Within the Best Practices

The Dynamic Software Licensing best practices are not a formal standard. However, they are intentionally structured in a standard-like way.

This means the document uses clear and disciplined language, especially where implementation guidance, conformance expectations, or interpretation rules are involved.

This chapter provides the common conventions that support that approach.

Core Topics

This chapter establishes three sets of conventions for the repository:

- normative keywords and statement strength,
- classification of normative, informative, and explanatory content,
- platform-agnostic boundaries for the core document and annex material.

Role of This Chapter

This chapter defines:

- how strong a statement should be,
- whether a passage is normative or informative,
- whether a statement belongs in the main best practices or in an annex,
- when controlled vocabulary must be used,
- how to keep the tone neutral and standard-like.

Subchapters

This chapter is divided into three subchapters:

- 03.01. Normative Keywords and Statement Strength — defines how terms such as **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are used and how statement strength is interpreted.
- 03.02. Content Classification — defines the distinction between normative, informative, and explanatory material and how mixed passages are handled.

- 03.03. Platform-Agnostic Boundaries and Annex Use — defines what belongs in the core document, what belongs in annexes, and how platform-specific material is contained.

Chapter Positioning

This chapter provides the interpretation rules that make the rest of the published baseline readable and reviewable in a consistent way.

It serves as the reference point for distinguishing informative text from normative intent, for controlling keyword usage, and for deciding when material belongs in the main document versus an annex.

03.01. Normative Keywords and Statement Strength

Purpose

This section defines how statement strength is expressed in the Dynamic Software Licensing best practices and how readers interpret normative keywords.

The objective is not to imitate a formal standard mechanically, but to apply disciplined wording where interpretation strength matters for interoperability, implementation behavior, or conformance assessment.

Normative Keywords

The core keyword set used in the best practices is:

- **MUST**
- **MUST NOT**
- **SHOULD**
- **SHOULD NOT**
- **MAY**

These keywords are used only where the text needs to distinguish clearly between required, prohibited, strongly recommended, discouraged, and optional behavior.

Interpretation of Statement Strength

The keywords are interpreted as follows:

- **MUST** expresses a required behavior, constraint, or interpretation rule that is necessary for consistent application of the best practices.
- **MUST NOT** expresses a prohibited behavior or disallowed interpretation that would violate the best-practices model if performed.
- **SHOULD** expresses a strong recommendation. Alternative approaches are possible, but a departure should be intentional and justified.

- **SHOULD NOT** expresses a strong recommendation against a behavior. Departures may exist, but they should be exceptional and intentionally justified.
- **MAY** expresses an allowed option or acceptable variation that remains compatible with the overall model.

Where no normative keyword is used, the text is normally read as descriptive, interpretive, or explanatory rather than as a testable requirement.

When Normative Keywords Belong

Normative keywords belong where the text defines:

- interoperability expectations,
- trust and security behavior,
- lifecycle or state handling constraints,
- API or data model interpretation rules,
- conformance-relevant implementation behavior.

They are generally avoided in background, motivation, or purely illustrative passages.

Use Discipline

Normative keywords are only useful when they remain selective and meaningful. Overuse weakens interpretation and makes review harder.

For that reason:

- introductory sections normally avoid normative wording,
- explanatory examples do not introduce accidental requirements,
- informative annexes contain normative terms only where they restate or clarify rules already grounded in the main document or normative annexes,
- mixed passages make it clear which sentence carries the requirement and which sentence only explains it.

Relationship to Conformance

Not every normative sentence has the same practical weight, but normative keywords form the main basis for conformance interpretation. They help identify which expectations are central to profile alignment, implementation review, and cross-vendor consistency.

Section Positioning

This section provides the baseline interpretation model for requirement strength across the full repository. It is the reference point whenever wording is assessed

for precision, consistency, or normative effect.

03.02. Content Classification

Purpose

This section defines how repository content is classified so that readers can distinguish between requirements, explanation, and supporting context.

Clear classification matters because the best practices combines architecture, requirements, examples, profiles, annexes, and migration guidance in one coherent body of material.

Content Classes

The document uses three practical content classes:

- normative content,
- informative content,
- explanatory or editorial support content.

Normative Content

Normative content defines behavior, constraints, obligations, or interpretation rules that are important for consistent implementation or assessment.

Normative content is usually found in:

- the normative requirements chapter,
- normative annexes,
- conformance interpretation passages,
- selected parts of security, API, and data model chapters where disciplined interpretation is necessary.

Normative content is written so that readers can identify what is being expected and why that expectation matters.

Informative Content

Informative content explains the model, gives context, illustrates choices, or helps readers apply the best practices correctly.

Informative content may include:

- introduction and background material,
- explanatory architecture text,
- operational profiles,
- migration discussion,
- examples and mappings,

- informative annexes.

Informative text may strongly influence understanding, but it is not mistaken for a requirement merely because it is useful or persuasive.

Explanatory and Editorial Support Content

Some passages exist to improve readability, document structure, or local interpretation. These include summaries, transition sentences, rationale, and limited editorial framing.

Such content is acceptable when it helps readers navigate the material, but it does not replace substantive content and does not blur the distinction between guidance and requirement language.

Handling Mixed Passages

Some sections contain a mix of requirement and explanation. In those cases:

- the requirement sentence is clearly identifiable,
- the explanatory sentence clarifies the rule rather than broadening it implicitly,
- examples remain recognizably illustrative,
- the structure avoids hiding the normative meaning inside long narrative prose.

If a reader cannot tell which part of a passage is normative, the wording needs improvement.

Relationship to Chapter Types

Different chapters have different dominant content classes:

- Chapters 01, 02, 05, 06, 07, 13, 14, and 15 are primarily informative or interpretive.
- Chapter 09 is the primary home of normative requirements.
- Chapters 08, 10, 11, and 12 may contain a mix of interpretive and normative material, depending on the subject and the precision required.
- Normative annexes support disciplined interpretation and may restate or summarize normative material where that improves consistency.

Section Positioning

This section is the classification guide for reading and reviewing the repository. It is used whenever a passage needs to be evaluated for whether it states a requirement, explains a requirement, or only provides supporting context.

03.03. Platform-Agnostic Boundaries and Annex Use

Purpose

This section defines how the core best-practices document remains platform-agnostic while still allowing concrete examples, deployment references, and implementation-specific material to be included in a controlled way.

Core Document Boundary

The main body of the best practices is written to be reusable across vendors, deployment models, and operational environments. Its primary role is to define the conceptual model, requirement structure, security posture, API interpretation, and shared data semantics.

For that reason, the core document avoids becoming:

- vendor-specific,
- cloud-provider-specific,
- tied to one product packaging model,
- overloaded with examples that narrow the interpretation of the general model.

When Material Belongs in the Core Text

Material belongs in the core document when it:

- defines reusable concepts,
- establishes interpretation rules,
- supports cross-vendor alignment,
- expresses requirements or conformance-relevant guidance,
- clarifies common lifecycle, trust, or policy behavior.

When Material Belongs in an Annex

Material belongs in an annex when it is primarily:

- platform-specific,
- deployment-profile-specific,
- example-oriented,
- mapping-oriented,
- explanatory support that would interrupt the flow of the core text.

Normative annexes may be used when consistent interpretation needs structured supporting material. Informative annexes are used for examples, reference architectures, walkthroughs, mappings, and FAQ-style support.

Handling Platform-Specific References

Platform-specific references are allowed where they improve clarity, but they are contained deliberately.

In practice this means:

- the main text describes the generic rule first,
- platform examples are separated clearly from the generic rule,
- examples do not redefine the general model,
- annexes carry the main burden of technology-specific elaboration.

The AWS reference architecture annex is an example of this separation: it interprets the model for one concrete platform without redefining the platform-neutral baseline.

Annex Selection Logic

The annex boundary is applied consistently:

- if the material is required for consistent interpretation across implementations, consider a normative annex,
- if the material is illustrative, comparative, or example-driven, use an informative annex,
- if the material is essential to understanding the model itself, keep it in the core document.

This prevents the main document from becoming either too abstract to implement or too specific to generalize.

Section Positioning

This section defines the boundary discipline that keeps the best practices reusable and maintainable. It is used when deciding whether new content belongs in a chapter of the main document or in a supporting annex.

04. Terminology and Abbreviations

Purpose

This chapter defines the controlled vocabulary used throughout the Dynamic Software Licensing best practices.

Its purpose is to ensure that key concepts are described consistently across the repository and that contributors, reviewers, and readers share a common understanding of the main terms used by the DSL initiative.

Role Within the Best Practices

Terminology is a foundational part of the Dynamic Software Licensing best practices.

A consistent vocabulary is necessary to:

- avoid ambiguity,
- reduce duplicate concepts,
- prevent the use of multiple labels for the same idea,
- support clearer architecture, API, conformance, and requirement chapters,
- make the best practices easier to review and evolve over time.

This chapter should therefore be treated as a reference point for the rest of the document.

Scope of This Chapter

This chapter covers:

- core terms,
- architecture-related terms,
- license and policy terms,
- operational terms,
- abbreviations used throughout the best practices.

It establishes the working vocabulary needed to keep the rest of the best practices coherent without attempting to catalog every implementation-specific detail.

Controlled Vocabulary Principle

Where a term is defined in this chapter, contributors should use that term consistently throughout the repository.

Contributors should avoid:

- introducing synonyms for established concepts,
- renaming concepts locally within a chapter,
- using platform-specific wording as the default term,
- mixing conceptual terms and implementation labels without explanation.

If a term needs to be changed, the change should be made deliberately and consistently across all affected chapters and annexes.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Core Terms Defines the most central concepts used across the best practices.

02. Architecture Terms Defines architecture-related vocabulary used in the reference model and related sections.

03. License and Policy Terms Defines terms related to licensing behavior, entitlement, lease handling, policy handling, and related concepts.

04. Operational Terms Defines terms related to operation, lifecycle handling, observability, migration, and deployment behavior.

05. Abbreviations Defines recurring abbreviations used throughout the best practices.

Controlled Vocabulary Maintenance

This chapter should be treated as the repository's controlled vocabulary baseline for the published best practices.

If review feedback requires term changes, the update should be applied consistently across the main document, relevant annexes, and any affected schema or API descriptions.

Relationship to Other Chapters

This chapter supports and informs:

- the scope chapter,
- the design principles chapter,
- the reference model,
- conformance and profiles,
- normative requirements,
- canonical API and data models,
- security and operational chapters,
- relevant annexes and schemas.

Where other chapters introduce concept names, those names should align with this terminology chapter.

Chapter Positioning

This chapter acts as the vocabulary anchor for the full published baseline and should be consulted whenever wording, concept naming, or cross-chapter consistency is reviewed.

Its role is to reduce ambiguity and prevent multiple labels for the same concept across the document set.

04.01. Core Terms

Purpose

This section defines the highest-level conceptual terms of the Dynamic Software Licensing (DSL) framework. These core terms span across technical, commercial, and operational domains and appear throughout the best practices document.

Scope Statement

This section:

- identifies foundational concepts that are not limited to a single technical area,
- establishes a common vocabulary to avoid introducing parallel labels or synonyms,
- anchors downstream architecture and requirement chapters.

It does not define low-level component schemas or platform-specific data elements.

Definitions

Application (Software Application) - An overarching structural term representing a complete end-user or system-level software product whose operational boundary extends beyond discrete signal or file manipulation. An application may embed, orchestrate, or manage one or more media functions.

Auditability - The structural quality that ensures all license lifecycle events, state transformations, and usage metrics are captured in a tamper-proof, chronologically sequential ledger to satisfy regulatory and corporate compliance.

Commercial Modality - The overarching business strategy and contract rules (such as subscription, pool allocation, pre-paid tokens, or post-hoc utility consumption) that govern how structural entitlements are priced and accounted for over time. This term should be used for business framing, not as a replacement for the runtime-facing term License Modality.

Dynamic Software Licensing (DSL) - The core collaborative initiative and architectural framework focused on modern software licensing approaches for broadcast and media workloads.

Ecosystem Fragmentation - The operational friction, administrative overhead, and multi-vendor incompatibility caused by disparate, proprietary, and isolated legacy licensing solutions implemented across a single production pipeline.

Identity-Based Binding - A trust mechanism that binds an active license lease to a cryptographically verifiable software or platform identity token rather than legacy, static hardware properties.

Licensing as Code - The architectural philosophy of managing, declaring, and provisioning license states using the exact same automated configuration tools (e.g., IaC pipelines) and version-control workflows as the underlying runtime infrastructure.

Media Functions - Following DMF Reference Architecture, the Media Functions layer provides well-defined units of functionality that act on media streams, files or objects.

Media Workloads - Following DMF Reference Architecture, Media Workloads combines multiple standalone, stateless “Media Functions” (such as video/audio mixing, format conversion, and graphics rendering) to execute a targeted production requirement.

Within DSL usage, Application is the broader product-level term, Media Functions refers to the runtime components that process media streams or objects, and Media Workloads refers to the collective chain or topology in which multiple media functions operate together.

Modality - Informal shorthand for a licensing usage model. In normative, API, schema, and runtime contexts, the preferred term is License Modality to avoid ambiguity between business framing and runtime enforcement behavior.

04.02. Architecture Terms

Purpose

This section defines the architectural layers, system planes, and logical structural components that make up the proposed reference model.

Scope Statement

This section covers the structural definitions within the licensing ecosystem’s functional boundaries. It explicitly focuses on platform-agnostic roles and components rather than cloud-provider-specific implementation details.

Definitions

Control Plane - The control plane is the central authority for entitlements, lease lifecycle management, cryptographic key operations, usage metering, and policy enforcement.

End User Portal - A customer-facing management interface providing aggregated visibility, multi-vendor asset views, self-service provisioning, and centralized administrative control over license allocation.

License Agent (Broker) - A functional component within the workspace plane that acts as an intermediary interface between applications and the Control Plane to coordinate requests such as lease renewal, usage recording, revocation handling, and local policy enforcement.

Platform Neutrality - The design and architectural attribute of the Licensing SDK that enables it to execute uniformly, predictably, and without modification across diverse operating systems, hardware architectures, and runtime environments.

Workspace Plane - The workspace plane is where licensed applications run and where licensing is enforced locally.

04.03. License and Policy Terms

Purpose

This section establishes consistent vocabulary for behavior rules, license management mechanics, data models, and enforcement policies across runtime environments.

Scope Statement

This section supports terminological alignment for the requirement, API, and data model chapters. It explicitly excludes legal contract wording, vendor-specific pricing models, and licensing economics.

Definitions

Entitlement - The legal and commercial right to use a software application, module, or specific feature set, as granted by a software publisher (ISV) to a customer organization.

Grace Period - A deterministic, policy-defined temporal safety envelope that permits a media function to bypass strict enforcement and continue normal operations during transient connectivity loss to the licensing authority.

Lease - An active, cryptographically signed, and time-bounded allocation of a specific entitlement to a designated executing media function or deployment context.

License Modality - The distinct commercial and operational framework (e.g., time-based, process-based, or outcome-based) that dictates how software usage metrics are mapped to commercial terms and entitlements.

Time-Based License Modality - A lease model governed by start time, end time, or continuous duration (for example hourly, daily, monthly, or annual use). Within DSL, use this term when the primary enforcement dimension is time or duration.

Process-Based License Modality - A lease model governed by throughput or capacity units, such as channel count, FPS, sessions, or bandwidth. Within DSL, use this term when the primary enforcement dimensions are continuous operational counters rather than elapsed time.

Outcome-Based License Modality - A consumption model governed by discrete outputs or completed results, such as generated clips, rendered frames, transcode jobs, or analysis reports. Within DSL, use this term when value is tied directly to produced outcomes that can be counted or attributed.

Credit or Unit-Pool License Modality - A model in which runtime usage depletes a pre-purchased or pre-allocated pool of generic licensing units. Within DSL, use this term when entitlements are managed as shared pools at tenant or organizational level, and distinguish explicitly between depleting-pool and floating-pool behavior where relevant.

Perpetual License - An entitlement model that grants a customer organization the indefinite, non-expiring legal right to use a specific software version or feature set.

Policy Object - A structured data envelope containing the control settings and configuration constraints governing a lease's behavior, including renewal windows, grace intervals, and alerting thresholds.

04.04. Operational Terms

Purpose

This section defines lifecycle transitions, operational workflows, observability mechanisms, and migration tracking vocabulary used in practical deployment scenarios.

Scope Statement

This section aligns the abstract conceptual framework with real-world production environments and deployment runbooks. It centers on target-state capabilities

and transition states without endorsing the permanent use of legacy approaches.

Definitions

Air-Gapped - A strict deployment context running within restricted networks entirely separated from the open internet, requiring special local trust validation boundaries and offline sync behavior.

Failover Auto-Mobility - The ability of an active license lease to move seamlessly from a failing primary instance to a secondary high-availability (HA) or disaster recovery (DR) instance through automation, completely eliminating manual ISV support intervention.

Headless Activation - A completely programmatic, scriptable path to request, bind, and activate a license lease without needing interactive user sessions, graphical interfaces, or manual input.

Lifecycle Operations - The complete programmatic sequence of runtime states, transition paths, and compliance validations that a license entitlement and its associated active leases navigate through from initial activation to eventual expiration or retirement.

Observability - The structural capability providing real-time transparency, audit logging, and diagnostics regarding active lease allocations, consumption trends, and cost distribution.

Parallel Operation (Pilot Phase) - A transitional migration phase where a customer securely deploys and runs legacy licensing mechanisms side by side with the modern DSL target framework to de-risk technical transformation.

04.05. Abbreviations

Purpose

This section defines the recurring abbreviations used throughout the Dynamic Software Licensing (DSL) best practices repository.

Scope Statement

This section provides a concise reference for abbreviated terms that cross technical, operational, and commercial boundaries. These abbreviations should be interpreted consistently across all chapters and annexes.

Definitions

Abbreviation	Full Term	Definition / Context within DSL
CapEx	Capital Expenditure	Traditional, static financial models where hardware and perpetual software licenses are bought upfront as long-lived assets.
DR	Disaster Recovery	High-availability failover configurations requiring rapid, automated license mobility without manual vendor intervention.
DSL	Dynamic Software Licensing	The core initiative and architectural framework defined by this document.
HA	High Availability	Architectural designs optimized for continuous production uptime, necessitating resilient, non-blocking license validation.
IaC	Infrastructure as Code	The practice of provisioning and managing infrastructure using configuration files (e.g., Terraform), into which DSL must natively integrate.
ISV	Independent Software Vendor	Software manufacturers and publishers who develop, license, and support broadcast and media products.
JWKS	JSON Web Key Set	A cryptographic standard contract used by the Workspace Plane to discover public keys and validate signed license tokens.

Abbreviation	Full Term	Definition / Context within DSL
OpEx	Operational Expenditure	Pay-as-you-go financial models matching consumption metrics, flexible workflows, and dynamic software-defined deployments.
SI	System Integrator	Third-party organizations responsible for designing, deploying, and operationalizing multi-vendor architectures for media clients.
SRE	Site Reliability Engineer	The platform-level operational persona focused on runtime uptime, failure-domain containment, and system recovery.

05. Stakeholders, Use Cases and Requirements

Purpose

This chapter defines the stakeholder landscape, the principal operational use case classes, and the requirement themes derived from them. It establishes a traceable statement of need for the best practices and provides the rationale against which later normative choices can be evaluated.

Role Within the Best Practices

This chapter provides the bridge between:

- who is affected,
- what situations they operate in,
- what problems they face,
- what capabilities the best practices needs to support.

Scope of This Chapter

This chapter covers:

- stakeholder groups,
- personas and perspectives,
- recurring use case classes,
- workflow definitions derived from those scenarios,
- requirement themes derived from those perspectives, workflows, and scenarios.

It does not define normative requirements. It explains why those requirements exist and provides context for later normative chapters.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Stakeholder Groups Defines the stakeholder categories addressed by the best practices: customer organizations, ISVs, system integrators, industry bodies, and platform providers.

02. Personas and Perspectives Captures the operational, technical, and commercial concerns of specific roles: engineering leadership, finance, operations, security, procurement, ISVs, and SI developers.

03. Use Case Classes Defines recurring operational patterns: 24/7 operations, frequent and infrequent event-driven production, test and validation environments, and AI-powered or outcome-based service scenarios.

04. Derived Requirement Themes Summarises the recurring requirement patterns that emerge from the stakeholder and use case analysis: automation, portability, transparency, security, observability, self-service, usage-based metering, and continuity under failure.

05. Workflow Definitions Defines canonical workflow patterns derived from the challenge and use-case analysis.

Relationship to Other Chapters

This chapter informs:

- the design principles chapter,
- the reference model,
- conformance and profiles,
- normative requirements,
- operational and migration chapters.

It provides the rationale for many of the later design and requirement decisions.

In particular, this chapter is the justification layer for later statements on:

- control-plane and workspace-plane responsibilities,
- lifecycle operations for issue, renew, reissue, revoke, and inspect,
- entitlement and lease handling,
- observability and audit expectations,
- license modality flexibility and profile differentiation.

Chapter Positioning

This chapter provides the needs, stakeholder, and workflow rationale behind the later architectural and normative choices.

It is used during review to test whether later chapters remain traceable back to real stakeholder pressures, recurring use cases, and explicit workflow patterns.

This chapter is descriptive and analytical in nature. Later chapters may derive normative statements from it, but this chapter itself primarily establishes scope, relevance, and traceability.

05.01. Stakeholder Groups

Purpose

This section defines the stakeholder categories addressed by these best practices. It establishes the stakeholder taxonomy used by Sections 5.2 through 5.5 and provides the basis for traceable requirement derivation in later chapters.

Stakeholder Model

The Dynamic Software Licensing (DSL) landscape comprises five primary stakeholder groups:

1. Customer Organizations
2. Independent Software Vendors (ISVs)
3. System Integrators (SIs)
4. Platform and Infrastructure Providers
5. Industry Bodies and Governance Participants

These groups are distinguished by decision authority, operating responsibility, and lifecycle accountability.

In practice, these stakeholder groups do not operate in isolation. Licensing outcomes are produced through coordinated decisions across commercial, technical, and governance boundaries. This is particularly visible in cloud and hybrid environments, where entitlement state, workload identity, deployment automation, and financial controls must remain aligned.

The reference perspective throughout this chapter is a dynamic software licensing model. The text therefore centers on target-state capabilities and operating behavior.

Scope Statement

This section:

- identifies stakeholder groups relevant to DSL adoption and operation,
- describes their primary concerns at category level,
- defines the boundary to persona-level analysis in Section 5.2.

This section does not:

- define role-specific user stories,
- define implementation-specific controls,
- introduce normative requirement statements.

Shared Dynamic Baseline

Across all stakeholder groups, a common dynamic baseline is expected:

- API-accessible lifecycle operations,
- elastic and highly available control functions,
- auditable and cryptographically protected license artifacts,
- operational observability for usage, limits, and forecast,
- automation-ready integration into deployment workflows.

This shared baseline is relevant because later chapter content is intended to remain platform-agnostic while still being implementation-oriented. The stakeholder

model therefore anchors concerns in capability terms rather than vendor-specific product terms.

1. Customer Organizations

Customer organizations operate media, broadcast, and adjacent production environments and consume licensed capabilities across steady-state, event-driven, and validation contexts.

Primary concern areas include:

- predictable and explainable licensing spend,
- timely activation and deactivation of licensed capabilities,
- continuity during failover and recovery scenarios,
- operational transparency for audit and compliance.

Customer organizations typically contain multiple internal roles (for example engineering, operations, finance, security, and procurement). Role-specific perspectives are addressed in Section 5.2.

From a lifecycle perspective, customer organizations are the primary source of demand signals: they define required service levels, resilience expectations, and acceptable commercial envelopes. Their constraints strongly influence what is considered operationally viable in later workflow and requirement sections.

2. Independent Software Vendors (ISVs)

ISVs define, deliver, and support licensable products and features. They balance commercial protection objectives with operational usability expectations.

Primary concern areas include:

- protecting license integrity and preventing misuse,
- supporting multiple commercial modalities (for example annual and usage-based),
- reducing onboarding and deployment friction across dynamic runtime environments,
- scaling support through automation and standard interfaces.

ISVs are also responsible for offering commercially coherent models that can combine time-based commitments, process-based consumption units, and outcome-based value metrics without creating opaque portfolio complexity.

Accordingly, ISVs carry dual accountability: they must preserve commercial integrity while ensuring that license controls remain operable within automated customer environments. This dual accountability is a recurring theme across Sections 5.3 to 5.5.

3. System Integrators (SIs)

SIs implement, integrate, and operate multi-vendor solutions for customer organizations. They translate architectural intent into repeatable delivery and operations patterns.

Primary concern areas include:

- automating licensing in IaC and CI/CD workflows,
- integrating heterogeneous vendor licensing behaviors,
- minimizing manual activation paths and exception handling,
- ensuring operational consistency across environments.

Because SIs typically bridge multiple vendors and customer teams, they are often the first to encounter interoperability gaps. Their perspective is therefore essential for identifying where lifecycle semantics need to be standardized to support repeatable delivery.

4. Platform and Infrastructure Providers

Platform and infrastructure providers supply execution, identity, orchestration, and observability capabilities required by licensed workloads.

Primary concern areas include:

- identity-based workload binding instead of host-bound constraints,
- secure and auditable interactions with licensing control functions,
- resilient operation during scale-out, replacement, and failover,
- interoperability between platform services and licensing workflows.

In many deployments, platform constraints determine what is practically achievable in license lifecycle automation. For this reason, platform and infrastructure perspectives are not merely implementation detail; they are preconditions for feasible design choices.

5. Industry Bodies and Governance Participants

Industry bodies and governance participants align terminology, baseline expectations, and interoperability direction across vendors and adopters.

Primary concern areas include:

- common language for lifecycle operations and entitlements,
- consistent interpretation of portability and compliance requirements,
- reducing fragmentation caused by siloed licensing approaches,
- maintaining traceability from industry pain points to best practice guidance.

Governance participants also provide continuity across drafting cycles by keeping core terminology and intent stable while implementations evolve. This continuity

is necessary to maintain comparability across profiles, conformance levels, and migration paths.

Cross-Group Interaction Summary

The stakeholder groups interact through recurring responsibility boundaries:

- customer organizations define operational and commercial demand,
- ISVs publish licensable capabilities and policy envelopes,
- SIs integrate and operationalize cross-vendor workflows,
- platform providers supply identity and runtime control primitives,
- governance participants sustain interoperability alignment and shared terminology.

Taken together, these interaction boundaries define the context in which use case classes and workflow definitions should be interpreted. They also provide the basis for evaluating whether proposed requirements are complete, balanced, and operationally credible.

Boundary to Section 5.2

This section defines stakeholder categories at group level. Section 5.2 defines role-level perspectives within and across those groups.

Section Positioning

This section is descriptive and analytical. It provides context and traceability input for downstream architecture and requirement chapters and does not itself introduce normative requirement language.

05.02. Personas and Perspectives

Purpose

This section defines role-level perspectives derived from the stakeholder groups in Section 5.1. It captures objectives, operating constraints, and measurable outcomes that inform use case classification and requirement-theme derivation.

The persona set is written against a dynamic software licensing target model. It therefore describes how roles expect the model to behave in operation, sales, delivery, and governance.

Method and Input Basis

The personas below synthesize:

- stakeholder categories from Section 5.1,
- recurring industry challenges,

- team prework (including Miro workshop material).

Personas are not organization charts. They are decision and concern viewpoints used to guide later analysis.

Persona Schema

Each persona is specified using the same structure:

- primary focus,
- licensing perspective,
- success criteria.

This schema is used to keep later mappings to use case classes and requirement themes consistent.

The schema is intentionally compact. It allows reviewers to compare personas on equal terms and identify where concerns overlap, conflict, or remain insufficiently represented. This consistency also improves traceability when persona inputs are transformed into use case classes and workflow decisions.

Persona Set

The persona set is intentionally cross-functional. It reflects both technical and commercial decision authority required for cloud-native licensing adoption.

To avoid customer-only bias, this section explicitly includes supplier-side and integrator-side decision lenses. This is necessary because portability, interoperability, and lifecycle automation outcomes depend on both product-side and deployment-side accountability.

In operational reality, several personas may be represented by one person or one team, especially in smaller organizations. The persona model should therefore be read as a set of decision lenses rather than a rigid staffing model.

Customer Organization Personas

1. Director of Engineering (Customer Organization)

Primary focus:

- architecture resilience,
- delivery speed,
- operational risk reduction.

Licensing perspective:

- requires licensing behavior compatible with high-availability architectures,
- expects controlled scaling without manual relicensing bottlenecks,
- requires predictable spend boundaries for planning and approvals.

Success criteria:

- no critical workflow interruption caused by license handling,
- licensing behavior compatible with high-availability design.

2. Operations and DevOps Engineer (Customer Organization)

Primary focus:

- deployability,
- runtime stability,
- repeatable operations.

Licensing perspective:

- requires automation-friendly issue, renew, move, and release behavior,
- expects licensing operations to be manageable through IaC and CI/CD pipelines,
- requires user- and role-based access control over license allocation and checkout,
- requires visibility into allocations, limits, and expiry state.

Success criteria:

- low manual operational load,
- transparent and recoverable license lifecycle under normal and failure conditions.

3. Broadcast Engineer / Production Operator (Customer Organization)

Primary focus:

- production continuity,
- practical flexibility during live operations.

Licensing perspective:

- expects floating and movable license behavior across changing runtimes,
- requires emergency adaptation paths during incident or burst conditions,
- expects minimal licensing friction in time-critical operations.

Success criteria:

- licenses do not block operational decisions during active production.

4. Finance and Procurement Lead (Customer Organization)

Primary focus:

- budget predictability,
- cost control,
- contractual clarity.

Licensing perspective:

- requires transparent metering and modality options aligned to actual usage,
- expects clear linkage between usage dimensions and commercial impact,
- requires forecasting inputs across planned and event-driven demand.

Success criteria:

- explainable spend with controllable variance,
- no recurring billing surprises caused by opaque license behavior.

5. Security and Compliance Lead (Customer Organization)

Primary focus:

- risk control,
- compliance assurance,
- auditable operations.

Licensing perspective:

- requires secure, least-privilege license interactions,
- requires role-scoped and user-scoped entitlements so teams can restrict access to unnecessary licenses,
- requires auditable lifecycle events and policy alignment,
- avoids workflows that require unnecessary privileged vendor access.

Success criteria:

- license operations are compliant, observable, and reviewable.

Access Control and Entitlement Segmentation (Cross-Persona)

Beyond technical lifecycle handling, multiple personas require policy-level control over who can access which license rights and when.

This requirement appears in three recurring patterns:

- user- and role-based entitlement assignment to avoid over-provisioning,
- project or production scoped license pools to separate teams and cost centers,
- temporary access windows for external participants (for example rental or guest teams).

Representative operational scenarios include:

- restricting premium feature licenses to specifically authorized operators,
- allowing studio-rental customers to use only the rights included in their contracted tier,
- enforcing least-privilege defaults while preserving emergency override paths with audit logs.

This cross-persona concern links operational efficiency, security policy, and commercial packaging. It should therefore be treated as a primary design input for use case and workflow modeling in Sections 5.3 to 5.5.

ISV Personas

6. ISV Product and Commercial Owner (ISV)

Primary focus:

- product value capture,
- market fit,
- support scalability.

Licensing perspective:

- requires enforceable yet customer-usable entitlement models,
- requires flexibility across customer maturity levels and deployment modes,
- seeks standard interfaces to reduce one-off integration overhead.

Success criteria:

- sustainable revenue protection without blocking customer adoption.

7. ISV Licensing Architect (ISV)

Primary focus:

- control-plane design quality,
- lifecycle correctness,
- interoperability and maintainability.

Licensing perspective:

- requires clear and stable lifecycle semantics for issue, renew, move, revoke, and inspect,
- requires identity-aware binding models suitable for elastic runtimes,
- requires cryptographic trust and key-rotation approaches that remain operationally practical.

Success criteria:

- licensing architecture is portable across target environments,
- lifecycle behavior is deterministic and testable under failure and recovery conditions.

8. ISV Sales Director (ISV)

Primary focus:

- faster sales cycles,
- higher conversion rates,

- scalable pipeline growth.

Licensing perspective:

- requires clear and easy-to-position commercial packages,
- expects transparent and defensible value metrics for enterprise negotiations,
- requires low-friction onboarding models that reduce pre-sales complexity.

Success criteria:

- reduced time-to-close,
- increased attach rate and expansion revenue across customer segments.

9. ISV Partner and Channel Manager (ISV)

Primary focus:

- channel enablement,
- partner revenue growth,
- predictable co-sell operations.

Licensing perspective:

- requires reseller-compatible pricing and entitlement mechanics,
- requires channel-ready visibility for rights, usage, and renewals,
- expects low-dispute alignment between contract terms and operational behavior.

Success criteria:

- partner-led deals scale without recurring licensing disputes,
- channel operations remain commercially and operationally auditable.

10. ISV Support and Customer Success Lead (ISV)

Primary focus:

- support scalability,
- operational transparency for customers,
- reduction of avoidable escalation volume.

Licensing perspective:

- requires clear observability signals and support-grade diagnostics,
- requires low-friction remediation paths for entitlement and lease mismatches,
- expects policy and product behavior to be explainable to mixed technical/commercial teams.

Success criteria:

- incident resolution time remains bounded,

- recurring support patterns are reduced through better lifecycle and telemetry design.

System Integrator Personas

11. SI Solution Architect / Integrator (System Integrator)

Primary focus:

- multi-vendor solution delivery,
- deployment repeatability,
- integration reliability.

Licensing perspective:

- requires consistent API and lifecycle semantics across products,
- expects deterministic behavior across cloud and hybrid environments,
- requires clear failure and recovery handling in orchestrated workflows.

Success criteria:

- integration patterns can be reused across projects with low customization overhead.

12. SI Delivery and Operations Lead (System Integrator)

Primary focus:

- delivery predictability,
- operational handover quality,
- incident and change governance.

Licensing perspective:

- requires repeatable deployment runbooks that include license lifecycle controls,
- requires clear ownership boundaries for cross-vendor failure handling,
- expects measurable service outcomes for activation latency, renewal reliability, and recovery.

Success criteria:

- projects can be operated in stable mode without recurring licensing exceptions,
- change and incident processes remain effective under peak and failover conditions.

13. SI Commercial and Reselling Lead (System Integrator)

Primary focus:

- channel economics and margin sustainability,

- contract alignment across vendor, SI, and customer,
- predictable commercial operations in project and managed-service models.

Licensing perspective:

- requires commercially clear resale and pass-through models,
- requires entitlement and billing data that can be reconciled across all parties,
- expects licensing terms that support both project delivery and recurring service operations.

Success criteria:

- reselling and procurement workflows are operationally simple and auditable,
- commercial disputes are minimized through clear rights, usage, and billing traceability.

14. SI Sales and Account Director (System Integrator)

Primary focus:

- faster customer acquisition,
- deal expansion,
- portfolio monetization across services.

Licensing perspective:

- requires licensing offers that can be explained and priced quickly in proposals,
- expects flexible packaging for project-based and managed-service sales motions,
- requires strong commercial traceability to support enterprise procurement reviews.

Success criteria:

- reduced proposal-to-close cycle time,
- higher conversion and expansion rates in SI-led opportunities.

15. SI Managed Services Portfolio Lead (System Integrator)

Primary focus:

- profitable managed service operations,
- SLA adherence,
- lifecycle cost control.

Licensing perspective:

- requires lifecycle controls aligned with managed-service operating models,
- expects predictable renewal and entitlement behavior across customer estates,

- requires auditable usage and billing linkage for service reporting.

Success criteria:

- managed-service margins remain stable,
- SLA outcomes are met without recurring licensing escalations.

Ecosystem and Governance Personas

16. Platform SRE / Runtime Reliability Lead (Platform Provider)

Primary focus:

- runtime reliability,
- failure-domain containment,
- observable and recoverable control-plane behavior.

Licensing perspective:

- requires licensing control components to satisfy reliability and recovery objectives,
- expects telemetry, alerting, and health semantics suitable for platform operations,
- requires lifecycle operations to remain robust under scaling, replacement, and rollback.

Success criteria:

- licensing control functions meet reliability targets in production-like conditions,
- platform operations teams can diagnose and recover license-related incidents quickly.

This persona ensures that licensing architecture is operable as part of real platform operations, not only valid as an application-level concept.

17. Industry Standards and Policy Representative (Governance)

Primary focus:

- interoperability alignment,
- terminological consistency,
- sustainable cross-vendor guidance.

Licensing perspective:

- requires lifecycle semantics and terminology that can be interpreted consistently,
- expects traceability from industry pain points to recommended practices,
- requires guidance that supports implementation diversity without semantic drift.

Success criteria:

- cross-vendor interpretations remain compatible,
- standardization artifacts remain stable across drafting iterations.

This persona represents long-term alignment pressure and helps prevent fragmentation of meaning across implementations.

Cross-Persona Tensions

The personas above reveal recurring trade-offs:

- protection vs operational agility,
- predictability vs burst flexibility,
- strict control vs low-friction self-service,
- platform neutrality vs deep platform optimization.

An additional recurring tension concerns commercial model design:

- simplicity and predictability vs fine-grained value-based charging.
- OPEX model flexibility (burn-down, pre-pay, post-pay, point-based) vs CAPEX/perpetual procurement consistency.

In practice, organizations often combine several of these models in one account landscape. This creates additional friction points:

- customer demand for predictable budgeting vs supplier need for usage-linked monetization,
- SI resale simplicity vs mixed vendor contract logic,
- entitlement clarity at runtime vs billing flexibility across tiers and periods.

Supplier and integrator perspectives add two additional recurring tensions:

- product differentiation vs interoperability standardization,
- implementation flexibility vs supportability at scale.

For SI-led delivery models, another recurring tension is visible:

- channel simplicity and resale viability vs high-granularity commercial customization.

These tensions are expected and should be addressed explicitly in Sections 5.3 to 5.5.

The purpose of naming these tensions is to make trade-offs explicit early. Later sections should treat them as design inputs rather than as unresolved side effects.

Traceability to Subsequent Sections

This section informs:

- Section 5.3 by grounding recurring use case classes,
- Section 5.4 by exposing recurring requirement themes,
- Section 5.5 by framing workflow decision points and failure paths.

The intent is explicit traceability from stakeholder groups (Section 5.1), to role-level perspectives (Section 5.2), to scenario and workflow structure (Sections 5.3 to 5.5).

Section Positioning

This section is descriptive and analytical. It does not prescribe normative requirement language; it captures role-level viewpoints that justify later design and requirement choices.

05.03. Use Case Classes

Purpose

This section defines the recurring operational use case classes addressed by these best practices. A use case class is a category of operating situation that places a characteristic set of demands on licensing behavior. The classes generalize specific customer scenarios into reusable patterns so that later requirement themes (Section 5.4) and workflow definitions (Section 5.5) can be derived once and reused across many concrete deployments.

This section is written against a dynamic software licensing target model. Each class is therefore described in terms of how licensing is expected to behave in that situation, not in terms of any specific product or platform.

Method and Input Basis

The use case classes below synthesize:

- the stakeholder groups defined in Section 5.1,
- the role-level perspectives defined in Section 5.2,
- team prework (including Miro workshop material),
- the recurring industry use cases captured during the initiative's early analysis.

The classes are intentionally a small, stable set. They are not an exhaustive catalog of scenarios; they are the recurring demand patterns that most strongly differentiate licensing behavior. Concrete customer situations are expected to map onto one primary class, often with secondary influence from others.

Use Case Class Schema

Each use case class is specified using the same structure to keep comparison and downstream mapping consistent:

- **Operating context** — the production situation that characterizes the class.

- **Demand profile** — how load, duration, and predictability behave over time.
- **Licensing pressure** — the specific demands the situation places on licensing behavior.
- **Dominant modalities** — the commercial and consumption modalities most relevant to the class (for example time-based, process-based, or outcome-based).
- **Representative scenarios** — illustrative examples that ground the class without becoming the definition.
- **Primary lifecycle emphasis** — the licensing lifecycle operations most exercised by the class (issue, renew, reissue, revoke, inspect, usage reporting).
- **Key personas** — the Section 5.2 perspectives most affected.

This schema is descriptive. It does not introduce requirement statements; it organizes the inputs that later sections turn into requirement themes and workflows.

Overview of the Use Case Classes

Five primary use case classes are defined:

1. Continuous (24/7) Operations
2. Frequent Event-Driven Production
3. Infrequent Event-Driven Production
4. Test and Validation
5. AI-Powered and Outcome-Based Services

A sixth pattern, Resilience and Continuity, is treated as a cross-cutting overlay rather than a standalone class, because it applies across all five primary classes.

The primary classes are differentiated mainly by **duration and predictability of demand** and by the **commercial granularity** they require. Continuous operations sit at the predictable, low-granularity end; infrequent events and outcome-based services sit at the elastic, high-granularity end.

1. Continuous (24/7) Operations

Operating context

Always-on production environments that run continuously and cannot tolerate licensing-induced interruption. The canonical example is playout, but the class covers any sustained, long-running service.

Demand profile

Stable and predictable over long periods. Capacity changes are infrequent and usually planned. The dominant operational concern is uninterrupted continuity rather than rapid elasticity.

Licensing pressure

- Licensing must never become a single point of failure for live operation.
- Renewal and validation must continue without manual intervention.
- High-availability architectures (including hot/hot and hot/standby) must be supported, including the ability to apply an entitlement across primary and backup environments serving a single production.
- Expiry must be signalled well in advance so renewals never surprise operations during a broadcast window.

Dominant modalities

Predictable, low-granularity modalities are acceptable and often preferred: perpetual or annual commitments. More granular modalities may still be required for bounded experiments (for example launching a new channel for a limited period) layered on top of the baseline.

Representative scenarios

- 24/7 playout of a linear channel.
- A standing production gallery operated continuously.
- Launch of a new international channel for a defined limited period on top of an existing continuous estate.

Primary lifecycle emphasis

Renew (continuous), inspect (continuity and expiry visibility), reissue (for failover within high-availability designs).

Key personas

Director of Engineering, Operations and DevOps Engineer, Broadcast Engineer / Production Operator, Finance and Procurement Lead.

2. Frequent Event-Driven Production**Operating context**

Recurring, scheduled events that repeat often enough to be a routine part of operations but that still scale up and down around each occurrence. Examples include daily news and regular sports-league coverage.

Demand profile

Cyclical and largely foreseeable. Demand rises and falls around known event windows. Capacity is added for an event and released afterwards, repeatedly.

Licensing pressure

- Modalities must offer more granularity than continuous operations, for example monthly options that align with recurring event cadence.
- Activation and release must be repeatable and low-friction, because the cycle recurs often.
- Per-product and per-feature differentiation must be supported, because vendors may license individual products and features separately.
- Forecasting must account for repeated, partly predictable peaks.

Dominant modalities

Time-based with moderate granularity (annual baseline plus monthly or event-scooped additions), often combined per product and per feature.

Representative scenarios

- Daily news production with predictable studio and graphics demand.
- A recurring sports league with weekly match coverage.
- Seasonal programming that scales a known set of products for a defined run.

Primary lifecycle emphasis

Issue and revoke (repeated activation and release), renew, usage reporting, inspect.

Key personas

Broadcast Engineer / Production Operator, Operations and DevOps Engineer, Finance and Procurement Lead, SI Delivery and Operations Lead.

3. Infrequent Event-Driven Production

Operating context

Large, one-off or rarely-occurring events where capacity is stood up specifically for the event and torn down afterwards. The canonical example is a major periodic sporting event.

Demand profile

Sharp, bounded, and high-magnitude. A significant capacity uplift is needed for a short, well-defined window, with little or no steady-state footprint between events.

Licensing pressure

- Highly granular, consumption-based modalities are required (for example monthly, weekly, daily, or hourly) so that cost aligns to a short, intense window.

- Burst capacity must be obtainable quickly and released cleanly when the event ends.
- Managed-service and integrator-led delivery models must be supported, because customers often outsource these events (for example outside-broadcast operations or SI-run setups) to avoid licensing complexity themselves.
- Forecasting and provisional caps matter, because a single event can dominate the licensing bill.

Dominant modalities

Fine-grained, consumption-based time modalities (hourly/daily/weekly/monthly), frequently delivered through a managed-service or reseller commercial arrangement.

Representative scenarios

- A major multi-week international sporting event.
- A one-off broadcast of national significance requiring temporary scale-out.
- An outside-broadcast or pop-up production operated on the customer's behalf by an integrator.

Primary lifecycle emphasis

Issue and revoke around a bounded window, burst-oriented scaling, usage reporting, inspect for burn-rate visibility.

Key personas

Finance and Procurement Lead, Broadcast Director / Engineer, SI Managed Services Portfolio Lead, SI Commercial and Reselling Lead, ISV Product and Commercial Owner.

4. Test and Validation**Operating context**

Non-production and pre-production activity: validating an end-to-end workflow, building and stress-testing a new pipeline, or running periodic jobs. Activity is often ad-hoc and short-lived.

Demand profile

Intermittent and bursty, with short durations and unpredictable timing. Environments are created, exercised, and discarded.

Licensing pressure

- Highly granular modalities are required (for example hourly or weekly) so that short validation runs do not require production-scale commitments.

- Usage rights must be categorizable by purpose (for example production, test, training, disaster recovery) so that test consumption is distinguishable and appropriately governed.
- Self-service access to demo or sandbox entitlements is needed for development and staging environments without manual vendor involvement.
- Activation and release must integrate cleanly into automated build and test pipelines.

Dominant modalities

Short-duration, granular time modalities and scoped demo or sandbox entitlements, separated from production entitlement pools.

Representative scenarios

- Validating an end-to-end broadcast workflow before go-live.
- Running a periodic transcode or quality-control job.
- Building and stress-testing a new pipeline in a staging account.

Primary lifecycle emphasis

Issue and revoke (frequent, short-lived), inspect (which entitlements are allocated and which features are restricted), usage reporting for cost attribution.

Key personas

Operations and DevOps Engineer, SI Solution Architect / Integrator, Director of Engineering, ISV Support and Customer Success Lead.

5. AI-Powered and Outcome-Based Services**Operating context**

Value-added services where the licensed value is most naturally expressed as a delivered result rather than as elapsed time or provisioned capacity. Examples include automated content analysis and highlight generation.

Demand profile

Variable and result-driven. Consumption tracks the volume of work performed or outcomes produced, which may not correlate cleanly with wall-clock time or instance count.

Licensing pressure

- Outcome-based and process-based metering must be supported (for example per generated clip, per minute of subtitles, per compliance report, or per normalized process unit).
- Usage dimensions must be expressive enough to capture units of delivered value, not only time.

- Cost transparency must link delivered outcomes to commercial impact so that value-based charging remains explainable and defensible.
- Metering integrity matters, because charging is tied directly to recorded units.

Dominant modalities

Outcome-based and process-based modalities (process units, per-result metrics), often layered onto a time-based or entitlement baseline.

Representative scenarios

- Automated content analysis charged per analyzed asset.
- Highlight or clip generation charged per produced clip.
- Subtitle or compliance services charged per minute or per report.

Primary lifecycle emphasis

Usage reporting (central to this class), issue and renew for the underlying service entitlement, inspect for burn-rate and value attribution.

Key personas

ISV Product and Commercial Owner, Finance and Procurement Lead, ISV Sales Director, Security and Compliance Lead (metering integrity).

Cross-Cutting Overlay: Resilience and Continuity

Resilience and continuity needs — failover, blue/green cutover, hybrid operation, and disaster recovery — appear across all five primary classes rather than forming a standalone class. They are therefore treated as an overlay that intensifies the licensing pressure of whichever primary class is in effect.

Characteristic demands of the overlay include:

- license mobility between instances and environments without manual vendor intervention,
- the ability to move or reissue an active entitlement to a replacement workload during failover,
- use of cloud capacity as a backup to on-premises resources in hybrid designs,
- portable identity evidence so that the same entitlement can follow active and standby workloads,
- defined behavior when connectivity to licensing services is briefly lost.

The overlay binds most strongly to Continuous Operations (where interruption is least tolerable) and to Event-Driven Production (where failover during a live event is business-critical). It corresponds to the hybrid and disaster-recovery operational profile in the operational and deployment profiles chapter.

Use Case Classes Versus Usage Types

The use case classes above describe a **demand pattern** — how load, duration, and predictability behave. They should not be confused with the **usage type** of an environment, which describes its *purpose*: for example production, test, training, disaster recovery, or event. These are two orthogonal axes:

- The use case class answers “what is the demand shape?” (continuous, event-driven, ad-hoc, outcome-driven).
- The usage type answers “for what purpose is this capacity being consumed?” (production versus test versus training versus DR versus event).

Any class can run under more than one usage type. A continuous environment usually runs as a production usage type but may have a parallel test instance; an event-driven environment may be exercised as a test usage type before going live. The names overlap in ordinary language — “Test and Validation” is a class while “test” is also a usage type, and “event” appears on both axes — so the distinction is stated explicitly here to avoid conflation.

Usage type is significant because the team prework (the Miro workshop) threads it through several candidate workflows: usage type is determined during automated activation and guided setup, and a dedicated workflow creates purpose-specific license types. It is therefore carried forward as a requirement theme (access control and entitlement segmentation, T7 in Section 5.4) and as a workflow (usage-type differentiation in Section 5.5), rather than as an additional use case class here.

Use Case Class Comparison

The following table summarizes the differentiating characteristics. It is informative and intended to aid orientation, not to constrain mapping.

Use Case Class	Demand pre-dictability	Duration	Required granular-ity	Dominant modality emphasis	Lifecycle emphasis
Continuous (24/7) Op-erations	High	Long-running	Low (baseline)	Perpetual / annual	Renew, inspect, reissue
Frequent Event-Driven	Cyclical, foreseeable	Recurring windows	Moderate	Annual + monthly / event-scoped	Issue, revoke, renew, usage
Infrequent Event-Driven	Sharp, bounded	Short, intense	High	Hourly / daily / weekly / monthly	Issue, revoke, burst, usage

Use Case Class	Demand pre-dictability	Duration	Required granularity	Dominant modality emphasis	Lifecycle emphasis
Test and Validation	Low, ad-hoc	Short-lived	High	Granular time + demo / sandbox	Issue, revoke, inspect
AI / Outcome-Based	Result-driven	Variable	High (per result)	Outcome / process-based	Usage, issue, renew

Relationship to Operational and Deployment Profiles

The use case classes in this section are demand patterns expressed from the stakeholder and scenario viewpoint. The operational and deployment profiles chapter expresses closely related patterns from the runtime and deployment viewpoint. The two are intended to align:

- Continuous (24/7) Operations aligns with the continuous broadcast profile.
- Frequent and Infrequent Event-Driven Production align with the event production profile.
- Test and Validation aligns with the test and validation profile.
- AI-Powered and Outcome-Based Services aligns with the AI outcome services profile.
- The Resilience and Continuity overlay aligns with the hybrid and disaster-recovery profile.

This alignment is intentional. Use case classes explain *why* a demand pattern exists from a stakeholder perspective; operational profiles explain *how* it is run. Conformance and profile differentiation in later chapters can therefore reference either viewpoint consistently.

Traceability to Subsequent Sections

This section informs:

- Section 5.4 by exposing the recurring pressures that aggregate into requirement themes,
- Section 5.5 by identifying the lifecycle operations and decision points each class exercises, which structure the canonical workflows.

The intended traceability chain is: stakeholder groups (5.1) → role perspectives (5.2) → use case classes (5.3) → requirement themes (5.4) and workflow definitions (5.5).

Section Positioning

This section is descriptive and analytical. It classifies operating situations and the licensing pressures they create; it does not prescribe normative requirement language and does not specify API-level or schema-level behavior. Those concerns are addressed in the canonical API model and in the normative requirements chapter.

05.04. Derived Requirement Themes

Purpose

This section summarizes the recurring requirement themes that emerge from the stakeholder groups (Section 5.1), persona perspectives (Section 5.2), and use case classes (Section 5.3). A requirement theme is a clustered statement of need: a recurring concern that appears across multiple personas or use case classes and that later chapters translate into structured, normative requirements.

The themes are the analytical bridge between *needs* and *requirements*. They explain why later requirement groups exist and provide the rationale against which those requirements can be evaluated.

Method and Input Basis

The themes below are derived by aggregation. Where a concern appears repeatedly across distinct personas, use case classes, or industry challenges, it is consolidated into a single theme rather than restated per source. The inputs are:

- the role-level success criteria and licensing perspectives in Section 5.2,
- the licensing pressures attached to each use case class in Section 5.3,
- team prework (including the Miro workshop material), in particular the high-impact, low-overhead problem statements that were worked into candidate workflows,
- the recurring industry challenges identified during the initiative’s early analysis (summarized in the challenge mapping below),
- the cross-persona tensions named in Section 5.2.

The themes are intentionally a stable, modest-sized set. They are not requirements themselves. They do not use normative keywords, and they do not specify API-level or schema-level behavior. Final normative language is reserved for the normative requirements chapter.

Theme Schema

Each theme is specified using the same structure:

- **Theme statement** — the consolidated need in one or two sentences.
- **Originating concerns** — the personas, use case classes, and challenges that give rise to the theme.
- **What good looks like** — the observable characteristics that would indicate the theme is satisfied, expressed descriptively rather than as requirements.
- **Forward traceability** — the normative requirement group(s) expected to carry the theme into formal requirements.

Requirement Themes

Ten requirement themes are defined.

T1. Automation and Licensing-as-Code Theme statement

Licensing operations must be expressible and executable as code, so that issuing, renewing, moving, and releasing licenses can be embedded in infrastructure provisioning and deployment pipelines without manual steps.

Originating concerns

Operations and DevOps Engineer, SI Solution Architect, SI Developer, CTO; the Test and Validation and Event-Driven use case classes; the “IaC integration gap”, “manual setup processes”, and “headless activation gap” challenges.

What good looks like

- License lifecycle steps are part of infrastructure-as-code modules and CI/CD pipelines.
- Activation requires no interactive console step and no manual vendor intervention.
- Environments can be created, recovered, and torn down repeatably, including licensing.

Forward traceability

General requirements; integration requirements; API requirements.

T2. Standardized Lifecycle and Interfaces Theme statement

The licensing lifecycle — issue, renew, reissue, revoke, inspect, and usage reporting — must have consistent, standardized semantics and be accessible through both programmatic and command-line interfaces.

Originating concerns

SI Solution Architect, ISV Licensing Architect, Operations and DevOps Engineer, Industry Standards Representative; all use case classes; the “missing standard APIs”, “vendor-specific processes”, and “CLI/API access gap” challenges.

What good looks like

- Lifecycle operations behave consistently across products and vendors.
- The same operations are reachable via API and CLI, suited to different maturity levels.
- Integrators can reuse lifecycle patterns across projects with low customization.

Forward traceability

API requirements; general requirements.

T3. Identity-Based Binding and License Mobility Theme statement

Licenses must bind to verifiable workload identity rather than to static host attributes, so that entitlements can move between instances and environments to support failover, blue/green cutover, and hybrid operation.

Originating concerns

Director of Engineering, Broadcast Director / Engineer, Platform SRE, Security and Compliance Lead; the Continuous Operations class and the Resilience and Continuity overlay; the “hardware binding issues”, “license mobility”, and “manual ISV intervention” challenges.

What good looks like

- Binding uses workload identity evidence, not hostnames or MAC addresses.
- An active entitlement can follow active and standby workloads without downtime or vendor involvement.
- Hybrid and disaster-recovery designs are supported by portable binding.

Forward traceability

Identity and binding requirements; lease and policy requirements.

T4. License Modality and Commercial Flexibility Theme statement

The model must support multiple commercial and consumption modalities — time-based (including fine-grained durations), process-based, and outcome-based — and allow them to be combined without creating opaque portfolio complexity.

Originating concerns

Finance and Procurement Lead, ISV Product and Commercial Owner, License Manager, Procurement; all use case classes, especially Infrequent Event-Driven and AI/Outcome-Based; the “business model misalignment”, “model transition challenges”, and “portfolio complexity” challenges.

What good looks like

- Modalities span perpetual, annual, monthly, weekly, daily, hourly, process-unit, and outcome-based forms.
- OpEx-oriented models (pre-pay, post-pay, burn-down, point-based) and CapEx/perpetual models can coexist in one account landscape.
- Granularity can be selected to match the use case without forcing a single model.

Forward traceability

License modality requirements; metering requirements.

T5. Usage-Based Metering and Cost Transparency Theme statement

Usage must be metered through expressive dimensions and surfaced transparently, so that consumption can be attributed, forecast, and reconciled, and so that licensing cost is clearly distinguishable from platform cost.

Originating concerns

Finance and Procurement Lead, ISV Product and Commercial Owner, SI Commercial and Reselling Lead; the AI/Outcome-Based and Event-Driven classes; the “cost transparency” and “usage type differentiation” challenges; the Miro cost-predictability and cost-usage problem statements.

What good looks like

- Dimensions capture time, capacity, process units, and outcomes as appropriate.
- Usage can be attributed by project, show, environment, and cost center.
- Licensing cost is reported separately from underlying platform cost, with forecasting inputs and historical reconciliation.

Forward traceability

Metering requirements; license modality requirements.

T6. Observability and Self-Service Theme statement

Stakeholders must have clear visibility into allocation, usage, limits, expiry, and forecast, and must be able to perform routine actions — including releasing licenses back to the pool and adding capacity — without vendor involvement.

Originating concerns

Operations and DevOps Engineer, License Manager, ISV Support and Customer Success Lead, Finance and Procurement Lead; the Continuous and Test and Validation classes; the “end user portal absence”, “self-service capabilities”, and “cost transparency” needs; the Miro license-inventory query problem statement.

What good looks like

- Active allocations, remaining capacity, feature flags, and expiry are visible across vendors, ideally through a unified view.
- Operators can release orphaned or failed-instance licenses and add capacity themselves.
- Advance expiry warnings prevent surprises during production windows.

Forward traceability

Operational requirements; general requirements; metering requirements.

T7. Access Control and Entitlement Segmentation Theme statement

Access to license rights must be governable by user, role, and scope, so that organizations can apply least-privilege defaults, separate teams and cost centers, and grant time-boxed access to external participants.

Originating concerns

Security and Compliance Lead, Operations and DevOps Engineer, ISV Product and Commercial Owner, SI Commercial and Reselling Lead; the Access Control and Entitlement Segmentation cross-persona concern in Section 5.2; the Test and Validation and Event-Driven classes; the Miro usage-type-differentiation problem statement.

What good looks like

- Entitlements can be assigned by user and role to avoid over-provisioning.
- Project- or production-scoped pools separate teams and cost centers.
- Temporary access windows (for example rental or guest teams) are supported, with least-privilege defaults and audited emergency overrides.

Forward traceability

Security requirements; operational requirements; license modality requirements.

T8. Security, Trust, and Auditability Theme statement

Licensing must be cryptographically secured, auditable, and operable without granting vendors access to customer infrastructure, balancing intellectual-property protection against production continuity.

Originating concerns

Security and Compliance Lead, ISV Licensing Architect, Platform SRE; all use case classes; the “security vulnerabilities”, “trust vs availability balance”, and “remote server dependencies” challenges.

What good looks like

- Artifacts are signed and validated locally; credentials are short-lived; keys are managed and rotated.

- All lifecycle operations are auditable, with append-only records of who did what and when.
- Vendors do not require privileged access to customer resources; identity evidence is presented by the workload instead.

Forward traceability

Security requirements; identity and binding requirements.

T9. Continuity Under Failure and Grace Behavior Theme statement

Licensing must tolerate transient connectivity loss and component failure through local validation, renewal retries, defined grace policies, and controlled degradation, so that licensing is never a single point of failure for live production.

Originating concerns

Director of Engineering, Broadcast Director / Engineer, Platform SRE, Operations and DevOps Engineer; the Continuous Operations class and the Resilience and Continuity overlay; the “grace policy gaps”, “offline licensing requirements”, and “trust vs availability balance” challenges; the Miro air-gapped / remote-server problem statement.

What good looks like

- Applications continue using a locally validated artifact when the licensing service is briefly unreachable.
- Grace windows and exhaustion actions (drain, degrade, stop) are defined, governed, and alerted on before they affect production.
- Air-gapped and offline operation is supported with appropriately reduced functionality.

Forward traceability

Lease and policy requirements; operational requirements; security requirements.

T10. Interoperability and Anti-Fragmentation Theme statement

The model must standardize core concepts and interfaces while allowing implementation flexibility, so that adopters avoid siloed, vendor-specific licensing and platform lock-in.

Originating concerns

Industry Standards Representative, SI Solution Architect, ISV Product and Commercial Owner; all use case classes; the “vendor lock-in risks”, “missing standard APIs”, and “industry knowledge gap” challenges; the standardization-vs-differentiation tension in Section 5.2.

What good looks like

- Core concepts, token structure, and lifecycle operations are standardized; vendor implementations remain free to differentiate.
- Solutions function across cloud, hybrid, on-premises, and air-gapped environments.
- Terminology and semantics remain stable across vendors and drafting cycles.

Forward traceability

General requirements; API requirements; integration requirements.

Challenge-to-Theme Coverage

The initiative's early analysis grouped recurring industry challenges by impact and overhead. The table below maps those challenges onto the themes above to confirm that the theme set provides coverage. It is informative.

Challenge	Primary theme(s)
IaC integration gap	T1
Manual setup processes	T1, T2
Usage type differentiation (prod/test/training/DR)	T5, T7
Remote server dependencies	T8, T9
Hardware binding issues	T3
CLI/API access gap	T2
Cost transparency	T5, T6
Model transition challenges	T4
License interdependencies	T4, T10
Business model misalignment	T4
Industry knowledge gap	T10
Vendor-specific processes	T2, T10
Missing standard APIs	T2, T10
Headless activation gap	T1
Manual ISV intervention	T3, T9
Portfolio complexity	T4
Security vulnerabilities	T8
License mobility	T3
Customer maturity variations	T2, T6
Offline licensing requirements	T9
Trust vs availability balance	T8, T9
Vendor lock-in risks	T10
End user portal absence	T6
Grace policy gaps	T9

Theme Tensions

Several themes pull against one another. These tensions are expected and are recorded here so that later requirement and design work treats them as explicit trade-offs rather than as defects:

- **Protection vs continuity** — T8 (security and trust) vs T9 (continuity under failure): stricter enforcement reduces availability headroom; longer grace increases exposure.
- **Flexibility vs simplicity** — T4 (modality flexibility) vs T5/T6 (transparency and self-service): finer-grained commercial models can reduce explainability.
- **Standardization vs differentiation** — T2/T10 (standardized interfaces and interoperability) vs vendor desire to differentiate implementation and packaging.
- **Control vs low friction** — T7 (access control and segmentation) vs T1 (automation): tighter access governance can add steps to automated flows.

These map directly to the cross-persona tensions named in Section 5.2.

Forward Traceability to Normative Requirements

The themes are intended to flow into the normative requirements chapter. The table below records the primary mapping; a theme may also contribute secondarily to other groups.

Theme	Primary normative requirement group
T1 Automation and Licensing-as-Code	Integration; General
T2 Standardized Lifecycle and Interfaces	API; General
T3 Identity-Based Binding and Mobility	Identity and Binding
T4 License Modality and Commercial Flexibility	License Modality
T5 Usage-Based Metering and Cost Transparency	Metering
T6 Observability and Self-Service	Operational
T7 Access Control and Entitlement Segmentation	Security; Operational
T8 Security, Trust, and Auditability	Security
T9 Continuity Under Failure and Grace Behavior	Lease and Policy
T10 Interoperability and Anti-Fragmentation	General; Integration

Section Positioning

This section is descriptive and analytical. It consolidates needs into themes and traces them forward; it does not state normative requirements, does not use normative keywords as requirement language, and does not define API-level or schema-level behavior. The normative requirements chapter is the place where these themes become disciplined, testable requirements.

05.05. Workflow Definitions

Purpose

This section defines the canonical workflow patterns used to derive system requirements from the stakeholder, use case, and challenge analysis in Sections 5.1 to 5.4. A workflow definition captures *what* happens — the actors, inputs, ordered steps, decisions, and outputs — independently of *how* any particular platform implements it.

These workflows provide the traceability bridge from challenges and use cases to derived requirement themes, and ultimately to the normative requirements chapter.

Method and Input Basis

The workflows below are derived from:

- team prework, in particular the Miro workshop board *IAMT DSL Workflows*, which is the primary source baseline for this section,
- the use case classes in Section 5.3 and the requirement themes in Section 5.4,
- the original proposal draft, which contributed the lifecycle-continuity workflows.

The Miro board worked the high-impact, low-overhead problem statements into candidate workflows using a shared method. That method is adopted here unchanged.

Workflow Method

A workflow in this section is expressed with a small, consistent vocabulary, following the method established in the prework:

- **Who** — the actor(s) and their responsibilities.
- **Input** — what enters the workflow.
- **Process step(s)** — the ordered actions, including decision points and branches.
- **Output** — the resulting state or artifact.

Abstraction principle: what, not how The defining characteristic of the method is deliberate abstraction. A workflow states what must happen and in what order; it does not prescribe the implementation. The prework illustrated this with a simple example — a “clean car” workflow that is *not concerned with* which route to drive, how payment is made, or how the wash is performed.

Applied to licensing, this means workflow definitions:

- name the licensing lifecycle operations they invoke (issue, renew, reissue, revoke, inspect, usage reporting) without specifying their request or response detail,
- describe identity, entitlement, and policy handling in capability terms, not in terms of any specific identity provider, key store, or transport,
- remain valid across cloud, hybrid, on-premises, and air-gapped deployments.

API-level and schema-level detail is therefore deliberately out of scope here; it belongs to the canonical API model chapter and the annexes. The prework’s own technical-solution notes (for example specific machine-to-machine authentication flows) are likewise treated as implementation detail.

Decision points and failure paths Workflows may branch. The prework’s transcode example shows a decision step — pass the signal through if it is already in the required format, otherwise transcode it. Licensing workflows contain analogous decisions (for example: is the entitlement available? is connectivity present? has grace been exhausted?). Each workflow below records its principal decision points and the failure paths that follow when a decision resolves unfavorably.

Workflow Schema

Each workflow is specified using the following structure, consistent with the chapter scope:

- **Trigger and objective** — what initiates the workflow and what it aims to achieve.
- **Actors** — the roles and components responsible (the “Who”).
- **Preconditions and inputs** — required state and inputs before the workflow runs.
- **Steps and decisions** — the ordered process steps, including branches.
- **Outputs and state changes** — resulting artifacts and state transitions.
- **Failure paths and recovery** — what happens when a step or decision fails.
- **Traceability** — the use case classes and requirement themes the workflow serves.

Workflow Catalog

The workflows are grouped into two sets:

- **Group A — Adoption and operations workflows.** Derived directly from the Miro problem statements. These address the friction of adopting and operating licensing in software-defined environments.
- **Group B — Lifecycle continuity workflows.** Derived from the original proposal draft. These address the runtime continuity of an active license over its lifetime.

ID	Workflow	Origin	Primary themes
WF-01	Automated license activation in deployment	Miro: IaC integration gap	T1, T2
WF-02	License cost estimation (TCO)	Miro: cost predictability	T5, T4
WF-03	Usage reporting and licensing-unit derivation	Miro: cost usage	T5, T6
WF-04	Usage-type differentiation	Miro: usage type	T7, T4
WF-05	Guided license setup	Miro: manual setup processes	T1, T2
WF-06	License inventory query and observability	Miro: CLI/API access gap	T6, T2
WF-07	Identity-based provisional licensing	Miro: hardware binding issues	T3
WF-08	Offline / air-gapped license acquisition	Miro: remote server dependencies	T9, T8
WF-09	Lease renewal and continuity	Original draft	T2, T9
WF-10	Failover reissue and license mobility	Original draft	T3, T9
WF-11	Revocation and enforcement	Original draft	T8, T2
WF-12	Grace and offline continuity	Original draft	T9
WF-13	Burst capacity for events	Original draft	T1, T4

Group A — Adoption and Operations Workflows

WF-01. Automated License Activation in Deployment Trigger and objective

A deployment pipeline provisions a system and must obtain a working license without any manual step. Objective: a licensed, running system at the end of an automated deployment.

Actors

Operations / DevOps engineer (owner of the pipeline); SI developer (authoring the IaC module); license agent and control plane (components).

Preconditions and inputs

- An entitlement exists for the product and intended environment.
- The deployment carries enough context to determine the usage type.
- Input: a DevOps deployment workflow.

Steps and decisions

1. Deploy the system through infrastructure-as-code.
2. Determine the usage type (for example development, test, production, or event).
3. Decision: is a valid entitlement available for that usage type? If not, follow the failure path.
4. Activate the license as part of the automated deployment (issue operation).

Outputs and state changes

A licensed system; an active lease bound to the workload; an audit record of activation.

Failure paths and recovery

- No matching entitlement: the pipeline surfaces a clear, actionable error rather than silently proceeding unlicensed; deployment is held or rolled back per pipeline policy.
- Control plane unreachable at activation: retry with backoff; if a pre-issued or cached artifact is permitted by policy, proceed under grace (see WF-12), otherwise fail the deployment cleanly.

Traceability

Use cases: all classes, especially Test and Validation and Event-Driven. Themes: T1, T2; secondary T7 (usage type).

WF-02. License Cost Estimation (TCO) Trigger and objective

A team needs to estimate licensing cost for a planned configuration before committing to it. Objective: a defensible total-cost estimate linked to expected usage.

Actors

Finance and procurement lead; director of engineering; ISV product and commercial owner (catalog/price input).

Preconditions and inputs

- A catalog that links dimensions and modalities to price.
- Input: a production or integration template describing the media functions to be deployed.

Steps and decisions

1. Map each media function to the media service usage it implies.
2. Map media service usage to the licensing needs it generates.
3. Aggregate usage according to the licenses needed.
4. Convert aggregated usage to costs using the catalog.

Outputs and state changes

A total-cost-of-ownership estimate. No runtime licensing state changes (this is a planning workflow).

Failure paths and recovery

- Missing price or dimension mapping for a function: flag the gap explicitly; the estimate marks the unpriced element rather than omitting it silently.
- Uncertain usage volumes: produce a ranged estimate with stated assumptions.

Traceability

Use cases: Event-Driven (frequent and infrequent), AI/Outcome-Based. Themes: T5, T4; secondary T6.

WF-03. Usage Reporting and Licensing-Unit Derivation Trigger and objective

Operations and finance need to understand actual consumption and express it in licensing terms distinct from infrastructure cost. Objective: derive licensing unit(s) from observed system usage.

Actors

Operations / DevOps engineer; finance and procurement lead; license manager.

Preconditions and inputs

- Systems emit usage signals.
- Input: observed system usage across the estate.

Steps and decisions

1. Collect usage reporting for each system.
2. Query that usage specifically on its licensing dimensions (separating licensing-relevant usage from general infrastructure usage).
3. Derive the licensing unit(s) consumed (for example duration- or value-based units).

Outputs and state changes

Licensing unit records suitable for reconciliation, forecasting, and billing inputs. Usage is stored for later querying.

Failure paths and recovery

- Gaps in usage signals: mark periods of missing data rather than interpolating silently.
- Buffered usage after an outage: reconcile idempotently so records are not double-counted.

Traceability

Use cases: AI/Outcome-Based, Event-Driven, Continuous. Themes: T5, T6; secondary T4.

WF-04. Usage-Type Differentiation Trigger and objective

A team needs license rights categorized by purpose so that non-production use is distinguished and governed. Objective: associate a purpose-specific license with a need.

Actors

Operations / DevOps engineer; security and compliance lead; ISV product and commercial owner.

Preconditions and inputs

- The model supports usage-type categorization (for example production, test, training, disaster recovery).
- Input: a development, test, training, or other categorized need.

Steps and decisions

1. Create or select a special license type matching the need's category.
2. Associate the license with the requesting context.

Outputs and state changes

Categorized development/test (or other purpose) capacity, distinguishable from production entitlement pools.

Failure paths and recovery

- Requested category not offered: surface the unavailable category; do not silently fall back to a production entitlement.
- Category misuse (for example test capacity used in production): detectable through the usage type recorded on the lease and reported via WF-03.

Traceability

Use cases: Test and Validation; Resilience and Continuity overlay (DR category).
Themes: T7, T4; secondary T5.

WF-05. Guided License Setup Trigger and objective

A system is set up outside a full IaC pipeline (a more manual path for lower-maturity adopters) but still needs a correctly associated license. Objective: a licensed application via a guided, repeatable setup.

Actors

Operations team; ISV support and customer success lead.

Preconditions and inputs

- A documented setup path exists.
- Input: a manual setup workflow.

Steps and decisions

1. Install the system.
2. Determine the usage type (development, test, production, or event).
3. Associate the appropriate license (issue operation).

Outputs and state changes

A licensed application; an active lease; an audit record.

Failure paths and recovery

- Wrong usage type selected: correctable by re-association; the change is audited.
- This workflow is intentionally the lower-automation counterpart to WF-01; adopters are expected to graduate from WF-05 to WF-01 as maturity increases.

Traceability

Use cases: all classes (lower-maturity adopters). Themes: T1, T2; secondary T7.

WF-06. License Inventory Query and Observability Trigger and objective

An integrator or operator needs transparent visibility of licenses across systems and vendors. Objective: observability into which licenses exist, which are in use, and where.

Actors

SI solution architect / integrator; operations team; license manager.

Preconditions and inputs

- Licenses are inspectable through a programmatic or command-line interface.
- Input: an integrator query workflow.

Steps and decisions

1. Query the license system(s) for licenses within a specific context (applying filters such as tenant, product, environment, or status) — inspect operation.
2. Decision: are results complete across all relevant systems? If a source is unreachable, follow the failure path.

Outputs and state changes

A consolidated view of licenses available and in use, including their location. No state change (read-only).

Failure paths and recovery

- A source system is unreachable: report partial results explicitly, marking which sources were not reached, rather than presenting an incomplete view as complete.

Traceability

Use cases: all classes; strongest for Continuous and Test and Validation. Themes: T6, T2.

WF-07. Identity-Based Provisional Licensing Trigger and objective

A workload must be licensed by its verifiable identity rather than by a host attribute, so the license is not node-locked. Objective: a provisionally licensed application bound to workload identity.

Actors

Platform SRE; operations / DevOps engineer; ISV licensing architect.

Preconditions and inputs

- The workload can present identity evidence.
- Input: a provisional system to be licensed.

Steps and decisions

1. Install the system.
2. Decision: is the presented identity evidence valid? If not, follow the failure path.
3. License the provisional application by binding to the workload identity (issue operation), not to a hostname or MAC address.

Outputs and state changes

A provisionally licensed application bound to workload identity; an active lease.

Failure paths and recovery

- Invalid or absent identity evidence: deny issuance with a clear reason; do not fall back to host-based binding.
- Identity rotation or replacement: handled by reissue (WF-10) rather than re-installation.

Traceability

Use cases: Continuous, Resilience and Continuity overlay. Themes: T3; secondary T8.

WF-08. Offline / Air-Gapped License Acquisition Trigger and objective

A system has no connection to a remote licensing service and must still be licensed. Objective: a licensed application in an air-gapped environment.

Actors

Security and compliance lead; operations team.

Preconditions and inputs

- An offline licensing path is supported, including transfer via portable media.
- Input: an air-gapped system.

Steps and decisions

1. Acquire a license for a defined period, potentially generated ahead of time and carried on transferable media.
2. Transfer the license to the air-gapped system.
3. Validate the license locally on the target system.

Outputs and state changes

A licensed application operating offline for the defined period, with reduced functionality where applicable.

Failure paths and recovery

- License period expiry with no connectivity: defined offline behavior applies (see WF-12), with advance local warnings.
- Transfer integrity failure: validation rejects a tampered or corrupt artifact.

Traceability

Use cases: Resilience and Continuity overlay; Test and Validation (isolated environments). Themes: T9, T8; secondary T3.

Group B — Lifecycle Continuity Workflows

These workflows describe the runtime continuity of an active license. They are expressed at the same abstraction level as Group A; the underlying lifecycle operations are defined in the canonical API model chapter and are not detailed here.

WF-09. Lease Renewal and Continuity Trigger and objective

An active lease approaches expiry and must be extended without interrupting operation. Objective: uninterrupted continuity through timely renewal.

Actors

License agent (component); operations / DevOps engineer (policy owner).

Preconditions and inputs

An active lease with a renewal window and policy; input: the approaching expiry of the lease.

Steps and decisions

1. Begin renewal ahead of expiry, within the renewal window (renew operation).
2. Decision: did renewal succeed before expiry? If not, enter grace handling (WF-12).
3. On success, continue operation under the extended lease.

Outputs and state changes

An extended lease and a refreshed local artifact; continuity preserved.

Failure paths and recovery

- Renewal fails: retry with backoff; on continued failure, WF-12 governs behavior; advance expiry warnings are raised so operations are not surprised during a production window.

Traceability

Use cases: Continuous, Event-Driven. Themes: T2, T9; secondary T6.

WF-10. Failover Reissue and License Mobility Trigger and objective

An active workload fails or is replaced (including planned blue/green cutover), and its entitlement must follow to a new workload without manual vendor intervention. Objective: mobility of an active entitlement across workloads.

Actors

Broadcast director / engineer; platform SRE; license agent and control plane.

Preconditions and inputs

An active lease; a replacement workload able to present its own identity evidence; input: a failover or cutover event.

Steps and decisions

1. Initiate transfer of the lease to the new workload identity (reissue operation).
2. Decision: planned cutover or unplanned failover? A planned cutover may use a pre-issued cutover artifact if connectivity is briefly unavailable.
3. Finalize the transfer: activate the new binding and revoke the old.

Outputs and state changes

The entitlement now bound to the new workload; the prior binding revoked; an audit record of the transfer.

Failure paths and recovery

- Control plane briefly unreachable during planned cutover: proceed on a pre-issued artifact, then finalize and revoke once reconnected.
- New identity evidence invalid: transfer is denied; the original binding remains until a valid target is presented.

Traceability

Use cases: Continuous, Event-Driven, Resilience and Continuity overlay. Themes: T3, T9.

WF-11. Revocation and Enforcement Trigger and objective

An administrator or a policy condition requires an active license to be terminated. Objective: prompt, auditable revocation with controlled application response.

Actors

Security and compliance lead / administrator; control plane; license agent.

Preconditions and inputs

An active lease; a revocation trigger (administrative action or policy violation); input: the revocation request.

Steps and decisions

1. Mark the lease revoked in the control plane (revoke operation).
2. Decision: is the affected workload currently reachable? If reachable, push the revocation; if not, it takes effect at the next renewal.
3. The application responds per policy (for example drain to a safe state).

Outputs and state changes

A revoked lease; the application brought to its policy-defined post-revocation state; an audit record.

Failure paths and recovery

- Workload offline at revocation: revocation is enforced at the next renewal; exposure is bounded by short artifact lifetimes.

Traceability

Use cases: all classes. Themes: T8, T2; secondary T9.

WF-12. Grace and Offline Continuity Trigger and objective

Connectivity to the licensing service is lost while a workload is running. Objective: defined, governed behavior that protects production continuity while bounding exposure.

Actors

License agent (enforces policy); operations / DevOps engineer and ISV (define policy).

Preconditions and inputs

An active, locally validated artifact and an applicable grace policy; input: loss of connectivity.

Steps and decisions

1. Continue operating on the locally validated artifact (no round-trip required).
2. Retry renewal in the background with backoff.
3. Decision: did connectivity return within the grace window? If yes, resume normal renewal (WF-09).
4. If the grace window is exhausted, apply the policy-defined action: drain, degrade, or stop.

Outputs and state changes

Either restored normal operation or a controlled, policy-defined degradation; alerts raised as the grace window counts down; buffered usage reconciled on reconnection.

Failure paths and recovery

- Grace exhausted: the configured action (drain/degrade/stop) takes effect, having alerted operations beforehand.

Traceability

Use cases: Continuous, Resilience and Continuity overlay. Themes: T9; secondary T8.

WF-13. Burst Capacity for Events Trigger and objective

A planned event requires additional capacity for a bounded window. Objective: scale licensed capacity up for the event and release it cleanly afterwards.

Actors

Director of engineering; broadcast director / engineer; finance and procurement lead.

Preconditions and inputs

Event capacity plans and applicable guardrail policies; input: an event capacity request.

Steps and decisions

1. Request additional entitlements or dimensions for the event window (issue operation under guardrail policy).
2. Decision: is the request within guardrails and forecasted budget? If not, it is held for approval.
3. Operate at scaled capacity during the event.
4. Release the additional capacity when the event ends (revoke operation), returning licenses to the pool.

Outputs and state changes

Temporarily scaled capacity during the event; capacity released and reconciled afterwards; usage recorded for forecasting.

Failure paths and recovery

- Guardrail or budget exceeded: the request is held rather than silently granted, preserving spend predictability.
- Failure to release after the event: detectable through inventory (WF-06) and reclaimed.

Traceability

Use cases: Event-Driven (frequent and infrequent). Themes: T1, T4; secondary T5, T6.

Traceability Summary

The table consolidates workflow-to-use-case and workflow-to-theme mappings. It is the chapter-internal bridge intended to support derivation of normative requirements.

Workflow	Primary use case class(es)	Requirement themes
WF-01 Automated activation	All; Test, Event-Driven	T1, T2, T7
WF-02 Cost estimation (TCO)	Event-Driven, AI/Outcome	T5, T4, T6
WF-03 Usage reporting / units	AI/Outcome, Event-Driven, Continuous	T5, T6, T4
WF-04 Usage-type differentiation	Test and Validation; DR overlay	T7, T4, T5
WF-05 Guided setup	All (lower maturity)	T1, T2, T7
WF-06 Inventory query	All; Continuous, Test	T6, T2
WF-07 Identity-based provisional licensing	Continuous; Continuity overlay	T3, T8
WF-08 Offline / air-gapped acquisition	Continuity overlay; Test	T9, T8, T3
WF-09 Renewal and continuity	Continuous, Event-Driven	T2, T9, T6
WF-10 Failover reissue / mobility	Continuous, Event-Driven, overlay	T3, T9
WF-11 Revocation and enforcement	All	T8, T2, T9
WF-12 Grace / offline continuity	Continuous, overlay	T9, T8
WF-13 Burst capacity	Event-Driven	T1, T4, T5, T6

Relationship to Other Chapters

- The canonical API model chapter defines the lifecycle operations these workflows invoke (issue, renew, reissue, revoke, usage reporting, inspect). This section names those operations but does not specify their payloads.
- The normative requirements chapter derives testable requirements from these workflows via the requirement themes in Section 5.4.

- The operational and deployment profiles chapter expresses the runtime environments in which these workflows execute.
- Implementation-specific realizations (for example particular authentication flows or reference architectures) belong in the informative annexes.

Section Positioning

This section is descriptive and analytical. It defines workflow patterns at the “what, not how” level established in the prework. It does not prescribe normative requirement language and does not contain API-level or schema-level detail; both are out of scope for this section.

06. Design Principles

Purpose

This chapter defines the core design principles of the Dynamic Software Licensing best practices.

Its purpose is to establish the high-level principles that should guide the architecture, interfaces, operational behavior, and implementation patterns described in later chapters.

Role Within the Best Practices

The design principles form the conceptual backbone of the DSL best practices.

They define the qualities that the target licensing model should embody and provide a stable reference point for:

- architectural choices,
- requirement language,
- implementation expectations,
- conformance discussions.

Foundation of This Chapter

This chapter is grounded in the same five core principles introduced in Section 01.04:

- API-first access
- security by design
- observability
- platform independence
- licensing-as-code

This chapter expands those five principles into more detailed design guidance for architecture, runtime behavior, interfaces, and implementation choices.

Scope of This Chapter

This chapter defines the core principles that influence the best practices as a whole. It describes:

- what these principles mean
- why they matter
- how they guide later chapters.

It does not attempt to define all detailed implementation rules.

Structure of This Chapter

This chapter is organized into the following subchapter:

01. Design Principles Defines the core design principles that guide the DSL best practices.

This section explains what each principle means, why it matters, and how it shapes later architecture, interface, operational, and conformance decisions.

Relationship to Other Chapters

This chapter informs:

- the reference model,
- conformance and profiles,
- normative requirements,
- canonical API and data model chapters,
- security and operational chapters.

It acts as a stable point of reference when contributors make decisions about detailed wording elsewhere in the best practices.

Chapter Positioning

This chapter provides the shared architectural philosophy for the DSL best practices. It translates recurring repository themes such as automation, security, portability, observability, and resilience into durable design guidance that later chapters can apply more concretely.

The chapter is intentionally platform-agnostic. It defines the design qualities the DSL should embody, while leaving API details, schema definitions, and deployment-specific realizations to the canonical model, security framework, and operational profiles.

06.01. Design Principles

These principles define how the licensing system should be designed, implemented, and evaluated. They serve as decision-making guardrails to ensure consistency, interoperability, and alignment across vendors, customers, and system integrators.

This section expands the five core principles introduced in Section 01.04 (API-First, Security by Design, Observability, Platform Independence, and Licensing as Code) into more detailed guidance for runtime binding, resilience, access patterns, and implementation flexibility.

Principles

1. API-First and Automation-Ready What it means

All licensing capabilities, including issue, renew, reissue, revoke, and usage reporting, must be accessible through well-defined APIs and CLI interfaces.

Why it matters

Modern software-based workflows rely on automation, repeatability, and integration. Licensing that depends on manual processes becomes a bottleneck for deployment, scaling, testing, and recovery. An API-first model enables full integration into automation pipelines and operational tooling.

2. Licensing as Code What it means

Licensing should be treated as part of system configuration and deployment, using declarative definitions that integrate with infrastructure provisioning and application lifecycle management.

Why it matters

Separating licensing from deployment introduces friction and inconsistency. Managing licenses alongside infrastructure ensures environments can be created, tested, and recovered in a repeatable and predictable way across production, test, and disaster recovery scenarios.

3. Identity-Based Binding Over Hardware Binding What it means

Licenses should bind to verifiable workload identity, such as service identity or workload attestation, rather than static machine attributes such as hostnames or MAC addresses.

Why it matters

Software-based systems are dynamic by nature. Hardware-bound licensing limits portability, complicates failover, and introduces operational risk. Identity-based binding enables agility, resilience, and seamless transitions across environments.

4. Security by Design What it means

Licensing mechanisms must incorporate cryptographic security, short-lived credentials, strong identity validation, and auditable operations as foundational elements.

Why it matters

Licensing systems enforce both access control and revenue protection. Weak security models introduce risk for both vendors and customers. Embedding security at the core ensures trust, compliance, and protection against misuse.

5. Resilience and Continuity What it means

Licensing systems should tolerate transient failures and support continued operation through mechanisms such as local validation, renewal retries, grace policies, seamless failover, and controlled degradation.

Why it matters

Media and broadcast workflows are often time-critical and cannot tolerate interruptions due to licensing dependencies. Licensing must not become a single point of failure. Systems should balance protection with operational continuity.

6. Observability and Transparency What it means

The system must provide clear visibility into license allocation, usage, limits, expiry, and cost-relevant dimensions across all stakeholders.

Why it matters

Licensing impacts both operations and financial planning. Transparent usage and allocation data enable better forecasting, auditing, troubleshooting, and decision-making for engineering, operations, and finance teams.

7. Platform-Agnostic by Design What it means

The licensing model should function consistently across different environments, including cloud, on-premises, air-gapped, and hybrid deployments, without reliance on platform-specific constructs.

Why it matters

Media workflows are inherently heterogeneous. A platform-agnostic approach prevents lock-in, supports interoperability, and ensures the system can be adopted across diverse infrastructure environments.

8. Multi-Interface Accessibility What it means

Licensing functionality should be accessible through multiple interaction models, including APIs, CLI tools, SDKs, and user interfaces.

Why it matters

Different users operate at different levels of technical maturity. Supporting multiple interfaces ensures accessibility for both automated systems and human operators, enabling broader adoption and usability.

9. Standardized Interfaces, Flexible Implementations What it means

Core concepts such as APIs, token structures, and lifecycle operations should be standardized, while allowing flexibility in how vendors implement their internal systems and business models.

Why it matters

Standardization reduces integration complexity and fragmentation, while flexibility allows vendors to innovate and differentiate. This balance enables ecosystem-wide consistency without constraining implementation choices.

10. Designed for Real-World Media Workloads What it means

The licensing model should support diverse usage patterns, including continuous operation, event-based workloads, burst capacity, testing environments, backup or disaster recovery workflows, and outcome-based services.

Why it matters

Media workflows vary significantly in duration, scale, and predictability. A one-size-fits-all licensing model is insufficient. Supporting multiple modalities ensures the system aligns with how software is actually used in production.

07. Reference Model

Purpose

This chapter defines the conceptual and functional reference model of the Dynamic Software Licensing best practices.

Its purpose is to describe the main structural elements of the proposed licensing model and the relationships between them.

Role Within the Best Practices

The reference model is one of the central chapters of the best practices.

It translates the design principles into an architectural view that can be used to explain:

- how the licensing model is structured,
- which components and planes exist,
- how responsibilities are separated,
- how trust and interaction flow through the system.

Architectural Basis

The current DSL material already distinguishes between major architectural areas such as:

- the Control Plane,
- the Workspace Plane,
- an End User Portal,
- associated components such as a License Agent (Broker), SDK integration, keys, usage services, and analytics.

This chapter organizes that model into a consistent reference structure.

Scope of This Chapter

This chapter describes:

- the conceptual model,
- the functional components,
- the major planes,
- trust relationships,
- responsibility boundaries.

It remains a platform-neutral reference chapter rather than a platform-specific implementation chapter.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Conceptual Model Defines the abstract concepts and their relationships.

02. Functional Components Defines the main logical components of the licensing model.

03. Control Plane Defines the responsibilities and characteristics of the Control Plane.

04. Workspace Plane Defines the responsibilities and characteristics of the Workspace Plane.

05. End User Portal Defines the role of an end-user-facing management interface.

06. Trust Model Defines how trust is established and maintained across the system.

07. Responsibility Model Defines which participant or component is responsible for which aspect of the model.

Relationship to Other Chapters

This chapter informs:

- conformance and profiles,
- normative requirements,
- canonical API and data models,
- security and operational chapters,
- relevant annexes and diagrams.

It acts as the main architectural reference point for the rest of the best practices.

Chapter Positioning

This chapter serves as the primary architectural overview for the published baseline.

It is used during review as the architectural reference point for API behavior, trust boundaries, operational responsibilities, and chapter-to-chapter consistency.

07.01. Conceptual Model

Purpose

This section defines the abstract concepts and their relationships within the Dynamic Software Licensing (DSL) model. It provides a platform-neutral view

of how licensing is structured before any implementation detail is introduced.

Core Concepts

The DSL model is built around the following foundational concepts:

Entitlement An entitlement represents the right granted to a user to use a specific application under defined conditions. It captures what the user has purchased or been allocated, including the dimensions (e.g., channels, FPS, bandwidth), the permitted license modality (e.g., hourly, daily, monthly, annual), and any associated policies.

An entitlement exists independently of any running workload. It is the source of truth for what a user is allowed to consume.

Lease A lease is a time-bound, active binding between an entitlement and a running workload. When a workload requests access to a licensed application, the system issues a lease that grants usage rights for a defined period.

A lease is derived from an entitlement through the following process: the license agent presents the workload's identity evidence and the requested application/dimensions to the control plane. The control plane looks up the tenant's entitlement for that application, verifies that the requested dimensions are within the entitlement's limits, confirms quota availability (i.e., not all lease slots are consumed), and — if all checks pass — creates a lease record and returns a signed token. The lease inherits its dimension limits and policy from the entitlement, scoped to a specific workload and bounded in time.

A lease is:

- derived from an entitlement (inheriting dimensions, license modality, and policy)
- bound to a specific workload identity
- expressed as a signed token that can be validated locally
- subject to renewal, reissue, and revocation

A lease may have one of three types:

1. Fixed-term up-front lease: licenses are paid for based on a start/end time or on the allowed runtime from initialization. When the time runs out, the application stops.
2. Credit-based or unit-consumption model: a pre-purchased pool of value is depleted based on actual usage metrics. When the pool runs out, the application stops.
3. Audit-based or post-paid model: elastic scaling is allowed, with periodic financial reconciliation based on peak or average usage. The application does not stop, but the customer is billed for actual usage.

Token A token is the portable, cryptographically signed artifact that represents a lease. It contains claims describing the user, application, dimensions, binding identity, expiry, and enforcement policy. Tokens are short-lived and validated locally by the application or SDK without requiring a round-trip to the control plane.

Dimensions Dimensions are the metered attributes that define what a user is entitled to use and how usage is measured. Examples include channel count, frames per second, bandwidth, concurrent users, or process units. Dimensions appear in entitlements, leases, and tokens.

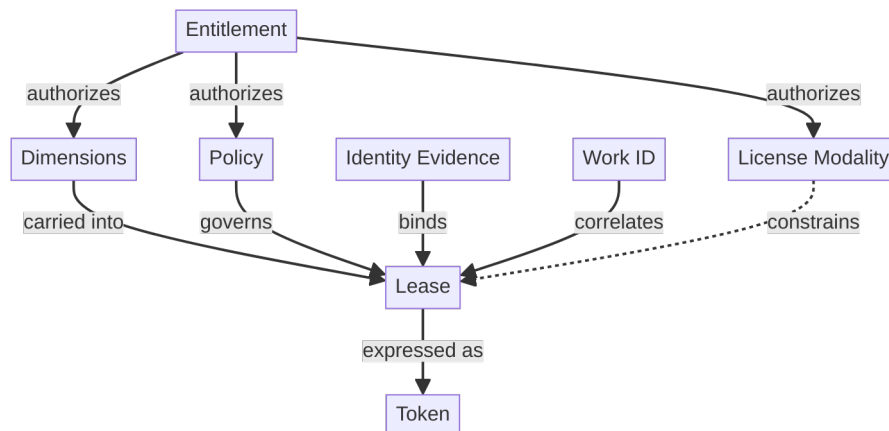
Identity Evidence Identity evidence is the proof that a workload presents to establish its identity when requesting or renewing a lease. Examples include OpenID Connect workload tokens, SPIFFE/SVID identities, X.509 workload certificates, or platform-issued service account tokens. Identity evidence replaces traditional hardware-bound identifiers such as hostnames or MAC addresses.

Policy A policy defines the behavioral rules that govern a lease at runtime. It includes renewal timing, offline grace periods, actions to take when grace is exhausted (e.g., drain, degrade, stop), and alert thresholds. Policies are defined by the vendor, optionally overridden by the user (if permitted), and enforced locally by the license agent or SDK.

Work ID A work ID is a correlation identifier associated with a specific deployment or run. It supports idempotency, audit trails, and reissue operations. It is not a binding — it is a tracking reference.

Concept Relationships

The relationships between these concepts follow a clear hierarchy:



Lease characteristics shown in the model:

- issued from an entitlement,
- bound via identity evidence,
- correlated via work ID,
- governed by policy,
- expressed as a token.

Token content shown in the model:

- dimensions,
- policy,
- binding,
- expiry.

Lifecycle Summary

The conceptual lifecycle of a license interaction is:

1. A user holds an entitlement for an application with defined dimensions and license modality.
2. A workload starts and presents identity evidence to request a lease.
3. The control plane validates the entitlement, verifies identity, and issues a signed token.
4. The workload validates the token locally and operates within the granted dimensions.
5. The license agent renews the lease before expiry.
6. If the workload fails over, a reissue transfers the lease to a new workload identity.
7. When usage ends or policy dictates, the lease is revoked.

Interaction Model

The DSL model assumes an interaction pattern where:

- The control plane is the authority for entitlements, lease issuance, policy, and centralized monitoring and observability.
- The workspace plane is where applications run and where tokens are validated locally.
- Communication between planes happens via well-defined API calls (REST) for all transactional operations.
- A push channel (WebSocket) MAY be used for real-time revocation and policy update delivery. The push channel is a complement to the REST API, not a requirement. If the push channel is unavailable or not implemented, revocation and policy changes take effect at the next renewal cycle (bounded by token TTL). Implementations that cannot support persistent connections remain fully functional.
- The license agent acts as the intermediary between the application and the control plane, handling all licensing operations transparently.

Platform Neutrality

This conceptual model is intentionally abstract. It does not prescribe specific cloud services, runtime environments, or implementation technologies. The concepts described here should apply equally to cloud, hybrid, and on-premises deployments.

Implementation-specific mappings (e.g., AWS Lambda, DynamoDB, KMS) are reserved for annexes and implementation guides.

07.02. Functional Components

Purpose

This section identifies the core services and functional building blocks that appear across implementations of the Dynamic Software Licensing model. Each component is described in terms of its responsibility, inputs, outputs, and relationship to other components.

Component Overview

The DSL model is composed of the following functional components:

Component	Plane	Primary Responsibility
Leases Service	Control	Issue, renew, reissue, and revoke leases
Entitlements Service	Control	Manage tenant entitlements, quotas, and policy
Keys / JWKS Service	Control	Manage signing keys and publish public keys
Usage Service	Control	Record and store usage events
Reissue Service	Control	Coordinate lease transfer during failover
Event Orchestrator	Control	Timer-driven events for expiry, alerts, and moves
API Gateway	Control	Entry point for all REST and WebSocket traffic
License Agent	Workspace	Mediate between application and control plane
Application SDK	Workspace	In-process token validation

Component	Plane	Primary Responsibility
End User Portal	Cross-cutting	Aggregated license visibility and self-service
Vendor Adapter	Control (optional)	Mirror usage to ISV back-office or marketplace

Component Descriptions

Leases Service The leases service is the central transactional component of the control plane. It handles:

- Lease issuance: validates entitlements, verifies identity evidence, signs tokens via the keys service, and persists the lease record.
- Lease renewal: extends the TTL of an active lease and returns a fresh token.
- Lease revocation: marks a lease as revoked and triggers push notification to the affected license agent.

The leases service enforces least-privilege access and writes to an append-only audit log for every operation.

Entitlements Service The entitlements service manages the source-of-truth records for what each tenant is permitted to use. It provides:

- Entitlement lookup by tenant, application, and dimensions.
- Quota enforcement to prevent over-issuance.
- Policy defaults per product or SKU, including grace policies and license modality constraints.

Entitlements are typically provisioned through onboarding or procurement workflows and are read by the leases service during issuance.

Keys / JWKS Service The keys service manages the cryptographic signing keys used to produce lease tokens. It:

- Interfaces with a key management system (e.g., HSM or managed key service) to sign tokens.
- Publishes the corresponding public keys at a well-known JWKS endpoint (`/well-known/jwks.json`).
- Supports key rotation via key identifiers (`kid` in token headers).

Applications and SDKs use the JWKS endpoint to validate tokens locally without contacting the leases service.

Usage Service The usage service records metered usage events submitted by license agents or SDKs. It:

- Accepts usage records with dimensions, quantities, timestamps, and correlation identifiers.
- Stores records in a hot store for operational queries and a cold store for analytics and billing.
- Supports idempotent submission to handle retries after network interruptions.

In a multi-vendor environment, usage data **MUST** be logically separated per ISV. Each vendor receives only the usage data associated with their own products. This prevents ISVs from accessing competitor usage data that may be collected within the same control plane deployment.

Usage data follows a metering model compatible with marketplace-style billing.

Reissue Service The reissue service coordinates the transfer of a lease from one workload to another, typically during failover or blue/green deployments. It:

- Creates a shadow lease for the target workload.
- Issues a cutover token to authorize the finalization.
- On finalization, revokes the original lease and activates the new one.
- Pushes a revocation event to the original license agent.

This two-phase approach (shadow + finalize) ensures atomic transfer without downtime.

Event Orchestrator The event orchestrator handles time-driven and event-driven operations. It:

- Triggers lease expiry processing.
- Sends renewal reminders and alert notifications.
- Manages move/reissue deadlines.
- Fans out events to notification channels (e.g., operations teams, monitoring systems).

API Gateway The API gateway is the unified entry point for all external communication with the control plane. It:

- Exposes REST endpoints for lease operations (POST /v1/leases, POST /v1/leases/{leaseId}:renew, POST /v1/leases/{leaseId}:reissue, POST /v1/leases/{leaseId}:finalizeReissue, DELETE /v1/leases/{leaseId}, POST /v1/usage-records).
- Exposes a WebSocket endpoint for push-based revocation and policy updates.
- Enforces authentication (IAM, OIDC, or custom), throttling, and perimeter security.
- Routes requests to the appropriate backend service.

License Agent The license agent is a sidecar, daemon, or CLI process that runs alongside the application in the workspace plane. It:

- Gathers identity evidence from the runtime environment.
- Calls the control plane API to issue, renew, and revoke leases.
- Delivers the signed token to the application (via file, socket, or environment variable).
- Maintains a WebSocket connection for receiving revocation and policy update events.
- Buffers usage events locally and uploads them when connectivity is available.
- Handles renewal scheduling, backoff, and grace policy enforcement.

The license agent abstracts all licensing complexity from the application.

Application SDK The application SDK is an in-process library that the application uses to:

- Validate the lease token locally using cached JWKS public keys.
- Expose granted dimension claims (e.g., channel count, FPS limit) to the application.
- React to token expiry and grace policy actions.

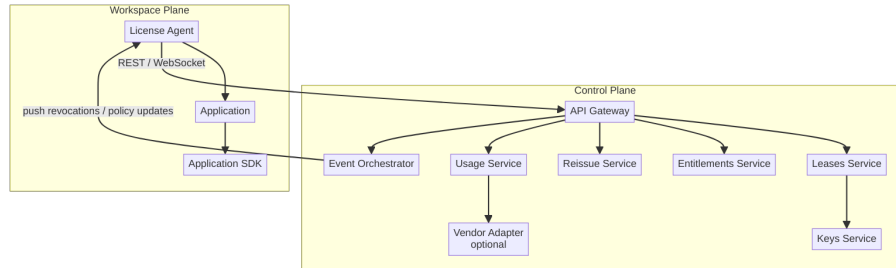
The SDK's role is to validate the token and make the granted entitlement data available to the application. Dimension enforcement — deciding whether to reject an operation that exceeds the licensed limits — is the responsibility of the application logic, not the SDK. This separation exists because dimension enforcement is highly application-specific, carries high liability, and cannot be generalized across different media functions. For example, the SDK exposes that the lease grants 4 channels; the application decides what happens when a 5th channel is requested.

The SDK is vendor-agnostic and open source.

End User Portal The end user portal provides a customer-facing interface for license management. It is described in detail in subchapter 07.05.

Vendor Adapter (Optional) The vendor adapter is an optional component that mirrors usage and lease events to an ISV's back-office system or a marketplace billing service. It enables vendors to reconcile usage data with their own billing and CRM systems.

Component Interaction Summary



Workspace-side responsibilities shown in the model:

- identity evidence collection,
- token delivery,
- renewal scheduling,
- usage buffering,
- local token validation,
- dimension data exposure.

07.03. Control Plane

Purpose

This section describes the responsibilities and characteristics of the Control Plane within the Dynamic Software Licensing model. The control plane is the central authority for entitlements, lease lifecycle management, cryptographic key operations, usage metering, and policy enforcement.

Role in the Architecture

The control plane serves as the trusted backend for all licensing decisions. It:

- Is the single source of truth for entitlements and active leases.
- Issues, renews, reissues, and revokes leases.
- Signs tokens using managed cryptographic keys.
- Records usage events for billing and analytics.
- Pushes revocation and policy update events to workspace-plane agents.

The control plane does not run application workloads. It is logically and operationally separated from the workspace plane.

Design Characteristics

Serverless and Elastic The control plane should favor elastic, low-overhead deployment patterns. A serverless implementation is one suitable approach because services execute only when invoked and scale elastically with demand.

Equivalent implementation models are also valid, provided they preserve the platform-agnostic API, trust, and operational behaviors defined by these best practices.

Event-Driven Asynchronous operations (lease expiry, renewal reminders, reissue deadlines, alerts) are handled through an event orchestration layer. Events trigger downstream processing without requiring polling as the primary control mechanism.

Multi-Tenant with Isolation The control plane supports multiple tenants with logical isolation. Each tenant's entitlements, leases, and usage data are separated. Cryptographic separation (per-tenant or per-product keys) may be applied depending on the deployment model.

API-First All control plane operations are exposed through well-documented REST APIs. There are no operations that require manual intervention or console-only access. This supports IaC integration, CI/CD pipelines, and programmatic license management.

Core Services

Entitlements Management Manages the catalog of what each tenant is entitled to use:

- Products and applications available to the tenant.
- Dimensions and limits per entitlement (e.g., 4 channels, 60 FPS).
- License modality (hourly, daily, monthly, annual, consumption-based).
- Default policies (grace periods, renewal windows, exhaustion actions).
- Quota tracking to prevent over-issuance.

Lease Lifecycle Handles the full lifecycle of leases through the following operations:

Operation	Description
ISSUE	Validate entitlement, verify identity evidence, sign and return a lease token
RENEW	Extend the TTL of an active lease, return a fresh token
REISSUE	Create a shadow lease on a new workload, finalize to transfer and revoke the original
REVOKE	Terminate a lease (admin action, policy trigger, or entitlement change)

Each operation writes to an append-only audit log.

Token Signing and Key Management

- Signing keys are held in a managed key service (e.g., HSM-backed).
- Tokens are signed at issuance and renewal.
- Public keys are published at a JWKS endpoint for local validation by workspace-plane components.
- Key rotation is supported via key identifiers (**kid**) in token headers.

Usage Metering

- Accepts usage records from license agents with dimensions, quantities, and timestamps.
- Stores records in both a hot store (operational queries) and a cold store (analytics, billing).
- Supports idempotent submission for reliability during network interruptions.
- Usage dimensions are modeled to be compatible with marketplace-style metering.

Event Orchestration

- Manages timers for lease expiry, renewal nudges, and reissue deadlines.
- Triggers alerts at configurable thresholds (e.g., 15 minutes of grace remaining).
- Fans out events to notification channels and downstream services.

Push Channel (Revocation and Policy Updates)

- May use a persistent push channel (for example WebSocket) to notify license agents.
- Pushes revocation events when a lease is revoked (admin, policy, or reissue finalization).
- Pushes policy updates when vendor changes grace or renewal parameters.
- Complements, but does not replace, the REST lifecycle API.
- If the agent is offline or the push channel is unavailable, revocation takes effect at the next renewal attempt or token expiry boundary.

API Surface

The control plane exposes the following primary endpoints:

Endpoint	Method	Purpose
/v1/leases	POST	Request a new lease
/v1/leases/{leaseId}:renew	POST	Renew an existing lease
/v1/leases/{leaseId}:reissue	POST	Initiate lease transfer (shadow)

Endpoint	Method	Purpose
/v1/leases/{leaseId}:finalizeReissue	POST	Complete lease transfer
/v1/leases/{leaseId}	DELETE	Revoke a lease
/v1/usage-records	POST	Submit usage events
/.well-known/jwks.json	GET	Retrieve public signing keys

Authentication is required for all endpoints (IAM, OIDC, or custom authorizer). Throttling and perimeter protection are applied at the gateway level.

Data Model (Logical)

The control plane manages the following logical data entities:

- Tenants: organizational identity and configuration.
- Products: application definitions and dimension catalogs.
- Entitlements: per-tenant, per-product allocation with dimensions, modality, and policy.
- Leases: active and historical lease records with status, binding, expiry, and audit trail.
- Usage Events: metered consumption records.
- Audit Log: append-only record of all lease operations.

Observability

The control plane provides:

- Logging of all API calls and service invocations.
- Metrics for lease issuance rates, renewal success/failure, revocation events, and usage volumes.
- Tracing across service boundaries for debugging and performance analysis.
- API-level auditing for compliance and security review.
- Per-tenant throttle and quota monitoring.

Depending on the system design, logs, metrics, and traces can be stored in a unified platform such as OpenTelemetry or Grafana stack.

Security Posture

- Zero-trust: no implicit trust between components; all requests are authenticated and authorized.
- Least-privilege: each service has only the permissions it needs.
- Encryption: data encrypted in transit (TLS) and at rest.
- Key management: signing keys are never exposed outside the key management service.

- Audit: all operations are logged with immutable audit trails.

Platform Neutrality

This description is intentionally platform-neutral. The control plane can be implemented using any combination of API gateways, compute services, databases, key management systems, and event buses that satisfy the described characteristics. Platform-specific reference implementations are provided in annexes.

07.04. Workspace Plane

Purpose

This section describes the responsibilities and characteristics of the Workspace Plane within the Dynamic Software Licensing model. The workspace plane is where licensed applications run and where licensing is enforced locally.

Role in the Architecture

The workspace plane is the runtime environment for applications. It is responsible for:

- Running the licensed application workload.
- Hosting the license agent that mediates between the application and the control plane.
- Providing identity evidence to prove workload authenticity.
- Validating lease tokens locally without requiring a constant connection to the control plane.
- Enforcing dimension limits and policy rules at the point of execution.

The workspace plane does not make licensing decisions. It enforces decisions made by the control plane using the information embedded in signed tokens.

Design Characteristics

Runtime Agnostic The workspace plane is not tied to a specific runtime environment. It may be:

- A cloud virtual machine (e.g., EC2 instance launched from an AMI).
- A container running in an orchestration platform (e.g., Kubernetes pod).
- A bare-metal or virtualized on-premises server.
- A hybrid deployment spanning cloud and on-premises infrastructure.

The licensing model adapts to the runtime through the identity evidence mechanism and the license agent's ability to operate in different environments.

Local Enforcement Once a lease token is delivered to the application, enforcement happens locally:

- The application or SDK validates the token signature using cached public keys (JWKS).
- Dimension limits (channels, FPS, bandwidth, etc.) are enforced in-process.
- Token expiry and grace policy are evaluated locally without contacting the control plane.

This design ensures that the application continues to operate during transient network interruptions, governed by the policy embedded in the token.

Identity-Based Binding Workloads are identified by their runtime identity rather than hardware attributes. This means:

- No binding to hostnames, MAC addresses, or hardware dongles.
- Identity evidence is derived from the platform (e.g., instance identity documents, IAM roles, Kubernetes service account tokens).
- Binding is verified at lease issuance and renewal, ensuring the token is used by the intended workload.

Core Components

License Agent The license agent is the primary workspace-plane component responsible for all licensing interactions. It runs as a sidecar process, daemon, or CLI alongside the application.

Responsibilities:

Function	Description
Identity collection	Gathers identity evidence from the runtime environment
Lease request	Calls the control plane to issue a new lease
Token delivery	Delivers the signed token to the application (file, socket, or environment variable)
Renewal	Schedules and executes lease renewal before token expiry
Revocation handling	Receives revocation events via WebSocket and triggers appropriate action
Usage reporting	Buffers usage events locally and uploads when connectivity is available
Grace enforcement	Monitors token expiry and applies grace policy actions (drain, degrade, stop)

Function	Description
Reissue coordination	Participates in lease transfer during failover scenarios

The license agent abstracts all licensing complexity from the application. From the application's perspective, it receives a valid token and enforces its claims.

Application SDK The SDK is an in-process library integrated into the application. It provides:

- Token parsing and signature validation using JWKS public keys.
- Exposure of granted dimension claims (e.g., licensed channel count, FPS limit) to the application.
- Expiry monitoring and grace policy evaluation.
- A simple interface for the application to query its current licensing state.

The SDK validates the token and exposes entitlement data. Dimension enforcement (deciding what happens when a limit is exceeded) is the responsibility of the application itself, since enforcement logic is application-specific and high-liability.

The SDK is designed to be vendor-agnostic and open source, enabling consistent integration across different ISV products.

Token Validation Flow The local validation flow within the workspace plane:

1. License agent delivers a signed token to the application.
2. SDK retrieves public keys from the JWKS endpoint (cached after first fetch).
3. SDK validates the token signature, expiry (**exp**), not-before (**nbf**), and audience (**aud**). (The audience claim identifies which application the token was issued for).
4. SDK extracts dimension claims and makes them available to the application.
5. Application enforces dimensions during operation.
6. On token refresh (renewal), the license agent delivers an updated token and the SDK re-validates.

No database or control plane call is in the hot path of token validation.

Operational Scenarios

Normal Operation

- License agent starts with the workload, collects identity evidence, requests a lease.
- Token is delivered to the application; SDK validates and enforces.
- License agent renews the lease on a schedule defined by the policy (**renewBeforeSeconds**).

- Usage events are reported periodically.

Network Interruption

- Application continues operating on the current valid token.
- License agent retries renewal with exponential backoff and jitter.
- If the token expires, the offline grace period begins.
- During grace, the application continues operating (full or degraded, per policy).
- If the outage is expected to outlast the configured grace period, the customer MAY request an out-of-band lease extension or a temporary offline token from the vendor or control-plane operator. The extension artifact is separately signed, manually entered through an administrative interface such as CLI or API, and later reconciled against the primary lease once connectivity is restored.
- If grace is exhausted, the configured exhaustion action applies (drain, degrade, or stop).
- When connectivity returns, the license agent renews the lease and resumes normal operation.
- Buffered usage events are uploaded.

Failover (Reissue)

- A new workload instance starts (e.g., backup EC2 in a blue/green deployment).
- The new license agent requests a reissue from the control plane, referencing the original lease.
- The control plane creates a shadow lease and returns a cutover token.
- On finalization, the original lease is revoked and the new lease is activated.
- The original license agent receives a revocation push and the original workload drains or stops.

Revocation

- The control plane pushes a revocation event to the license agent via WebSocket.
- The license agent notifies the application/SDK.
- The application executes the configured exhaustion action (drain, degrade, or stop).
- If the license agent is offline when revocation occurs, the revocation is not lost. Because tokens are short-lived, the license agent must eventually contact the control plane to renew. At that point, the control plane

rejects the renewal and the revocation takes effect. The maximum delay is bounded by the token's TTL.

Deployment Patterns

The workspace plane supports multiple deployment patterns:

Pattern	License Agent Model	Identity Evidence
Cloud Instance (eg AWS Machine Image (AMI) on EC2)	Daemon or sidecar process	Instance Identity Document (IID) + IAM role
Kubernetes	Sidecar container	Service Account JWT
On-premises	Daemon or CLI	Platform-specific identity or certificate-based
Hybrid	Varies by environment	Mixed evidence types

The license agent adapts its identity collection and communication patterns to the deployment environment.

Security Considerations

- Tokens are validated cryptographically; no trust is placed in the delivery mechanism alone.
- Identity evidence is verified by the control plane at every issuance and renewal.
- The license agent communicates with the control plane over TLS. Private connectivity (e.g., VPN, private endpoints) is supported for environments that require it.
- The SDK never stores or transmits the signing private key; it only uses public keys for validation.
- Token TTLs are kept short to limit exposure if a token is compromised.

Platform Neutrality

This description applies to any runtime environment that can host an application alongside a license agent process and provide some form of workload identity. Platform-specific deployment guides and identity evidence formats are documented in annexes.

07.05. End User Portal

Purpose

This section describes the role of an end-user-facing management interface within the Dynamic Software Licensing model. The End User Portal provides aggregated visibility, self-service capabilities, and operational controls for customers managing licenses across one or more vendors.

Role in the Architecture

The End User Portal is a cross-cutting component that sits above the control plane and workspace plane. It does not participate in the runtime licensing flow (token issuance, renewal, or validation). Instead, it provides a human-facing layer for:

- Viewing and managing license entitlements and active leases.
- Monitoring usage and capacity across vendors and applications.
- Performing self-service operations (e.g., releasing a stuck license, ordering additional capacity).
- Supporting financial and operational decision-making.

Core Capabilities

Multi-Vendor Aggregation Customers in broadcast and media workflows typically use products from multiple ISVs. The portal provides a unified view across all vendors, showing:

- Active entitlements per vendor and product.
- Current lease status (active, expired, revoked) per workload.
- Dimension utilization (e.g., 3 of 4 licensed channels in use).
- License modality and renewal schedule.

This eliminates the need to log into separate vendor portals to understand overall licensing posture.

Real-Time License Status The portal reflects the current state of the licensing system:

- Which leases are active and on which workloads.
- Remaining capacity (available entitlements not yet leased).
- Upcoming expirations and renewal deadlines.
- Feature flags and dimension limits per active lease.

Data is sourced from the control plane's lease and entitlement records.

Self-Service Operations The portal enables customers to perform operational actions without contacting the vendor or SI:

Operation	Description
Force release	Release a license from a failed or terminated instance, returning it to the available pool
View history	Review lease history, including issuance, renewals, reissues, and revocations
Request capacity	Initiate a request for additional entitlements or dimension increases
Cancel / downgrade	Request cancellation or reduction of entitlements (subject to vendor approval)
Alert configuration	Set notification preferences for expiry warnings, grace period triggers, and usage thresholds

Usage and Financial Visibility The portal provides data to support cost management and forecasting:

- Usage summaries by application, dimension, time period, and environment tag (production, staging, dev).
- Cost breakdowns mapped to licensing dimensions and modalities.
- Cost estimation for pre-production planning.
- Historical usage trends for capacity planning.
- Export capabilities for integration with financial systems and reporting tools.

Emergency Operations For production-critical scenarios, the portal supports:

- Forced license release when a workload has terminated without proper revocation.
- Visibility into grace period status for workloads experiencing connectivity issues.
- Rapid capacity burst requests for unplanned scaling events.

Personas Served

The End User Portal serves multiple personas within a customer organization:

Persona	Primary Use
Operations team	Monitor active leases, release stuck licenses, verify feature availability
Finance officer	Review usage costs, forecast spend, audit license compliance

Persona	Primary Use
License manager	Compare models, track pool utilization, manage renewals
Director of engineering	Plan capacity for events, monitor burst usage, review cross-vendor posture
Broadcast engineer	Check license status for assigned workloads, verify dimension limits

Interaction with the Control Plane

The portal interacts with the control plane through the same API surface used by license agents, plus additional administrative and reporting endpoints:

- Read entitlements and lease status via query APIs.
- Invoke revocation for force-release operations.
- Access usage and analytics data from the reporting layer.
- Receive event notifications for status changes.

The portal does not bypass the control plane’s authorization model. All operations are subject to the same authentication, tenant isolation, and audit logging.

Interface Considerations

Multi-Interface Support The portal complements (but does not replace) the API and CLI interfaces. Customers who prefer programmatic access can achieve the same outcomes through the API. The portal provides a visual layer for scenarios where a graphical overview is more effective.

Platform Neutrality

The End User Portal is described as a functional capability, not a specific technology. It may be implemented as a web application, integrated into an existing operations dashboard, or provided as a vendor-hosted service. The key requirement is that it surfaces the data and operations described above in a way that is accessible to the personas it serves.

07.06. Trust Model

Purpose

This section defines how trust is established, maintained, and verified across the Dynamic Software Licensing model. It describes the trust boundaries between components, the mechanisms used to prove identity and integrity, and the assumptions that underpin secure licensing operations.

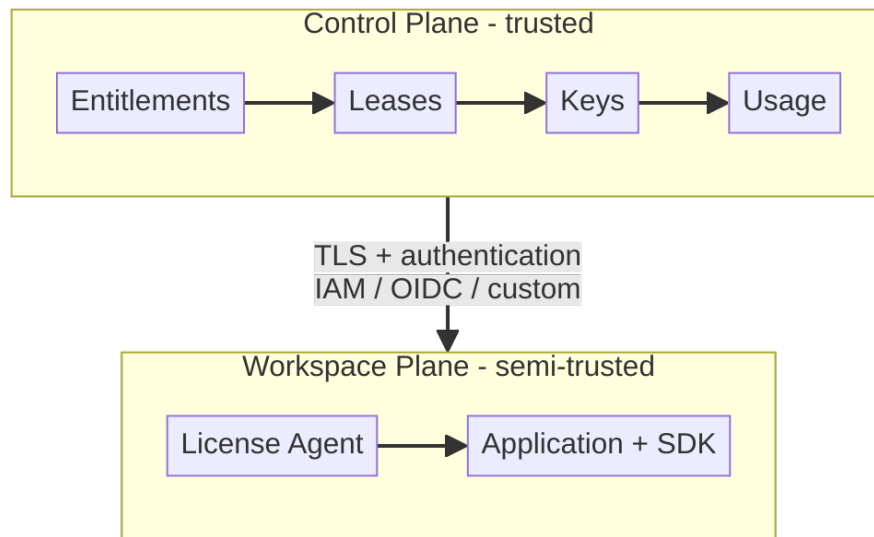
Trust Principles

The DSL trust model is based on the following principles:

1. Zero implicit trust: no component trusts another by default. Every interaction requires authentication and verification.
2. Short-lived credentials: lease tokens have limited lifetimes, reducing the window of exposure if a token is compromised.
3. Cryptographic verification: trust is established through cryptographic signatures, not through network location or shared secrets alone.
4. Identity-based binding: workloads are identified by verifiable runtime identities, not hardware attributes.
5. Least privilege: each component has only the permissions necessary for its function.

Trust Boundaries

The DSL model defines the following trust boundaries:



Control plane trust basis:

- managed infrastructure,
- key custody,
- audit logging.

Workspace plane trust basis:

- identity evidence,
- signed tokens,
- local validation.

Control Plane Trust The control plane is the root of trust for the licensing system. It:

- Holds the private signing keys (via a managed key service).
- Makes all entitlement and lease decisions.
- Maintains the authoritative record of active leases and their status.
- Is responsible for the integrity of the audit log.

Trust in the control plane is established through:

- Secure infrastructure management (access controls, encryption, monitoring).
- Key custody in a hardware-backed key management service.
- Immutable audit trails for all operations.

Workspace Plane Trust The workspace plane is semi-trusted. It does not make licensing decisions, but it is trusted to:

- Present valid identity evidence during lease requests.
- Enforce the dimension limits and policy rules embedded in the token.
- Report usage accurately.

Trust in the workspace plane is bounded by:

- Token expiry: even if a workload is compromised, the token expires and must be renewed (which requires valid identity evidence).
- Signature verification: the SDK validates that the token was signed by the control plane.
- Binding claims: the token contains binding information (e.g., IAM role ARN) that can be checked against the runtime environment.

Identity Evidence

Identity evidence is the mechanism by which a workload proves its identity to the control plane. The DSL model supports multiple forms of identity evidence depending on the runtime environment:

Environment	Identity Evidence	Verification
Cloud VM (e.g., EC2)	Instance Identity Document (IID) + PKCS7 signature	Control plane verifies the IID signature against the cloud provider's public key
Kubernetes	Service Account JWT	Control plane validates the JWT against the cluster's OIDC issuer
On-premises	Platform certificate or custom identity token	Control plane validates against a pre-registered trust anchor

Environment	Identity Evidence	Verification
Hybrid	Mixed evidence types per workload	Control plane supports multiple evidence types per tenant

Identity evidence is presented at:

- Lease issuance (ISSUE): to establish the initial binding.
- Lease renewal (RENEW): to re-verify that the same workload is still requesting the lease.
- Lease reissue (REISSUE): to establish the identity of the new target workload.

Identity vs. Hardware Binding Traditional licensing often binds to hardware identifiers (hostnames, MAC addresses, hardware dongles). The DSL model explicitly replaces this with identity-based binding because:

- Hardware identifiers are not stable in cloud and containerized environments.
- Identity evidence is cryptographically verifiable.
- Identity-based binding supports failover, scaling, and migration without manual license re-binding.

Token Trust Chain

The trust chain for a lease token follows this path:

1. The control plane's key management service holds the private signing key.
2. The leases service requests a signature from the key management service when issuing or renewing a token.
3. The signed token is delivered to the license agent, which passes it to the application.
4. The application SDK retrieves the public key from the JWKS endpoint (cached locally).
5. The SDK verifies the token signature, confirming it was issued by the control plane.
6. The SDK checks the token claims: expiry, not-before, audience, binding, and dimensions.

At no point does the private key leave the key management service. The trust chain is:

```

Key Management Service (private key)
  → Leases Service (requests signature)
    → Token (signed artifact)
      → SDK (validates with public key from JWKS)

```

Revocation Trust

Revocation is a trust-sensitive operation. The model handles it as follows:

- Active revocation: the control plane pushes a revocation event to the license agent via WebSocket. The agent acts immediately.
- Passive revocation: if the agent is offline, the revocation takes effect when the token expires (since renewal will fail) or when the agent reconnects and receives the event.
- Token TTL as a safety net: short token lifetimes ensure that even without active revocation, a compromised or revoked lease cannot be used indefinitely.

Grace Period Trust

During a grace period (when the license agent cannot reach the control plane to renew), the system operates on a trust-but-verify basis:

- The application continues operating based on the locally held token and its embedded policy.
- The grace period is bounded (defined by `offlineGraceSeconds` in the policy).
- When grace expires, the configured exhaustion action applies (drain, degrade, or stop).
- When connectivity returns, the license agent must successfully renew to continue operation.
- Upon successful renewal after a grace period, operation **SHOULD** resume automatically without manual intervention. This is the expected default behavior for programmatically configured deployments. Implementations that require manual restart after grace recovery **MUST** document this clearly.

The grace period represents a deliberate trade-off between production continuity and strict license enforcement. The parameters of this trade-off are controlled by the vendor (with optional tenant overrides).

Grace Period Configuration Guidance The `offlineGraceSeconds` value may be configured as either:

- A fixed pre-defined value (e.g., 1800 seconds / 30 minutes) set by the vendor as a default.
- A function-based value derived from operational context (e.g., a multiple of the token TTL, or proportional to the application's typical session duration).

The choice between fixed and function-based grace periods is a vendor policy decision. The following guidelines apply:

Consideration	Guidance
Short-lived workloads (event-driven, serverless)	Shorter grace periods (minutes) are appropriate; the workload can restart quickly
Long-running production chains (live broadcast)	Longer grace periods (30–60 minutes) protect against transient network issues
High-value workloads	Grace periods should be long enough to allow human investigation before disruption
Metered / consumption-based entitlements	Grace periods should be bounded to limit unbilled usage during outages

Vendors **SHOULD** document their grace period defaults and whether tenant overrides are permitted. Tenants **SHOULD NOT** need to guess how much time they have before an exhaustion action fires.

Anti-Tampering Considerations

- Tokens are cryptographically signed; modification invalidates the signature.
- The SDK validates signatures before accepting any token claims.
- The JWKS endpoint is served over TLS to prevent key substitution.
- Token claims include a unique identifier (`jti`) for anti-replay protection.
- The audit log records all issuance and renewal events, enabling detection of anomalous patterns.

Trust Between Participants

The DSL model involves multiple organizational participants, each with different trust relationships:

Relationship	Trust Mechanism
Customer → Control Plane	Authentication (IAM/OIDC), TLS, audit trails
Control Plane → Workload	Identity evidence verification at issuance and renewal
Workload → Token	Cryptographic signature validation
Vendor → Control Plane	Key custody, entitlement management, policy definition
SI → Control Plane	API access with scoped permissions for deployment automation
Customer → Vendor	Entitlement agreement, usage transparency via portal and analytics

Multi-Vendor Trust in a Shared Control Plane In a multi-vendor environment, the expectation is that a single DSL control plane serves multiple ISVs. This raises the question of how each ISV can trust that their licensing logic is enforced correctly and that their entitlements are not tampered with.

The DSL model addresses this through:

1. Cryptographic isolation: each ISV MAY have dedicated signing keys managed within the control plane's key management service. Tokens signed with ISV-specific keys cannot be forged or repurposed across vendors.
2. Logical tenant and vendor separation: entitlements, leases, and usage data are scoped per vendor and per tenant. No ISV can access, modify, or observe another ISV's entitlement records or usage data.
3. Immutable audit logging: all lease operations (issue, renew, revoke) are recorded in an append-only audit log. ISVs can access audit data for their own products to verify correct enforcement.
4. Vendor-specific policy encapsulation: each ISV defines their own entitlement catalog, dimension definitions, policies (grace, renewal, exhaustion), and pricing. These are stored and enforced per-vendor within the control plane. The control plane applies the correct vendor's policy when processing a lease request for that vendor's product.
5. Optional call-home / audit webhook: the control plane MAY support an outbound webhook or event stream per ISV, allowing vendors to receive real-time notifications of lease operations for their products. This enables independent auditing without requiring ISVs to trust the control plane operator blindly.
6. Concurrent lease enforcement: the control plane enforces quota constraints (e.g., a license with dimension of 1 can only have one active lease at a time). Lease binding to a verified workload identity prevents a single license from being concurrently leased to multiple workloads.

This architecture ensures that adding a new vendor to a shared control plane does not require changes to the API or message flow. Each ISV's business logic, policy, and security boundaries are encapsulated within their entitlement and key configuration.

Platform Neutrality

The trust mechanisms described here (signed tokens, identity evidence, key management, TLS) are available across cloud, hybrid, and on-premises environments. The specific implementations of identity evidence and key management vary by platform, but the trust model itself is platform-neutral.

07.07. Responsibility Model

Purpose

This section defines which participant or component is responsible for which aspect of the Dynamic Software Licensing model. It clarifies the boundaries between vendors (ISVs), customers, system integrators (SIs), the control plane, and the workspace plane to avoid ambiguity in implementation and operation.

Participants

The DSL model involves the following participants:

Participant	Role
Independent Software Vendor (ISV)	Produces the licensed application; defines entitlement catalogs, dimensions, pricing, and default policies
Customer	Consumes the licensed application; manages workloads, monitors usage, and controls spend
System Integrator (SI)	Deploys and integrates licensed applications on behalf of customers; automates licensing via IaC
Control Plane Operator	Operates the licensing control plane infrastructure (may be the ISV, a shared service, or a managed platform)
Workspace Operator	Operates the runtime environment where applications and license agents run (typically the customer or SI)

Responsibility Matrix

Entitlement Lifecycle

Responsibility	ISV	Customer	SI	Control Plane
Define product catalog and dimensions	Owns			
Define pricing and modalities	Owns			

Responsibility	ISV	Customer	SI	Control Plane
Define default policies (grace, renewal, exhaustion)	Owns			
Provision entitlements for a tenant	Initiates (direct sale)	Initiates (self-service or marketplace)	Initiates (on behalf of customer)	Stores and enforces
Request entitlement changes (upgrade, downgrade)		Requests	Requests	Processes
Approve entitlement overrides (e.g., extended grace)	Approves	Requests	Requests (on behalf of customer)	Applies
Monitor entitlement utilization		Monitors	Monitors	Provides data

Lease Lifecycle

Responsibility	License Agent	Control Plane	Application/Software	Customer/Operator	SI
Collect identity evidence	Owns				
Request lease (ISSUE)	Initiates	Validates and issues			
Deliver token to application	Delivers		Receives		

Responsibility	License Agent	Control Plane	Application/SDN	Customer/OpS	SI
Validate token signature and claims			Owns		
Enforce dimension limits			Owns		
Schedule and execute renewal (RENEW)	Initiates	Validates and extends			
Initiate reissue for failover (REISSUE)	Initiates	Creates shadow, finalizes		Triggers (manual or auto-mated)	Triggers (auto-mated via IaC/orchestration)
Handle revocation event	Receives and acts	Pushes event	Executes exhaustion action	Monitors	Monitors
Apply grace policy on connectivity loss	Enforces		Enforces locally	Monitors	Monitors

Usage and Metering

Responsibility	License Agent	Control Plane	Customer	ISV	SI
Collect usage events	Owns				
Buffer usage during connectivity loss	Owns				
Submit usage records	Submits	Receives and stores			

Responsibility	License Agent	Control Plane	Customer	ISV	SI
Provide usage analytics and reports		Owns	Consumes	Consumes	Consumes (on behalf of customer)
Reconcile usage with billing		Provides data	Reviews	Reconciles	Reviews (on behalf of customer)
Mirror usage to market-place or back-office		Owns (via Vendor Adapter)		Receives	
Dispute or query usage discrepancies		Provides audit data	Raises	Responds	Raises (on behalf of customer)

Security and Trust

Responsibility	Control Plane	License Agent	Application/SDK	Customer	SI
Manage signing keys	Owns				
Sign lease tokens	Owns				
Publish JWKS public keys	Owns				
Validate token signatures			Owns		
Present identity evidence		Owns			

Responsibility	Control Plane	License Agent	Application/ISDK	Customer	SI
Verify identity evidence	Owns				
Enforce TLS for all communication	Enforces	Enforces		Enforces (infrast- ructure)	Enforces (infrast- ructure)
Maintain audit logs	Owns			Reviews	Reviews (on behalf of customer)
Define security policies			Defines	Overrides (if permitted)	Advises customer
Monitor for anomalies	Monitors		Monitors	Monitors	Monitors (on behalf of customer)
Manage network security (VPN, private endpoints)	Supports			Configures	Configures (on behalf of customer)

Deployment and Integration

Responsibility	ISV	SI	Customer	Control Plane
Provide IaC modules (Terraform/CFN)	Provides	Extends and customizes		
Include licensing in deployment automation		Integrates	Integrates (if self-deploying)	

Responsibility	ISV	SI	Customer	Control Plane
Provide license agent binaries/packages	Provides	Distributes to customer environments		
Provide application SDK	Provides		Integrates (if building custom apps)	
Configure license agent for the environment		Configures	Configures (if self-managing)	
Operate the control plane	Operates (or shared)			Operates
Operate the workspace environment		Operates (on behalf of customer)	Operates (if self-managing)	
Provide CI/CD integration examples	Provides	Provides		
Troubleshoot licensing issues in deployment	Supports (product-level)	Troubleshoots (environment-level)	Reports issues	Provides diagnostics

End User Portal

Responsibility	Control Plane	Customer	ISV	SI
Provide portal interface	Provides (or ISV)		Provides (or shared)	
View license status and usage		Views	Views	Views (on behalf of customer)
Force-release stuck licenses		Executes		Executes (on behalf of customer)
Request additional capacity		Requests	Approves	Requests (on behalf of customer)

Responsibility	Control Plane	Customer	ISV	SI
Configure alerts and notifications		Configures		Configures (on behalf of customer)
Provide multi-vendor aggregated view	Provides	Consumes	Contributes data	Consumes (on behalf of customer)
Onboard new vendor products into portal	Provides integration		Registers products	Coordinates

Responsibility Boundaries

Vendor (ISV) Boundary The vendor is responsible for:

- Defining what is licensed (products, dimensions, modalities).
- Setting the rules of licensing (policies, grace periods, exhaustion actions).
- Providing the tools for integration (SDK, license agent, IaC modules).
- Providing detailed usage reports to the customer.

The vendor is not responsible for:

- Operating the customer's workloads.
- Managing the customer's infrastructure or identity systems.
- Making runtime enforcement decisions (these are delegated to the application logic, using claims exposed by the license token and SDK).

Customer Boundary The customer is responsible for:

- Operating their workloads and ensuring the license agent is deployed.
- Presenting valid identity evidence from their runtime environment.
- Monitoring their license usage and managing spend.
- Responding to alerts and managing capacity for events.

The customer is not responsible for:

- Defining licensing terms or policies (though they may request overrides).
- Operating the control plane (unless self-hosted).
- Implementing the SDK or license agent (these are provided by the vendor).

System Integrator Boundary The SI is responsible for:

- Deploying and configuring licensed applications in the customer's environment.
- Integrating licensing into IaC and CI/CD pipelines.

- Providing operational support for licensing workflows.
- Acting on behalf of the customer for provisioning, capacity requests, monitoring, and troubleshooting where the SI is the engaged operational partner.
- Advising the customer on policy settings, modality choices, and capacity planning.

The SI is not responsible for:

- Defining licensing terms (these are set by the ISV).
- Operating the control plane.
- Developing the SDK or license agent.
- Approving entitlement changes (this remains with the ISV).

Control Plane Boundary The control plane is responsible for:

- All licensing decisions (issuance, renewal, reissue, revocation).
- Key management and token signing.
- Usage recording and analytics.
- Audit logging and compliance data.
- Push notifications for revocation and policy updates.

The control plane is not responsible for:

- Running application workloads.
- Enforcing dimensions at runtime (this is the application logic's responsibility, with the SDK exposing the granted claims).
- Managing the customer's infrastructure.

Shared Responsibilities

Some responsibilities are shared between participants:

Area	Shared Between	Nature of Sharing
Policy definition	ISV + Customer (+ SI as advisor)	Vendor sets defaults; customer may request overrides; SI advises on appropriate settings
Entitlement provisioning	ISV + Customer + SI	Any party may initiate depending on the commercial relationship
Deployment automation	ISV + SI + Customer	Vendor provides modules; SI integrates and customizes; customer may self-integrate

Area	Shared Between	Nature of Sharing
Security monitoring	Control Plane + Customer + ISV + SI	Each monitors their own boundary; SI monitors on behalf of customer where engaged
Usage reconciliation	Control Plane + ISV + Customer + SI	Control plane provides data; ISV and customer reconcile; SI may review on customer's behalf
Observability	Control Plane + Customer + SI	Control plane provides metrics and logs; customer or SI integrates into monitoring
Troubleshooting	ISV + SI + Control Plane	ISV handles product issues; SI handles environment issues; control plane provides diagnostics

Platform Neutrality

This responsibility model applies regardless of the underlying platform. The specific tools and services used by each participant vary by environment, but the allocation of responsibilities remains consistent across cloud, hybrid, and on-premises deployments.

08. Conformance and Profiles

Purpose

This chapter defines how the Dynamic Software Licensing best practices can express implementation targets, maturity levels, and profile-based adoption paths.

Its purpose is to create a structured way to describe what it means to align with the best practices at different levels of capability and complexity.

Role Within the Best Practices

The DSL best practices are not a formal standard, but they are intended to support standard-like consistency.

This chapter helps by introducing a structured view of:

- what kinds of implementations are in scope,
- what conformance may mean in practice,
- how different levels of maturity or completeness can be expressed,
- how staged adoption can be supported without forcing all participants into the same starting point.

Basis for This Chapter

The current DSL material already points toward different access patterns and maturity levels, including varying support for documented processes, custom integrations, standard process models, CLI or API usage, and self-service capabilities.

This chapter provides the structure for those ideas.

Scope of This Chapter

This chapter defines:

- conformance targets,
- conformance levels,
- profile-based adoption structure.

It is not presented as a full certification framework.

How to Use This Chapter

This chapter is easiest to use in the following order:

1. Identify the conformance target in Section 08.01: control plane, workspace plane, interface, or end-to-end system.
2. Use Section 08.02 to decide which profile best matches the current implementation context.

3. Read the matching profile in Sections 08.03 to 08.05 as the practical expectation bundle.
4. Use Annex A to test whether the claim is supported by observable behavior.

In practical terms, this chapter answers four questions:

- what exactly is being assessed,
- which profile best fits the implementation,
- what level of maturity that profile implies,
- how that claim can be checked in review.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Conformance Targets Defines the kinds of implementations or components to which conformance language may apply.

02. Conformance Levels Defines how alignment with the best practices may be expressed at different levels.

03. Profile A – Foundational Defines a baseline profile.

04. Profile B – Cloud Native Defines a more advanced profile.

05. Profile C – Advanced Interoperable Defines an advanced profile.

Relationship to Other Chapters

This chapter is closely related to:

- conventions and normative language,
- the reference model,
- normative requirements,
- operational profiles,
- conformance checklist annexes.

It provides a structure that later requirement language can map onto.

Chapter Positioning

This chapter provides the adoption and interpretation framework for implementations at different levels of maturity and interoperability.

For review purposes, it should be assessed as the bridge between architectural intent, requirement depth, and practical implementation profiles.

08.01. Conformance Targets

Purpose

This section defines the kinds of implementation surfaces to which DSL conformance language can reasonably apply.

Why Conformance Targets Matter

The best practices describe a complete operating model, but not every participant implements every part of that model. Conformance therefore needs to state what exactly is being assessed.

Without this distinction, profile claims become ambiguous. A control plane, a license agent, an administrative portal, and a vendor integration may each align with the best practices in different ways.

Primary Conformance Targets

The DSL best practices should be understood as applying to the following target types:

- control-plane implementations,
- workspace-plane implementations,
- supporting operator and automation interfaces,
- end-to-end deployment patterns that combine these elements.

Control-Plane Conformance

A control-plane implementation is the system that manages entitlements, leases, policy, revocation state, token issuance, and usage intake.

Conformance at this layer concerns:

- lifecycle semantics,
- identity verification and binding,
- signed artifact issuance,
- policy handling,
- auditability and observability,
- documented API behavior.

Workspace-Plane Conformance

A workspace-plane implementation is the runtime-side component that requests, validates, enforces, renews, and reports according to DSL semantics.

Conformance at this layer concerns:

- local validation behavior,
- policy enforcement,
- identity presentation,

- renewal and retry behavior,
- bounded continuity and grace handling,
- usage attribution where applicable.

Interface Conformance

Administrative interfaces, SDKs, CLI tools, portals, and automation adapters may also be assessed for DSL alignment where they expose or rely on canonical lifecycle operations.

Conformance here concerns whether these tools preserve the same semantics rather than whether they share a specific implementation style.

System-Level Conformance

Some conformance statements only make sense when assessed across the combined system. Examples include:

- end-to-end traceability,
- cutover and reissue behavior,
- tenant isolation,
- coherence between API, token, policy, and operational behavior.

Out of Scope

Conformance in this document does not imply:

- legal certification,
- commercial fairness,
- product feature parity across vendors,
- a requirement that every implementation expose identical internal architecture.

Section Positioning

This section defines the implementation boundaries to which the profile and checklist logic of the DSL best practices may be applied.

08.02. Conformance Levels

Purpose

This section defines how different degrees of DSL alignment can be expressed without reducing the best practices to a binary pass or fail model.

Role of Conformance Levels

Organizations and vendors will adopt the DSL model in stages. A structured conformance view must therefore distinguish foundational alignment from more mature automation, interoperability, and operational capability.

The purpose of the levels is to support realistic adoption sequencing while keeping the direction of travel clear.

Level Interpretation

Conformance levels should be read as cumulative. Each higher level assumes the capabilities of the previous one and adds stronger expectations.

The levels defined in this chapter are expressed through three implementation profiles:

- Foundational,
- Cloud Native,
- Advanced Interoperable.

Quick Profile Selection

The following table is intended as the fastest way to understand which profile best matches an implementation context.

If the implementation mainly needs...	Most likely profile
a first credible production baseline with signed tokens, identity binding, and bounded continuity	Foundational
API-first automation, dynamic workloads, and operations-integrated lifecycle handling	Cloud Native
repeatable cross-vendor semantics, hybrid portability, and mature interoperability evidence	Advanced Interoperable

Practical Comparison

Dimension	Foundational	Cloud Native	Advanced Interoperable
Main goal	Replace manual or hardware-bound licensing with a credible DSL baseline	Make DSL a normal part of automated cloud or orchestrated operations	Make DSL reliable across products, teams, partners, and environments
Typical runtime shape	One main environment, limited hybrid complexity	Dynamic cloud, container, or automation-first workloads	Hybrid, DR, multi-product, or ecosystem-facing environments
Identity expectations	Verifiable workload identity is present and used consistently	Identity evidence supports dynamic runtime behavior and stronger automation	Identity semantics remain stable across different platforms and partner boundaries
Operational focus	Predictable issue, renew, revoke, and bounded grace handling	Retry, renewal, reissue, and usage behavior remain reliable under workload churn	Portability, traceability, tenant isolation, and semantic consistency remain dependable at scale
Integration depth	Limited but real automation	CI/CD, IaC, and operational tooling are normal integration points	External partners and multiple internal teams can rely on the same semantics
Best fit for	first DSL production rollout	cloud-native rollout	mature interoperability target

How to Read a Profile Claim

A profile claim is useful only when it answers three practical questions:

1. Which conformance target is being claimed: control plane, workspace plane, interface, or end-to-end system?
2. Which operational behaviors can be demonstrated, not just designed?
3. Which later profile expectations are intentionally out of scope for the current implementation?

Without those three clarifications, a profile statement is too vague to help operators, customers, or integration partners.

Assessment Logic

When an implementation claims alignment with a profile, the claim should cover the relevant target surface identified in Section 08.01 and should be supportable by documented behavior, not only architectural intention.

Reasonable evidence may include:

- documented API or workflow behavior,
- observed lifecycle handling,
- token and policy structures,
- operational controls and audit records,
- integration artifacts used in deployment and recovery.

What the Levels Are Not

These levels are not intended to become:

- a formal certification program,
- a maturity score for the entire business,
- a replacement for detailed security or legal review,
- a mandate that every implementation immediately target the highest profile.

Relationship to Requirements

The conformance levels rely on Chapter 09 for substantive requirements. The levels interpret those requirements as practical adoption bundles rather than redefining them.

For review and implementation assessment, Annex A is the most direct companion to this chapter because it translates the profiles into compact checklist statements.

Section Positioning

This section establishes the staged interpretation model that the profile subchapters and conformance checklist build on.

08.03. Profile A - Foundational

Purpose

This section defines the baseline profile for a practical DSL-aligned implementation.

Profile Intent

The Foundational profile represents the minimum credible implementation shape for adopting the DSL model. It is intended for organizations or vendors that need a stable baseline before moving toward more advanced automation or interoperability.

Core Expectations

An implementation aligned with this profile should provide at least:

- documented lifecycle operations for issue, renew, revoke, and inspection,
- signed runtime artifacts with local validation capability,
- identity-based binding semantics,
- explicit policy handling for renewal and grace,
- basic auditability and operational visibility,
- a path away from manual or hardware-bound licensing.

Typical Characteristics

- one primary deployment environment or a limited hybrid footprint,
- limited but real automation,
- clear operator ownership,
- moderate observability,
- bounded offline continuity rather than full event-driven control integration.

Suitable Adoption Scenarios

This profile is typically suitable for:

- first production-grade DSL deployments,
- vendors replacing legacy activation with a control-plane model,
- customers piloting identity-based licensing in a narrow scope,
- continuous or test-oriented workloads with controlled risk.

Boundaries

This profile does not imply advanced cross-vendor interoperability, broad self-service, or highly dynamic multi-environment portability. Those belong to later profiles.

Section Positioning

This profile defines the baseline threshold at which DSL alignment becomes operationally credible.

08.04. Profile B - Cloud Native

Purpose

This section defines the profile for implementations that apply the DSL model in a strongly automated, cloud-native, and operations-integrated manner.

Profile Intent

The Cloud Native profile builds on the Foundational profile and adds stronger expectations for automation, observability, and dynamic workload handling.

Core Expectations

An implementation aligned with this profile should provide:

- API-first lifecycle handling as the normal operating mode,
- integration with deployment automation and infrastructure workflows,
- stronger identity evidence flows for dynamic workloads,
- mature renewal, retry, and grace handling under transient failure,
- inspectable runtime state and usage reporting suitable for operations and finance,
- support for elastic or event-driven operational behavior.

Typical Characteristics

- cloud or orchestrated runtime environments,
- regular CI/CD and infrastructure automation,
- higher workload churn and stronger need for reissue or rapid issue behavior,
- broader telemetry and tracing expectations,
- tighter dependency on tenant isolation and secure default behavior.

Suitable Adoption Scenarios

This profile is typically suitable for:

- dynamic media services running on cloud or container platforms,
- event-driven capacity changes,
- operational models that expect automation-first deployment and recovery,
- organizations that already treat licensing as part of platform engineering.

Section Positioning

This profile defines the level at which DSL is no longer only compatible with cloud-native operations, but actively supports them.

08.05. Profile C - Advanced Interoperable

Purpose

This section defines the profile for implementations that pursue broad interoperability, multi-environment portability, and ecosystem-ready DSL behavior.

Profile Intent

The Advanced Interoperable profile builds on the earlier profiles and adds expectations for cross-vendor consistency, richer portability, and mature operational governance across more complex environments.

Core Expectations

An implementation aligned with this profile should provide:

- stable and well-documented lifecycle semantics across products and environments,
- strong alignment between canonical API, token, and data-model behavior,
- controlled portability across hybrid, failover, or disaster-recovery scenarios,
- mature auditability, traceability, and tenant isolation,
- support for richer modality and usage patterns without semantic drift,
- implementation evidence that can be relied on by ecosystem partners.

Typical Characteristics

- more than one deployment profile in active use,
- higher expectation of partner or integrator interoperability,
- stronger governance over terminology and schema consistency,
- reduced reliance on implementation-specific operator knowledge,
- readiness for broader industry normalization.

Suitable Adoption Scenarios

This profile is typically suitable for:

- vendors with more than one product line or deployment style,
- customer environments spanning cloud, on-premises, and resilience overlays,
- ecosystems that need repeatable semantics across organizational boundaries,
- implementations intended to serve as reference points for broader adoption.

Section Positioning

This profile defines the level at which DSL becomes a mature interoperability model rather than only a well-implemented individual solution.

09. Normative Requirements

Purpose

This chapter contains the main implementation-oriented requirements of the Dynamic Software Licensing best practices.

Its purpose is to organize the requirement structure of the best-practices document in a way that is clear, reviewable, and compatible with the standard-like intent of the DSL initiative.

Role Within the Best Practices

This chapter is one of the core chapters of the DSL best practices.

It translates the best practices's principles, stakeholder needs, and reference model into structured requirement areas that can later be expressed using disciplined best practices language.

Basis for This Chapter

The current DSL material already points to recurring requirement domains such as:

- general architecture and interface expectations,
- identity-based binding,
- lease and policy behavior,
- API interactions,
- usage metering,
- license modalities and modality-driven behavior,
- security expectations,
- operational continuity,
- integration with automation and deployment workflows.

This chapter organizes those domains into a stable structure.

Scope of This Chapter

This chapter defines the major requirement groups of the best practices and organizes the requirement areas into a stable structure.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. General Requirements Defines the broad baseline expectations of the best practices.

02. Identity and Binding Requirements Defines the requirements related to workload identity, identity evidence, and binding behavior.

03. Lease and Policy Requirements Defines requirements related to lease lifecycle, policy handling, renewals, grace behavior, and state transitions.

04. API Requirements Defines requirements related to the licensing API and related interaction patterns.

05. Metering Requirements Defines requirements related to usage tracking, dimensions, and metering behavior.

06. License Modality Requirements Defines requirements related to supported license modalities and modality-driven system behavior.

This section covers how time-, process-, and outcome-oriented licensing modalities influence metering, dimensions, and lifecycle handling in the best practices structure. It avoids pricing, commercial terms, or market coordination topics.

07. Security Requirements Defines requirements related to security, trust, auditability, and protection of licensing assets.

08. Operational Requirements Defines requirements related to continuity, outage handling, alerts, observability, and operational transparency.

09. Integration Requirements Defines requirements related to external integration patterns such as IaC, CI/CD, automation, and deployment workflows.

This section ensures that licensing is treated as an integrated component of modern software delivery rather than an isolated mechanism.

Relationship to Other Chapters

This chapter is closely tied to:

- conventions and normative language,
- the reference model,
- conformance and profiles,
- canonical API and data models,
- security and operational chapters,
- relevant normative annexes.

It forms the main requirement backbone of the best practices.

Chapter Positioning

This chapter forms the normative backbone of the published baseline.

The chapter is assessed not only for structure, but also for whether the requirement groups are complete, coherent, and aligned with the stakeholder, workflow, and model chapters.

09.01. General Requirements

Purpose

This section defines the general requirements that apply across the Dynamic Software Licensing (DSL) best practices. These requirements establish the baseline expectations that every DSL-aligned implementation must satisfy before topic-specific requirements for identity, policy, API behavior, metering, security, operations, or integration are applied.

The requirements in this section are intentionally cross-cutting. They express the structural expectations that make the rest of the best practices coherent and interoperable.

Requirement Scope

The requirements in this section apply to the DSL model as a whole, including:

- the control plane,
- the workspace plane,
- the interfaces between them,
- administrative and automation-facing clients,
- the interpretation of policies, tokens, and lifecycle operations.

Detailed requirements for specific domains are defined in later sections of this chapter.

General Requirements

GEN-1 — Open and documented interfaces All licensing lifecycle behavior that is required for normal operation **MUST** be available through open, documented interfaces. An implementation **MUST NOT** depend on undocumented, manual, or vendor-private operational steps for routine issue, renew, reissue, revoke, usage reporting, or administrative inspection behavior.

GEN-2 — Separation of decision and enforcement The DSL model **MUST** separate licensing decision authority from runtime enforcement. The control plane **MUST** remain the authoritative source for entitlements, leases, policy, and signing, while the workspace plane **MUST** enforce granted capability locally based on the artifacts issued by the control plane.

GEN-3 — Local runtime continuity An implementation **MUST** support local validation and local enforcement of the currently granted license state so that licensed workloads do not require a per-use synchronous round trip to the control plane. Any continued operation during interruption **MUST** remain bounded by explicit token and policy constraints.

GEN-4 — Identity-based operation Issue, renew, and reissue behavior **MUST** operate on verifiable workload or actor identity rather than on static hardware fingerprints alone. An implementation **MUST NOT** treat host attributes such as hostname, MAC address, or manually copied machine identifiers as the sole basis for trust.

GEN-5 — Policy-governed lifecycle behavior Lifecycle behavior **MUST** be governed by explicit policy rather than by hidden client logic or ad hoc operator intervention. Renewal timing, grace behavior, exhaustion actions, usage handling, and revocation effects **MUST** be determinable from documented server-defined policy and issued runtime artifacts.

GEN-6 — Tenant-aware isolation and control The model **MUST** preserve tenant separation across entitlements, leases, usage, policy, and administrative actions. A tenant **MUST NOT** be able to observe, modify, or consume another tenant's licensing state except where an explicitly delegated cross-tenant governance model is defined and audited.

GEN-7 — Operational transparency An implementation **MUST** provide sufficient observability for operators and administrators to understand current license state, recent lifecycle actions, and material failures. At a minimum, the model **MUST** support inspection of lease state, visibility into renewal or revocation outcomes, and traceability of significant license lifecycle events.

GEN-8 — Platform-neutral applicability The core DSL model **MUST** remain platform-neutral at the best-practices level. Platform-specific realization details **MAY** vary by deployment environment, but those variations **MUST NOT** change the core lifecycle semantics, trust model, or interoperability assumptions defined by these best practices.

GEN-9 — Interoperable extension boundaries Implementations **MAY** extend the canonical model with vendor-specific fields, controls, or automation patterns where needed, but such extensions **MUST NOT** undermine the canonical interpretation of lifecycle operations, token validity, tenant isolation, or policy behavior. Where extensions affect interoperability, they **SHOULD** be explicitly documented.

Relationship to Other Sections

- Section 09.02 refines the identity and binding requirements introduced by GEN-4.
- Section 09.03 refines the policy and lifecycle expectations introduced by GEN-3 and GEN-5.
- Section 09.04 specifies the interface expectations introduced by GEN-1.
- Sections 09.07 and 09.08 further develop the security and operational expectations introduced by GEN-6 and GEN-7.

Section Positioning

This section provides the baseline requirement layer for the rest of Chapter 09. It should be used to assess whether later domain-specific requirements remain aligned with the core DSL principles of openness, local enforceability, identity-based trust, tenant separation, and platform-neutral interoperability.

09.02. Identity and Binding Requirements

Purpose

This section defines the requirements for workload identity, identity evidence, and binding behavior in the Dynamic Software Licensing (DSL) model. These requirements ensure that a lease is granted and renewed for the intended workload and that mobility and failover do not depend on static host fingerprints.

Requirements in this section are identified as **IDB-n**.

Identity and Binding Requirements

IDB-1 — Verifiable identity evidence Issue and renew operations **MUST** require verifiable identity evidence representing the requesting workload or actor. The control plane **MUST** validate that evidence against a trusted identity source before granting or extending licensed capability.

IDB-2 — Identity-based binding Lease binding **MUST** be based on verifiable workload identity rather than on static hardware attributes alone. Hostname, MAC address, serial number, or similar machine properties **MUST NOT** be treated as the sole basis of binding.

IDB-3 — Binding continuity on renewal Renewal **MUST** re-verify that the requester remains the identity to which the lease was granted. A renewal request with missing, invalid, or mismatched identity evidence **MUST** be rejected.

IDB-4 — Controlled mobility through reissue When licensed capability moves from one workload identity to another, the move **MUST** occur through an explicit reissue or equivalent transfer operation governed by the control plane. Identity changes **MUST NOT** be handled as silent renewal of the original binding.

IDB-5 — Platform-specific evidence, common semantics Different deployment environments **MAY** use different identity evidence formats, but the meaning of successful verification **MUST** remain consistent: the control plane has verified that the requester is authorized to act for the identified workload in the stated tenant context.

IDB-6 — No privileged vendor dependency The identity and binding model **MUST NOT** require standing privileged access by the vendor or control-plane operator into the customer runtime environment. Trust **MUST** be established through presented evidence and protocol interaction rather than remote operator access.

IDB-7 — Binding visibility and auditability The effective binding state **MUST** be auditable and inspectable. For each active lease, the system **MUST** be able to show which verified identity it is bound to, when that binding was established, and when it was last re-verified.

Relationship to Other Sections

- Section 09.03 defines how identity-bound leases behave over time through renewal, grace, and revocation policy.
- Section 09.04 defines the interface behavior through which issue, renew, and reissue are exposed.
- Section 09.07 defines the security controls that protect identity evidence and binding trust.

Section Positioning

This section translates the DSL design principle of identity-based binding into explicit requirements. It should be read together with the trust model and security controls, which define how the verification is secured, and with the API model, which defines how binding is carried through lifecycle operations.

09.03. Lease and Policy Requirements

Purpose

This section defines the requirements governing lease lifecycle behavior, policy ownership, renewal timing, grace behavior, exhaustion actions, and revocation effects. These requirements ensure that runtime continuity and enforcement behavior remain explicit, bounded, and consistent across implementations.

Requirements in this section are identified as **LPR-n**.

Lease and Policy Requirements

LPR-1 — Explicit lease state An implementation **MUST** manage leases as explicit state with identifiable lifecycle phases, including at least issued, active, renewed, reissued, revoked, expired, and exhausted or terminated outcomes where applicable.

LPR-2 — Policy-defined renewal behavior Renewal behavior **MUST** be governed by explicit policy parameters communicated by the control plane. At a minimum, policy **MUST** define when renewal begins, how retry behavior operates, what grace period applies after expiry, and what action is taken when grace is exhausted.

LPR-3 — Local policy enforceability The effective lease policy **MUST** be conveyed to the workspace plane in a form that can be enforced locally during interruption. Runtime behavior during disconnection **MUST NOT** depend on an implicit, server-only interpretation of grace or exhaustion rules.

LPR-4 — Bounded continuity Continued operation after missed renewal **MUST** remain bounded by explicit token lifetime and grace policy. An implementation **MUST NOT** permit indefinite continuation without successful contact with the control plane or an explicitly governed offline mechanism.

LPR-5 — Defined exhaustion actions When grace is exhausted or a lease can no longer continue, the resulting action **MUST** be defined by policy. The implementation **MUST** apply a documented outcome such as drain, degrade, or stop rather than failing unpredictably.

LPR-6 — Revocation semantics The model **MUST** support revocation of active leases. When revocation cannot be delivered immediately, its effect **MUST** take hold no later than the next successful lifecycle contact or the bounded validity window of the currently issued token.

LPR-7 — Reissue without double consumption Reissue behavior **MUST** support controlled mobility of licensed capability without producing an undefined interval in which both the original and replacement workload appear

fully authorized beyond policy. The cutover semantics **MUST** be explicit and auditable.

LPR-8 — Inspectable lifecycle state Lease lifecycle state, effective policy, expiry, and recent transitions **MUST** be inspectable through documented administrative or programmatic interfaces so that operators can understand why a workload is active, renewing, degraded, or stopped.

Relationship to Other Sections

- Section 09.02 defines the identity rules that govern issue, renew, and reissue.
- Section 09.07 defines the revocation, token, and trust protections that make bounded continuity safe.
- Section 09.08 defines the operational handling of alerts, interruption, and recovery.

Section Positioning

This section defines the runtime semantics of leasing in the DSL model. It is the bridge between the conceptual lifecycle in the reference model and the operational continuity rules in the security and operations chapters.

09.04. API Requirements

Purpose

This section defines the requirements for the canonical licensing interfaces of the Dynamic Software Licensing (DSL) model. These requirements cover lifecycle operations, interface documentation, request semantics, error handling, and inspection capabilities.

Requirements in this section are identified as **API-n**.

API Requirements

API-1 — Documented lifecycle surface The core licensing lifecycle **MUST** be exposed through documented interfaces covering issue, renew, reissue, revoke, usage reporting, and administrative inspection or reporting behavior. Routine operation **MUST NOT** depend on undocumented vendor-private endpoints or manual-only actions.

API-2 — Stable operation semantics Each lifecycle operation **MUST** have stable, documented semantics that are consistent across implementations. In particular, renew **MUST** extend an existing lease, reissue **MUST** move or transfer authority to a new workload identity under control-plane governance,

and revoke **MUST** terminate or invalidate continued use within the defined bounded window.

API-3 — Authenticated, tenant-scoped requests All control-plane operations **MUST** require authentication, and every request **MUST** be authorized within its tenant context. The interface **MUST NOT** permit a caller to read or modify another tenant’s licensing state without explicit delegated authorization.

API-4 — Idempotent retry behavior Operations that may be retried due to transient failure **MUST** support idempotent or safely repeatable behavior. The interface **MUST** provide correlation or request-identification mechanisms sufficient to avoid accidental duplicate grants, duplicate transfers, or double counted usage.

API-5 — Structured error model Non-success outcomes **MUST** be expressed through a documented, structured error model that distinguishes authentication, authorization, policy, quota, validation, and transient service failures. Error responses **SHOULD** provide enough information to drive retry, operator investigation, or client fallback without exposing sensitive internals.

API-6 — Traceability and diagnostics The interface **SHOULD** support request correlation for diagnostics and observability across workspace-plane, control-plane, and external automation clients. When distributed tracing is supported, request correlation data **SHOULD** propagate consistently across participating components.

API-7 — Versioned compatibility The canonical API **MUST** provide explicit versioning and documented compatibility rules so that clients can rely on stable contract boundaries over time. Backward-compatible evolution within a version **SHOULD** preserve existing operation semantics.

API-8 — Inspection without lifecycle ambiguity The model **MUST** provide a documented way to inspect current lease, entitlement, or usage state without conflating read-only inspection with state-changing lifecycle operations. Query behavior **MUST** remain distinguishable from issue, renew, reissue, revoke, and usage record operations.

Relationship to Other Sections

- Section 09.03 defines the lifecycle semantics that these interfaces expose.
- Section 09.05 defines the structure and handling expectations for usage reporting.
- Section 09.07 defines the transport, token, and access-control protections required by the interface surface.

Section Positioning

This section defines the behavioral contract expected of DSL interfaces. It does not replace the canonical API chapter, which specifies the preferred surface and patterns, but it makes the interface obligations explicit at requirement level.

09.05. Metering Requirements

Purpose

This section defines the requirements for usage metering, reporting granularity, correlation, deduplication, and operational transparency of consumed license dimensions.

Requirements in this section are identified as **MTR-n**.

Metering Requirements

MTR-1 — Structured usage dimensions Where a license modality depends on consumption, usage **MUST** be expressed through structured dimensions with unambiguous semantics. The model **MUST** support vendor-defined dimensions, but each dimension **MUST** be documented well enough for customers and integrators to understand what is being measured.

MTR-2 — Correlated usage attribution Usage events **MUST** be attributable to at least a tenant, product or application context, lease or equivalent grant context, and time of use. Implementations **SHOULD** also support correlation to a higher-level work or deployment context where that improves transparency and reconciliation.

MTR-3 — Bounded buffering during interruption When connectivity is unavailable, usage events **MUST** be bufferable for later submission so that short outages do not create blind spots in consumption reporting. Buffered reporting **MUST** remain bounded by documented policy and storage assumptions.

MTR-4 — Idempotent reconciliation The control plane **MUST** process retried or delayed usage events idempotently so that recovered or repeated submissions do not produce double counting.

MTR-5 — Separation of usage reporting and runtime enforcement Usage reporting **MUST** be treated as a reporting and reconciliation function rather than as a per-use synchronous enforcement dependency. Failure to submit usage immediately **MUST NOT** by itself force abrupt runtime termination when the current lease and policy remain otherwise valid.

MTR-6 — Transparent consumption visibility The model **MUST** provide enough visibility for customers and operators to understand consumed dimensions, relevant time windows, and the relationship between active runtime state and reported usage. Usage transparency **SHOULD** support forecasting, reconciliation, and audit.

MTR-7 — Compatibility with different modalities The metering model **MUST** support time-based, process-based, capacity-based, and outcome-based usage expressions where applicable, without assuming that a single dimension type fits all products.

Relationship to Other Sections

- Section 09.06 defines how license modalities determine whether and how usage dimensions are applied.
- Section 09.08 defines the operational visibility and alerting expectations associated with metering.
- Section 09.09 defines how usage reporting participates in broader automation and external system integration.

Section Positioning

This section translates the DSL need for cost transparency and expressive metering into practical requirement language. It ensures that usage data is meaningful, attributable, and reconcilable without turning metering into a fragile runtime dependency.

09.06. License Modality Requirements

Purpose

This section defines the requirements governing license modality support in the Dynamic Software Licensing (DSL) model. It focuses on how different modality types influence runtime, metering, entitlement interpretation, and lifecycle handling, without prescribing pricing or commercial packaging.

Requirements in this section are identified as **MOD-n**.

License Modality Requirements

MOD-1 — Support for multiple modality types The DSL model **MUST** support more than one license modality where product and entitlement policy require it. At a minimum, the best-practices model **MUST** be capable of expressing time-based, process-based, and outcome-based modality semantics.

MOD-2 — Explicit modality semantics For every supported modality, the implementation **MUST** define what is granted, what is measured, what lifecycle events matter, and what usage dimensions or counters determine consumption. Modality interpretation **MUST NOT** rely on undocumented commercial shorthand.

MOD-3 — Consistent modality behavior across lifecycle operations Issue, renew, reissue, revoke, and usage reporting behavior **MUST** remain coherent with the selected license modality. A modality **MUST NOT** imply hidden exceptions to core lifecycle semantics unless those exceptions are explicitly documented and governed by policy.

MOD-4 — Mixed-modality coexistence The model **SHOULD** support coexistence of multiple modalities within the same tenant or account landscape, provided entitlement boundaries remain explicit. Multi-product or multi-environment deployments **MUST NOT** require a single modality model across all licensed capabilities.

MOD-5 — Modality transparency to operators The active modality **MUST** be visible to the customer or operator together with the granted dimensions, relevant policy, and resulting lifecycle behavior. A user **MUST** be able to tell whether a grant behaves as time-based, process-based, or outcome-based without reverse-engineering vendor logic.

MOD-6 — No contradiction with policy and metering A chosen modality **MUST** align with the metering model and policy model used for the same grant. Implementations **MUST NOT** expose a modality label whose billing or enforcement behavior cannot be explained by the documented dimensions, policy, and lifecycle controls.

Relationship to Other Sections

- Section 09.05 defines the metering behavior that operationalizes modality semantics.
- Section 09.03 defines the lifecycle and policy controls that remain applicable regardless of modality.
- Section 09.09 defines the external integration expectations for modality-aware automation and reporting.

Section Positioning

This section keeps the DSL model flexible enough for different licensing strategies while requiring that modality-specific behavior remain explicit, inspectable, and aligned with the canonical lifecycle and metering model.

09.07. Security Requirements

Purpose

This section defines the security requirements that apply to the normative core of the Dynamic Software Licensing (DSL) model. These requirements summarize the cross-cutting security expectations that every DSL-aligned implementation must satisfy and connect them to the more detailed security and trust framework in Chapter 12.

Requirements in this section are identified as **SREQ-n**.

Security Requirements

SREQ-1 — Signed and locally validatable grants License grants **MUST** be represented by signed artifacts that can be validated locally by the workspace plane. A licensed workload **MUST NOT** depend on an online control-plane call for every runtime enforcement decision.

SREQ-2 — Authenticated, confidential transport All communication carrying lifecycle requests, usage submissions, key-discovery material, or push events **MUST** use authenticated and confidential transport.

SREQ-3 — Verified identity before grant or extension The control plane **MUST** verify identity evidence before issuing, renewing, or reissuing licensed capability. Missing, invalid, or unverifiable identity evidence **MUST** prevent the grant or extension of new authority.

SREQ-4 — Least privilege and tenant isolation Control-plane and workspace-plane components **MUST** operate with least privilege, and every request **MUST** be evaluated in a tenant-isolated context. The model **MUST NOT** permit one tenant to consume, inspect, or alter another tenant's licensing state without explicit, audited delegation.

SREQ-5 — Bounded revocation and expiry exposure The security model **MUST** bound how long a revoked, expired, or otherwise unauthorized grant can continue to be used. Token lifetime, renewal policy, and grace behavior **MUST** work together so that exposure remains limited and governed.

SREQ-6 — Auditable security-relevant actions Security-relevant lifecycle actions **MUST** be auditable, including issue, renew, reissue, revoke, policy override, access override, and material validation failure events.

SREQ-7 — No client-side capability secrets Client-side components **MUST NOT** hold secrets that would allow them to mint capability, forge entitlement, or bypass validation. Client credentials **MUST** be limited to request authentication and authorized interaction.

Relationship to Other Sections

- Section 09.02 defines the identity and binding obligations enforced by the security model.
- Section 09.03 defines the policy and lifecycle semantics that create bounded exposure.
- Chapter 12 defines the detailed control families and threat mitigations that realize these high-level requirements.

Section Positioning

This section provides the normative summary of security expectations for Chapter 09. It is not intended to duplicate every detailed security control from Chapter 12, but to make clear which security properties the general DSL requirement backbone depends on.

09.08. Operational Requirements

Purpose

This section defines the operational requirements for continuity, observability, outage handling, recovery, and routine operator interaction in the Dynamic Software Licensing (DSL) model.

Requirements in this section are identified as **OPS-n**.

Operational Requirements

OPS-1 — Defined continuity behavior The licensing model **MUST** behave predictably during transient control-plane or network interruption. Continued operation, degraded operation, or stop behavior **MUST** be governed by documented policy rather than by implicit client-specific fallback.

OPS-2 — Background renewal and recovery The license agent or equivalent runtime component **MUST** attempt renewal in the background before expiry and continue retrying during interruption using a bounded retry strategy. When connectivity returns, recovery **MUST** restore normal licensed operation without requiring full redeployment where the grant remains valid.

OPS-3 — Operator visibility into current state Operators **MUST** be able to inspect current lease state, expiry horizon, effective policy, and recent lifecycle outcomes. The system **SHOULD** make it clear whether a workload is in normal operation, renewal retry, grace, degraded mode, or exhaustion.

OPS-4 — Advance alerting The model **MUST** support advance alerting for conditions that may affect licensed operation, including renewal failure trends,

impending expiry, grace countdown, repeated usage-submission failure, and revocation or policy changes where those require operator action.

OPS-5 — Routine self-service actions The model **SHOULD** support documented, customer-visible mechanisms for routine administrative actions such as inspection, controlled release or revocation initiation where authorized, and capacity-related visibility, without requiring vendor-side manual intervention for every such task.

OPS-6 — Recovery with audit continuity Recovery after interruption **MUST** preserve the ability to reconstruct what occurred. Renewal attempts, policy transitions, grace entry, exhaustion actions, reissue cutover, and delayed usage submission **MUST** remain auditable across the outage boundary.

OPS-7 — Profile-sensitive behavior Operational behavior **MUST** be configurable or governable in a way that can reflect the needs of different deployment profiles, such as continuous production, burst event workloads, testing environments, and stricter protection scenarios, without changing the underlying DSL core semantics.

Relationship to Other Sections

- Section 09.03 defines the lifecycle and policy semantics behind continuity behavior.
- Section 09.05 defines the usage buffering and reconciliation expectations relevant to outage handling.
- Section 09.07 defines the security bounds that limit how far operational continuity may extend.

Section Positioning

This section expresses the practical operating expectations of the DSL model. It ensures that the best practices remain usable in production environments where licensing behavior is part of day-to-day operations rather than an abstract contract only.

09.09. Integration Requirements

Purpose

This section defines the requirements for integrating the Dynamic Software Licensing (DSL) model into automation, deployment, orchestration, reporting, and operational toolchains.

Requirements in this section are identified as **INT-n**.

Integration Requirements

INT-1 — Automation-ready lifecycle access All routine lifecycle behavior needed to deploy, operate, recover, or decommission licensed software **MUST** be automatable through documented interfaces. A team **MUST NOT** be forced to break an automated workflow because licensing requires an opaque manual step.

INT-2 — Infrastructure and deployment integration The model **SHOULD** support integration with infrastructure-as-code, deployment automation, and orchestration systems so that license acquisition, renewal readiness, and controlled lifecycle termination can participate in normal environment provisioning and teardown flows.

INT-3 — Non-interactive operation Where a workflow is expected to run unattended, the licensing model **MUST** support non-interactive operation. This includes machine-to-machine authentication, headless issue and renew behavior, and unattended usage reporting.

INT-4 — Correlation across external systems The model **SHOULD** support correlation identifiers or equivalent linkage that allows license lifecycle and usage events to be associated with external deployment, workflow, or production contexts. This enables consistent traceability across orchestration, observability, and financial systems.

INT-5 — Tooling-neutral consumption The canonical DSL behaviors **MUST** be consumable by different client types, including API clients, CLI tools, SDKs, portals, or integration adapters, without changing the underlying lifecycle semantics or trust assumptions.

INT-6 — Reporting and reconciliation interfaces Where licensing usage or allocation data needs to flow into external analytics, billing, or governance systems, the model **SHOULD** provide documented export, query, or event-delivery mechanisms sufficient for reconciliation and reporting.

INT-7 — Controlled extensibility for vendor adapters Vendor- or marketplace-specific adapters **MAY** exist, but they **MUST NOT** replace or contradict the canonical lifecycle and policy semantics of the DSL model. Where adapters add translation logic, that logic **SHOULD** remain explicit and bounded.

Relationship to Other Sections

- Section 09.04 defines the interface obligations that make integration possible.

- Section 09.05 defines the usage structures most commonly exported into external systems.
- Section 09.08 defines the operator-facing behavior that integrated tooling must surface or respond to.

Section Positioning

This section translates the DSL principles of API-first access, licensing as code, and multi-interface accessibility into concrete integration requirements. Its role is to ensure that licensing behaves as a first-class part of modern software delivery and operations rather than as a disconnected special case.

10. Canonical API Model

Purpose

This chapter defines the canonical API structure of the Dynamic Software Licensing best practices.

Its purpose is to organize the core interaction model for licensing workflows in a way that is implementation-oriented, reusable, and consistent across the best practices.

Role Within the Best Practices

The DSL material already treats API-based interaction as a central principle. Open, documented interfaces are one of the recurring foundations of the proposed licensing model.

This chapter translates that principle into a structured API view.

Basis for This Chapter

The current DSL drafts already describe recurring operations such as:

- issue,
- renew,
- move or reissue,
- revoke,
- usage recording,
- public key discovery,
- related request and response patterns.

This chapter provides the canonical structure for those operations.

Scope of This Chapter

This chapter defines:

- the overall API interaction model,
- the main operation types,
- the relationship between operations,
- the structure of related behavior such as key discovery and error handling.

It is not a vendor-specific API manual.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Overview Defines the overall purpose and interaction model of the canonical API.

02. Issue Defines the initial license issuance interaction.

03. Renew Defines the renewal interaction.

04. Move or Reissue Defines the move or reissue interaction.

05. Revoke Defines the revocation interaction.

06. Usage Record Defines the usage reporting interaction.

07. JWKS and Public Keys Defines the key-discovery aspect of the API model.

08. Error Model Defines the structure of API error handling.

Relationship to Other Chapters

This chapter is closely related to:

- the reference model,
- normative requirements,
- the canonical data model,
- the security and trust framework,
- API contracts in the annexes.

It serves as the main API-oriented interpretation layer of the best practices.

Chapter Positioning

This chapter provides the API-oriented interpretation layer for the DSL best practices. It turns the lifecycle operations, trust constraints, and data-model concepts from adjacent chapters into a coherent interaction model with consistent endpoints, request patterns, authentication expectations, error semantics, and key-discovery behavior.

The chapter is intentionally implementation-oriented but remains vendor-neutral. It defines the canonical structure and standards profile for API interactions while leaving deployment- specific realization details and concrete contracts to annexes, schemas, and profile-specific guidance.

10.01. Overview

Purpose

This section defines the overall purpose and interaction model of the canonical API within the Dynamic Software Licensing (DSL) best practices. It explains

how the API fits into the broader architecture and establishes the foundational patterns that all subsequent API operations follow.

Design Standards

The canonical API adopts the following established standards to ensure interoperability, tooling compatibility, and alignment with industry practice:

Concern	Standard	Reference
URL structure	Google AIP (resource-oriented with custom methods)	AIP-136
JSON field naming	lowerCamelCase	AIP-140
Error responses	RFC 9457 — Problem Details for HTTP APIs	RFC 9457
Key discovery	RFC 7517 — JSON Web Key (JWK)	RFC 7517
Distributed tracing	W3C Trace Context	W3C Trace Context
Well-known URIs	RFC 8615	RFC 8615

Rationale The API URL design follows the Google AIP custom-method pattern rather than pure CRUD REST or verb-based paths. Lease operations such as “renew” and “reissue” are domain-specific actions with side effects (identity re-verification, token re-signing, state transitions) that do not map cleanly to standard HTTP verbs like PATCH or PUT. The custom-method pattern (`:renew`, `:reissue`) preserves resource-oriented structure while being explicit about intent.

JSON payloads use lowerCamelCase for consistency with JWT custom claims, the existing draft content, and the broader ecosystem of JavaScript/TypeScript API consumers.

Role of the API in the DSL Architecture

The DSL model is built on an API-first principle: all licensing functions are fully accessible via open, well-documented interfaces. There are no operations that require manual intervention, console-only access, or vendor-specific tooling to complete.

The canonical API is the primary interface between:

- The workspace plane (license agents, SDKs, orchestration tools) and the control plane (entitlements, leases, keys, usage, analytics).
- Administrative tools (portals, CLI, IaC modules) and the control plane.
- Event-driven systems (revocation push, policy updates) and workspace-plane agents.

Interaction Model

The API follows a request-response model over REST for all transactional operations, complemented by a push channel (WebSocket) for asynchronous events.

REST Operations (Synchronous) All lease lifecycle and usage operations are exposed as versioned REST endpoints following the Google AIP resource-oriented design:

Operation	Endpoint	Method	Purpose
Issue	<code>/v1/leases</code>	POST	Request a new lease for a workload
Renew	<code>/v1/leases/{leaseId\}:\nrenew</code>	POST	Extend an active lease
Reissue	<code>/v1/leases/{leaseId\}:\nreissue</code>	POST	Initiate lease transfer to a new workload
Reissue Finalize	<code>/v1/leases/{leaseId\}:finalizeReissue</code>	POST	Complete a lease transfer
Inspect	<code>/v1/leases/{leaseId\}</code>	GET	Retrieve current lease state without mutating lifecycle state
Revoke	<code>/v1/leases/{leaseId\}</code>	DELETE	Terminate a lease
Usage Record	<code>/v1/usage-records</code>	POST	Submit metered usage events
JWKS	<code>/.well-known/jwks.json</code>	GET	Retrieve public signing keys (RFC 7517)

Standard CRUD operations (create a lease, inspect a lease, delete a lease) use standard HTTP methods. Domain-specific actions that have side effects beyond simple state mutation use custom methods (`:renew`, `:reissue`, `:finalizeReissue`) following AIP-136.

Push Channel (Asynchronous) The control plane maintains a WebSocket connection to license agents for:

- Revocation events: immediate notification when a lease is revoked.

- Policy updates: notification when vendor changes grace, renewal, or exhaustion parameters.

The push channel is a complement to the REST API, not a replacement. If the push channel is unavailable, revocation and policy changes take effect at the next renewal cycle. Implementations that cannot support WebSocket connections remain fully functional — they rely on token TTL and the renewal cycle for eventual consistency.

Authentication and Authorization

All API endpoints require authentication. The canonical model supports multiple authentication mechanisms:

- IAM-based authentication (cloud-native workloads).
- OIDC token-based authentication (Kubernetes, federated identity).
- Custom authorizers (on-premises or hybrid environments).

Authorization is enforced per-tenant: a request can only operate on entitlements and leases belonging to the authenticated tenant.

Common Request Patterns

All REST operations share common structural patterns.

Request Headers Every request SHOULD include W3C Trace Context headers for distributed tracing:

Header	Description
traceparent	Identifies the incoming request in a trace (version, trace-id, parent-id, trace-flags)
tracestate	Carries vendor-specific trace data

The control plane MUST propagate **traceparent** in responses and internal spans. This enables end-to-end observability across the workspace plane, control plane, and ISV systems using OpenTelemetry-compatible tooling.

Request Structure Every request includes:

- An application identifier (**app**) indicating which licensed product is being addressed.
- A tenant identifier (**tenantId**) indicating the organizational context.
- Identity evidence (**identityEvidence**) proving the workload's identity.
- A client descriptor (**client**) with agent type and version for diagnostics.

Response Structure Every successful response includes:

- A lease identifier (**leaseId**) for correlation and subsequent operations.
- A signed token (**token**) for local validation and enforcement.
- An expiry timestamp (**expiresAt**) indicating when the lease or token expires.
- A policy object (**policy**) with enforcement parameters.

Error Responses Failed requests return a structured error response following RFC 9457 — Problem Details for HTTP APIs. The error model is defined in subchapter 10.08.

Idempotency

Operations that may be retried (e.g., due to network interruption) support idempotency through:

- The **workId** field, which correlates a request to a specific deployment or run.
- The **leaseId** field (in the URL path) for renewal and revocation operations.
- Usage records include timestamps and correlation identifiers to prevent double counting.

Versioning

The API uses URL-based versioning with a `/v1/` prefix. This provides clear contract boundaries and supports non-breaking evolution within a version and breaking changes across versions. The client descriptor (**client.version**) additionally communicates the agent's software version for diagnostics and forward compatibility negotiation.

Transport Security

All API communication occurs over TLS. Private connectivity options (VPN, private endpoints) are supported for environments that require them. The push channel (WebSocket) also operates over TLS (WSS).

Relationship to Other Chapters

- The reference model (Chapter 07) defines the architectural context in which the API operates.
- The canonical data model defines the structure of the objects exchanged through the API.
- The security and trust framework defines the authentication, signing, and validation mechanisms referenced here.
- Detailed payload schemas and examples are provided in annexes.

10.02. Issue

Purpose

This section defines the initial license issuance interaction within the canonical API. The Issue operation is the entry point for a workload to acquire a lease and begin operating under a licensed entitlement.

When Issue Is Used

The Issue operation is invoked when:

- A workload starts for the first time and needs to acquire a license.
- A license agent boots alongside an application and requests initial authorization.
- An orchestration tool or IaC deployment triggers license acquisition as part of provisioning.

Issue is a first-run operation. Once a lease is active, subsequent interactions use Renew (to extend) or Reissue (to transfer).

Endpoint

POST /v1/leases

Interaction Flow

1. The license agent starts alongside the application workload.
2. The license agent collects identity evidence from the runtime environment (e.g., Instance Identity Document, IAM role, Kubernetes Service Account JWT).
3. The license agent generates or retrieves a `workId` to correlate this deployment.
4. The license agent sends a POST request to `/v1/leases` with the required fields.
5. The control plane validates the request:
 - Verifies the tenant's entitlement for the requested application.
 - Checks that the requested dimensions are within the entitlement's limits.
 - Verifies the identity evidence against the expected trust anchor.
 - Checks quota availability (e.g., available lease slots).
6. If validation passes, the control plane:
 - Creates a lease record.
 - Signs a token containing the granted dimensions, binding, policy, and expiry.
 - Writes an entry to the audit log.
 - Returns the lease details to the license agent.
7. The license agent delivers the signed token to the application.

8. The application SDK validates the token locally and begins enforcing dimensions.

Request Structure

The Issue request contains the following fields:

Field	Type	Required	Description
<code>app</code>	string	Yes	Application identifier (e.g., <code>â€œapp-Aâ€œ</code>)
<code>tenantId</code>	string	Yes	Tenant identifier
<code>dimensions</code>	array	Yes	Requested dimension limits (e.g., channels, FPS)
<code>licenseModality</code>	string	Yes	Requested license modality (e.g., <code>â€œhourlyâ€œ</code> , <code>â€œdailyâ€œ</code> , <code>â€œmonthlyâ€œ</code>)
<code>identityEvidence</code>	object	Yes	Proof of workload identity
<code>workId</code>	string	Yes	Correlation identifier for this deployment or run
<code>version</code>	string	No	Application version
<code>environment</code>	string	No	Environment tag (e.g., <code>â€œproductionâ€œ</code> , <code>â€œstagingâ€œ</code> , <code>â€œdevâ€œ</code>)
<code>client</code>	object	No	License agent type and version for diagnostics
<code>policyHints</code>	object	No	Requested policy parameters (e.g., heartbeat interval, grace period)

Dimensions Each dimension entry specifies a name and a requested quantity:

```
{
  "name": "CHANNELS",
  "quantity": 4
}
```

The control plane may grant fewer than requested if the entitlement has lower limits. The granted values are reflected in the response and token.

Identity Evidence The identity evidence structure varies by runtime environment. The canonical model uses a nested object keyed by platform:

```
{
  "identityEvidence": {
    "aws": {
      "iidDocument": "<base64 json>",
      "iidSignature": "<base64 pkcs7>"
    }
  }
}
```

Other platforms (Kubernetes, on-premises) use their own evidence format within the same structure.

Policy Hints The license agent may suggest policy parameters. The control plane is not obligated to honor them — it applies the vendor-defined policy, optionally adjusted by tenant overrides:

```
{
  "policyHints": {
    "heartbeatSeconds": 60,
    "offlineGraceSeconds": 1800
  }
}
```

Response Structure

A successful Issue response contains:

Field	Type	Description
leaseId	string	Unique identifier for the issued lease
app	string	Application identifier
tenantId	string	Tenant identifier
token	string	Signed JWT or PASETO token
issuedAt	string (ISO 8601)	Timestamp of issuance

Field	Type	Description
<code>expiresAt</code>	string (ISO 8601)	Timestamp of token/lease expiry
<code>licenseModality</code>	string	Granted license modality
<code>dimensions</code>	array	Granted dimension limits (may differ from requested)
<code>boundIdentity</code>	object	The verified identity the lease is bound to
<code>policy</code>	object	Enforcement parameters (renewal window, grace period, exhaustion action, alerts)

Policy Object The policy returned in the response governs runtime behavior:

```
{
  "policy": {
    "heartbeatSeconds": 60,
    "renewBeforeSeconds": 300,
    "offlineGraceSeconds": 1800,
    "onGraceExhausted": "DRAIN",
    "revocationEndpoint": "wss://lic.example.com/revocations"
  }
}
```

Validation Rules

The control plane applies the following validation during Issue:

Check	Failure Condition
Entitlement exists	Tenant has no entitlement for the requested application
Dimensions within limits	Requested dimensions exceed entitlement limits
Quota available	All lease slots for this entitlement are already in use
Identity evidence valid	Evidence signature verification fails or evidence is expired
License modality permitted	Requested license modality is not available for this entitlement
Environment permitted	Requested environment tag is not allowed by policy

Failed validation returns a structured error response (see subchapter 10.08).

Audit

Every Issue operation — successful or failed — is recorded in the audit log with:

- Timestamp
- Tenant and application identifiers
- Requested and granted dimensions
- Identity evidence summary (not the raw evidence)
- Outcome (success or failure reason)
- Lease identifier (if successful)

Relationship to Other Operations

- After Issue, the license agent uses Renew to maintain the lease.
- If the workload fails over, Reissue transfers the lease to a new identity.
- If the lease must be terminated, Revoke invalidates it.
- The token issued here is validated locally using keys from the JWKS endpoint.

10.03. Renew

Purpose

This section defines the renewal interaction within the canonical API. The Renew operation extends the lifetime of an active lease, ensuring continuous operation of a licensed workload without interruption.

When Renew Is Used

The Renew operation is invoked:

- Periodically by the license agent before the current token expires, as governed by the `renewBeforeSeconds` policy parameter.
- After a transient network interruption, when the license agent reconnects and needs to re-establish a valid lease.
- As a background operation — the application is typically unaware that renewal is happening.

Renew is a continuation operation. It assumes a lease has already been issued via the Issue operation.

Endpoint

POST `/v1/leases/{leaseId}:renew`

Interaction Flow

1. The license agent monitors the current token's expiry and the `renewBeforeSeconds` policy value.
2. When the renewal window opens (e.g., 5 minutes before expiry), the license agent sends a POST request to `/v1/leases/{leaseId}:renew`.
3. The control plane validates the request:
 - Confirms the lease is still active (not revoked or expired beyond grace).
 - Re-verifies the identity evidence to confirm the same workload is requesting renewal.
 - Checks that the underlying entitlement is still valid.
4. If validation passes, the control plane:
 - Extends the lease TTL.
 - Signs a new token with an updated expiry.
 - Updates the lease record.
 - Returns the new token and expiry to the license agent.
5. The license agent delivers the updated token to the application.
6. The application SDK re-validates the new token and continues operating.

Request Structure

The lease identifier is provided in the URL path. The request body contains:

Field	Type	Required	Description
<code>identityEvidence</code>	object	Yes	Current proof of workload identity
<code>workId</code>	string	No	Correlation identifier for the deployment or production chain

POST `/v1/leases/L-01HY6G1Q3Z5W6E3Z2N7X0:renew`

```
{
  "identityEvidence": {
    "aws": {
      "iidDocument": "...",
      "iidSignature": "..."
    }
  },
  "workId": "deploy-i-0abc123def456"
}
```

Identity evidence is required at every renewal to re-verify that the requesting workload is the same one the lease was originally bound to. This prevents a compromised or migrated token from being renewed by an unauthorized workload.

The `workId` field is optional but recommended. It enables correlation of individual leases to a higher-level production chain or workload group, supporting aggregated billing and operational visibility across multi-resource deployments.

Response Structure

A successful Renew response contains:

Field	Type	Description
<code>leaseId</code>	string	Lease identifier (unchanged)
<code>token</code>	string	New signed token with updated expiry
<code>expiresAt</code>	string (ISO 8601)	New expiry timestamp

```
{
  "leaseId": "L-01HY6G1Q3Z5W6E3Z2N7X0",
  "token": "<new signed token>",
  "expiresAt": "2025-10-13T18:10:00Z"
}
```

If the vendor has updated the policy since the last renewal, the response may also include an updated `policy` object. The license agent SHOULD apply the new policy parameters going forward.

Renewal Timing and Backoff

The license agent follows this renewal strategy:

1. Begin renewal attempts at `expiresAt - renewBeforeSeconds` (e.g., 5 minutes before expiry).
2. If the first attempt fails, retry with exponential backoff and jitter as defined by the policy's `renewBackoff` parameters:

```
{
  "renewBackoff": {
    "initialMs": 2000,
    "maxMs": 30000,
    "jitter": true
  }
}
```

3. Continue retrying until either:
 - Renewal succeeds, or
 - The token expires and the offline grace period begins.

What Happens When Renewal Fails

If the license agent cannot successfully renew before the token expires:

Phase	Duration	Behavior
Token valid	Before <code>expiresAt</code>	Application operates normally; license agent retries renewal
Grace period	<code>expiresAt</code> to <code>expiresAt + offlineGraceSeconds</code>	Application continues operating (full or degraded per policy); license agent continues retrying
Grace exhausted	After grace period ends	Configured exhaustion action applies: DRAIN, DEGRADE, or STOP

When renewal eventually succeeds (e.g., after network recovery), the application returns to normal operation with a fresh token.

Validation Rules

The control plane applies the following validation during Renew:

Check	Failure Condition
Lease exists and is active	Lease not found, already revoked, or expired beyond grace
Identity evidence matches	Workload identity does not match the lease binding
Entitlement still valid	Underlying entitlement has been revoked or modified
Quota still available	Entitlement quota has been reduced below current usage

A failed renewal returns a structured error response (see subchapter 10.08). The license agent treats a rejected renewal the same as an expired token — the grace period and exhaustion logic apply.

Audit

Every Renew operation is recorded in the audit log with:

- Timestamp
- Lease identifier

- Identity evidence summary
- Outcome (success, failure reason)
- New expiry (if successful)

Relationship to Other Operations

- Renew follows Issue and maintains the lease during normal operation.
- If renewal is permanently rejected (e.g., entitlement revoked), the effect is equivalent to Revoke.
- Renew does not change the lease binding, dimensions, or modality. To change these, a new Issue or entitlement modification is required.
- If the workload needs to move to a different identity (e.g., failover), Reissue is used instead of Renew.

10.04. Move or Reissue

Purpose

This section defines the move or reissue interaction within the canonical API. The Reissue operation transfers an active lease from one workload to another, supporting failover, blue/green deployments, and planned migration scenarios without requiring a new entitlement or causing license downtime.

When Reissue Is Used

The Reissue operation is invoked when:

- A workload fails and a backup instance needs to take over the license (disaster recovery).
- A blue/green deployment cutover requires the license to move to the new environment.
- A planned migration moves the application to a different host, container, or region.
- An instance is replaced due to corruption and the license must follow.

Reissue is not used for routine renewal (use Renew) or for acquiring a new license (use Issue).

Two-Phase Design

Reissue uses a two-phase approach to ensure atomic transfer without downtime:

1. Shadow phase: a shadow lease is created on the target workload while the original lease remains active.
2. Finalize phase: the shadow lease is activated and the original lease is revoked.

This design ensures that at no point are both leases fully active simultaneously (preventing double-use), and at no point are both leases inactive (preventing downtime).

Endpoints

POST /v1/leases/{leaseId}:reissue (Phase 1: shadow)
POST /v1/leases/{leaseId}:finalizeReissue (Phase 2: finalize)

Phase 1: Shadow

Interaction Flow

1. A new workload instance starts (e.g., backup EC2, new Kubernetes pod).
2. The license agent on the new instance collects its identity evidence.
3. The license agent sends a POST request to `/v1/leases/{leaseId}:reissue` referencing the original lease.
4. The control plane validates:
 - The original lease exists and is active.
 - The requesting tenant owns the original lease.
 - The new identity evidence is valid.
5. The control plane creates a shadow lease:
 - The shadow lease is bound to the new workload identity.
 - The original lease remains active during this phase.
 - A cutover token is generated to authorize finalization.
6. The control plane returns the shadow lease details, including the cutover token.

Request Structure The original lease identifier is provided in the URL path. The request body contains:

Field	Type	Required	Description
<code>toIdentityEvidence</code>	object	Yes	Identity evidence of the target workload
<code>toWorkId</code>	string	Yes	Work ID for the target deployment

POST /v1/leases/L-01HY6G1Q3Z5W6E3Z2N7X0:reissue

```
{  
  "toIdentityEvidence": {  
    "aws": {  
      "iidDocument": "...",  
      "iidSignature": "..."  
    }  
  }  
}
```

```

    }
  },
  "toWorkId": "deploy-i-0def789abc123"
}

```

Response Structure

Field	Type	Description
reissueId	string	Identifier for this reissue operation
cutoverToken	string	Authorization token required to finalize
toLease	object	Shadow lease details (lease ID, token, expiry, dimensions, policy)

```

{
  "reissueId": "M-9kqj8f",
  "cutoverToken": "ct-7f2e...",
  "toLease": {
    "leaseId": "L-02...",
    "token": "<signed token for new workload>",
    "expiresAt": "...",
    "dimensions": [...],
    "policy": {...}
  }
}

```

Phase 2: Finalize

Interaction Flow

1. The orchestrator or license agent on the new workload confirms the target is ready.
2. A POST request is sent to `/v1/leases/{leaseId}:finalizeReissue` with the reissue ID and cutover token.
3. The control plane:
 - Validates the cutover token.
 - Revokes the original lease.
 - Activates the shadow lease as the new active lease.
 - Pushes a revocation event to the original license agent via WebSocket.
4. The original workload receives the revocation and executes its exhaustion action (drain or stop).
5. The new workload continues operating on the now-active lease.

Request Structure The lease identifier is provided in the URL path. The request body contains:

Field	Type	Required	Description
<code>reissueId</code>	string	Yes	Identifier from the shadow phase
<code>cutoverToken</code>	string	Yes	Authorization token from the shadow phase

POST /v1/leases/L-01HY6G1Q3Z5W6E3Z2N7X0:finalizeReissue

```
{
  "reissueId": "M-9kqj8f",
  "cutoverToken": "ct-7f2e..."
}
```

Response Structure

Field	Type	Description
<code>state</code>	string	Outcome of the finalization (e.g., <code>COMPLETED</code>)
<code>revoked</code>	string	Lease ID of the original (now revoked) lease

```
{
  "state": "COMPLETED",
  "revoked": "L-01HY6G1Q3Z5W6E3Z2N7X0"
}
```

Timeout and Deadline

The reissue operation has a deadline. If finalization does not occur within a configured time window:

- The shadow lease is automatically expired.
- The original lease remains active.
- The reissue operation is recorded as abandoned in the audit log.

This prevents orphaned shadow leases from consuming entitlement quota indefinitely.

Validation Rules

Shadow Phase

Check	Failure Condition
Original lease exists and is active	Lease not found, already revoked, or expired
Tenant owns the original lease	Requesting tenant does not match
Target identity evidence is valid	Evidence verification fails
No pending reissue for this lease	Another reissue is already in progress

Finalize Phase

Check	Failure Condition
Reissue ID exists and is pending	Reissue not found or already completed/expired
Cutover token is valid	Token does not match or has expired

Audit

Both phases are recorded in the audit log:

- Shadow phase: original lease ID, target identity, reissue ID, timestamp.
- Finalize phase: reissue ID, revoked lease ID, activated lease ID, timestamp, outcome.
- Abandoned reissues (timeout) are also logged.

Relationship to Other Operations

- Reissue assumes a lease was previously created via Issue.
- After finalization, the new lease follows the normal Renew cycle.
- The revocation of the original lease during finalization follows the same logic as Revoke.
- If the original workload is offline when revocation is pushed, the revocation takes effect at the next renewal attempt (which will be rejected).

10.05. Revoke

Purpose

This section defines the revocation interaction within the canonical API. The Revoke operation terminates an active lease, invalidating the workload's right to continue operating under that license.

When Revoke Is Used

The Revoke operation is invoked when:

- An administrator explicitly terminates a license (e.g., policy violation, contract termination).
- A policy rule triggers automatic revocation (e.g., entitlement expiry, quota enforcement).
- A reissue operation finalizes and the original lease must be invalidated.
- A workload is decommissioned and the license **SHOULD** be returned to the available pool.
- A customer force-releases a stuck license via the End User Portal.

Revoke is a terminal operation. Once a lease is revoked, it cannot be renewed or reissued. A new Issue is required to acquire a fresh lease.

Endpoint

DELETE /v1/leases/{leaseId}

Interaction Flow

When the License Agent Is Online

1. An administrator, policy engine, or system operation sends a DELETE request to /v1/leases/{leaseId}.
2. The control plane validates the request:
 - Confirms the lease exists and is active.
 - Confirms the requester has authorization to revoke (admin, policy engine, or tenant with appropriate permissions).
3. The control plane:
 - Sets the lease status to **revoked**.
 - Records the revocation reason.
 - Writes an entry to the audit log.
 - Pushes a revocation event to the license agent via WebSocket.
4. The license agent receives the revocation event and notifies the application/SDK.
5. The application executes the configured exhaustion action:
 - DRAIN: finish current work, refuse new sessions.
 - DEGRADE: apply reduced functionality (e.g., fewer channels, disable premium features).
 - STOP: halt the licensing gate, application stops.

When the License Agent Is Offline

1. The control plane sets the lease status to **revoked** and records the event.
2. The WebSocket push fails because the agent is not connected.
3. The application continues operating on its current token until it expires.

4. When the token expires, the license agent attempts to renew.
5. The control plane rejects the renewal because the lease is revoked.
6. The license agent applies the exhaustion action.

The maximum delay before revocation takes effect is bounded by the token's TTL. Short token lifetimes limit this window.

Request Structure

The lease identifier is provided in the URL path. The request body contains:

Field	Type	Required	Description
reason	string	Yes	Reason for revocation (e.g., <code>policy_violation</code> , <code>admin_action</code> , <code>entitlement_expired</code> , <code>decommissioned</code> , <code>force_release</code>)

```
DELETE /v1/leases/L-01HY6G1Q3Z5W6E3Z2N7X0
```

```
{
  "reason": "policy_violation"
}
```

Response Structure

Field	Type	Description
status	string	Resulting lease status (<code>revoked</code>)

```
{
  "status": "revoked"
}
```

Revocation Reasons

The canonical model defines the following standard revocation reasons:

Reason	Description
<code>admin_action</code>	Explicit revocation by an administrator
<code>policy_violation</code>	Automated revocation triggered by a policy rule
<code>entitlement_expired</code>	The underlying entitlement has expired or been removed
<code>decommissioned</code>	The workload has been terminated and the license is being returned
<code>force_release</code>	Customer-initiated release via the End User Portal for a stuck or failed instance
<code>reissue_cutover</code>	Revocation as part of a reissue finalization (system-initiated)
<code>quota_exceeded</code>	Entitlement quota has been reduced below current active leases

Implementations may extend this list with additional vendor-specific reasons.

Authorization

Revocation requires elevated permissions. Not all API consumers can revoke a lease:

Actor	Can Revoke	Typical Scenario
Tenant administrator	Yes	Admin action, force release, decommissioning
Policy engine (control plane)	Yes	Automated policy enforcement, entitlement expiry
Reissue service (control plane)	Yes	Cutover finalization
License agent	No	Agents do not self-revoke; they receive revocation events
Application/SDK	No	Applications execute exhaustion actions, not revocations

Audit

Every Revoke operation is recorded in the audit log with:

- Timestamp

- Lease identifier
- Revocation reason
- Requesting actor (admin, policy engine, reissue service)
- Delivery status (WebSocket push succeeded or agent was offline)

Relationship to Other Operations

- Revoke terminates a lease that was created via Issue.
- After revocation, the lease cannot be renewed. A new Issue is required.
- Revoke is called internally by the Reissue finalization flow to invalidate the original lease.
- The exhaustion action executed by the application is governed by the policy that was embedded in the token at issuance or last renewal.

10.06. Usage Record

Purpose

This section defines the usage reporting interaction within the canonical API. The Usage Record operation allows license agents and SDKs to submit metered consumption events to the control plane for billing, analytics, and compliance purposes.

When Usage Record Is Used

The Usage Record operation is invoked:

- Periodically by the license agent during normal operation (e.g., per-minute heartbeats).
- At session boundaries (session start/stop events).
- After connectivity is restored, to upload buffered usage events that were collected during an outage.

Usage recording is a continuous background operation. It does not affect the application's ability to operate — it is a reporting function, not an enforcement function.

Endpoint

POST `/v1/usage-records`

Interaction Flow

1. The license agent or SDK collects usage data during operation (e.g., minutes of active use, sessions, process units).
2. At the configured interval, the license agent sends a POST request to `/v1/usage-records`.

3. The control plane validates and stores the usage event:
 - Hot store for operational queries and near-real-time dashboards.
 - Cold store (e.g., data lake) for long-term analytics, billing reconciliation, and reporting.
4. The control plane acknowledges receipt.

During Connectivity Loss

1. The license agent buffers usage events locally.
2. When connectivity is restored, the license agent uploads all buffered events.
3. The control plane processes them idempotently — duplicate submissions (same timestamp, lease, dimension) are deduplicated, preventing double counting.

Request Structure

Field	Type	Required	Description
<code>leaseId</code>	string	Yes	Identifier of the active lease
<code>tenantId</code>	string	Yes	Tenant identifier
<code>productId</code>	string	Yes	Application identifier
<code>dimension</code>	string	Yes	The metered dimension being reported (e.g., <code>“HOURS”</code> , <code>“SESSIONS”</code> , <code>“PROCESS_UNITS”</code>)
<code>quantity</code>	number	Yes	Amount consumed
<code>granularity</code>	string	Yes	Time granularity of the measurement (e.g., <code>“minutes”</code> , <code>“hours”</code>)
<code>timestamp</code>	string (ISO 8601)	Yes	When the usage occurred
<code>workId</code>	string	Yes	Correlation identifier for the deployment

```
{
  "leaseId": "L-...",
```

```

    "tenantId": "t-123",
    "productId": "app-A",
    "dimension": "HOURS",
    "quantity": 1,
    "granularity": "minutes",
    "timestamp": "2025-10-13T16:15:00Z",
    "workId": "deploy-i-0abc123def456"
  }

```

Response Structure

A successful response acknowledges receipt:

```

{
  "status": "accepted"
}

```

The control plane does not return detailed processing results in the synchronous response. Analytics and aggregated usage data are available through the reporting layer and End User Portal.

Usage Dimensions

Usage dimensions align with the dimension model defined in the entitlement and lease. Common usage dimensions include:

Dimension	Description	Typical Granularity
HOURS	Time-based usage	Per minute or per hour
SESSIONS	Session-based usage (start/stop)	Per session
CHANNELS	Active channel count over time	Per minute
PROCESS_UNITS	Normalized compute units (transcoding, inference, pixel operations)	Per operation or per minute
BANDWIDTH	Data throughput	Per GB or per minute
CLIPS	Outcome-based (e.g., generated clips, subtitles)	Per item

Vendors define which dimensions apply to their products. The canonical model does not prescribe a fixed set — it provides the structure for any dimension to be reported.

Idempotency

Usage records support idempotent submission through the combination of:

- `leaseId` + `timestamp` + `dimension` as a natural deduplication key.
- The `workId` for additional correlation.

This ensures that retried submissions (e.g., after a network timeout where the agent is unsure if the original request was received) do not result in double counting.

License Modality Compatibility

The usage recording model is designed to be compatible with marketplace-style metering. This means:

- Usage events can be mirrored to an ISV's back-office billing system via the optional Vendor Adapter.
- Usage events can be mapped to marketplace metering APIs where applicable.
- The dimension and quantity structure aligns with common marketplace metering schemas.

Audit

Usage events are stored with full traceability:

- Tenant, application, and lease identifiers.
- Timestamp and dimension details.
- Work ID for deployment correlation.
- Submission timestamp (when the event was received, which may differ from the usage timestamp if buffered).

Relationship to Other Operations

- Usage recording runs alongside the Renew cycle but is independent of it. A failed renewal does not prevent buffered usage from being submitted later.
- Usage data informs the analytics and reporting available through the End User Portal.
- Usage reconciliation is a shared responsibility between the control plane, ISV, and customer (as defined in the responsibility model).

10.07. JWKS and Public Keys

Purpose

This section defines the key discovery aspect of the canonical API. It describes how token validation material (public signing keys) is published by the control plane and consumed by applications and SDKs in the workspace plane.

Standards Compliance

This section implements the following standards:

- RFC 7517 — JSON Web Key (JWK) for the key set structure and format.
- RFC 8615 — Well-Known URIs for the discovery endpoint path convention.

Why Key Discovery Matters

The DSL model relies on local token validation — the application SDK verifies lease tokens without calling the control plane on every check. To do this, the SDK needs the public key that corresponds to the private key used to sign the token.

The JWKS (JSON Web Key Set) endpoint provides this public key in the standard format defined by RFC 7517.

Endpoint

GET `/.well-known/jwks.json`

This follows the standard well-known URI convention used in OAuth 2.0 and OpenID Connect ecosystems. The path is fixed and does not require authentication, since public keys are not sensitive — they are meant to be distributed.

Interaction Flow

1. On first startup (or when the cached keys expire), the SDK or license agent fetches the JWKS from the control plane.
2. The response contains one or more public keys, each identified by a key ID (`kid`).
3. The SDK caches the keys locally.
4. When validating a token, the SDK:
 - Reads the `kid` from the token header.
 - Looks up the corresponding public key in the local cache.
 - Verifies the token signature using that key.
5. If the token references a `kid` that is not in the cache (e.g., after key rotation), the SDK fetches the JWKS again to obtain the new key.

Response Structure

The JWKS response follows RFC 7517 — JSON Web Key Set:

```

{
  "keys": [
    {
      "kty": "RSA",
      "kid": "key-2025-10",
      "use": "sig",
      "alg": "RS256",
      "n": "<base64url-encoded modulus>",
      "e": "<base64url-encoded exponent>"
    }
  ]
}

```

Field	Description
kty	Key type (e.g., <code>â€œRSAâ€</code> , <code>â€œECâ€</code>)
kid	Key identifier — matches the <code>kid</code> in token headers
use	Key usage — always <code>â€œsigâ€</code> (signature) for this purpose
alg	Algorithm (e.g., <code>â€œRS256â€</code> , <code>â€œES256â€</code>)
n, e	Key material (format depends on key type)

The response may contain multiple keys to support key rotation (the current key and the previous key during a transition period).

Key Rotation

Key rotation is the process of replacing the signing key periodically to limit the impact of a potential key compromise. The DSL model supports key rotation as follows:

1. The control plane generates a new signing key in the key management service.
2. New tokens are signed with the new key (new `kid`).
3. The JWKS endpoint publishes both the new and old public keys.
4. SDKs that encounter the new `kid` refresh their cache and obtain the new key.
5. After a transition period (long enough for all active tokens signed with the old key to expire), the old key is removed from the JWKS response.

This ensures that tokens signed with either key can be validated during the transition, and no workload experiences a validation failure due to rotation.

Caching Behavior

The SDK **SHOULD** cache JWKS responses to avoid unnecessary network calls:

- Cache the keys after the first fetch.
- Refresh the cache only when:
 - A token contains a `kid` not found in the cache (indicates key rotation).
 - A configurable cache TTL has elapsed.
- Do not fetch JWKS on every token validation — this would defeat the purpose of local validation.

The control plane may include standard HTTP cache headers (`Cache-Control`, `max-age`) to guide caching behavior.

Availability Considerations

The JWKS endpoint **SHOULD** be highly available because:

- If the endpoint is unreachable during first startup, the SDK cannot validate any tokens.
- If the endpoint is unreachable during key rotation, SDKs cannot obtain the new key.

However, because keys are cached, a temporary outage of the JWKS endpoint does not affect workloads that already have the keys cached. The impact is limited to:

- New workloads starting for the first time.
- Key rotation transitions.

For environments with strict availability requirements, the JWKS response can be pre-distributed or cached at an edge layer.

Security Considerations

- The JWKS endpoint serves public keys only. Private signing keys never leave the key management service.
- The endpoint is served over TLS to prevent key substitution via man-in-the-middle attacks.
- The endpoint does not require authentication — public keys are intended to be freely accessible.
- Key identifiers (`kid`) are included in both the JWKS response and token headers to ensure unambiguous key matching.

Relationship to Other Operations

- Issue and Renew produce signed tokens that reference a `kid`.
- The SDK uses the JWKS endpoint to obtain the public key for that `kid`.
- Token validation (signature, expiry, claims) happens locally using the cached key — no further API call is needed.

- Key rotation is managed by the control plane's key management service and reflected automatically in the JWKS response.

10.08. Error Model

Purpose

This section defines the structure of API error handling within the canonical API. It describes how failures, rejected requests, and non-successful outcomes are expressed consistently across all operations.

Standards Compliance

This error model implements RFC 9457 — Problem Details for HTTP APIs (the successor to RFC 7807). All error responses use the `application/problem+json` media type.

Design Principles

The error model follows these principles:

1. Consistency: all operations use the same RFC 9457 error response structure.
2. Actionability: errors include enough information for the license agent or operator to determine the appropriate next step.
3. Machine-readability: error types are URIs for programmatic handling; extension members provide structured context.
4. Security: error responses do not leak sensitive information about the control plane's internal state, other tenants, or infrastructure details.

Error Response Structure

All failed API requests return a JSON response with `Content-Type: application/problem+json` following RFC 9457:

Field	Type	Required	Source	Description
<code>type</code>	string (URI)	Yes	RFC 9457	URI identifying the error type (e.g., <code>urn:dsl:error:entitlement-exhausted</code>)

Field	Type	Required	Source	Description
title	string	Yes	RFC 9457	Short, human-readable summary of the problem type
status	integer	Yes	RFC 9457	HTTP status code
detail	string	Yes	RFC 9457	Human-readable explanation specific to this occurrence
instance	string (URI)	No	RFC 9457	URI identifying the specific occurrence (for support correlation)
category	string	Yes	DSL extension	Error category for programmatic classification
retryable	boolean	Yes	DSL extension	Whether the client should retry the request

Additional context relevant to the specific error type MAY be included as extension members at the top level of the response object, as permitted by RFC 9457.

```
{
  "type": "urn:dsl:error:entitlement-exhausted",
  "title": "Entitlement Exhausted",
  "status": 429,
  "detail": "All lease slots for this entitlement are currently in use.",
  "instance": "urn:dsl:request:req-8f3a2b",
  "category": "quota",
  "retryable": true,
  "entitled": 4,
  "active": 4
}
```

}

Error Categories

Errors are grouped into categories that indicate the nature of the failure:

Category	Description	Typical HTTP Status	Retryable
validation	Request is malformed or missing required fields	400	No (fix the request)
authentication	Caller identity could not be verified	401	No (fix credentials)
authorization	Caller does not have permission for this operation	403	No
not_found	Referenced resource (lease, entitlement) does not exist	404	No
conflict	Operation conflicts with current state (e.g., lease already revoked, reissue already in progress)	409	No (resolve conflict)
quota	Entitlement quota is exhausted	429 or 403	Yes (after a lease is released)
identity	Identity evidence verification failed	403	No (fix identity)
policy	Request violates a policy constraint (e.g., modality not permitted, environment not allowed)	403	No (adjust request)

Category	Description	Typical HTTP Status	Retryable
expired	Referenced resource has expired (lease, cutover token, reissue deadline)	410	No
internal	Unexpected control plane error	500	Yes (with backoff)
unavailable	Control plane is temporarily unavailable	503	Yes (with backoff)

Standard Error Types

The following error types are defined for the canonical API operations. Each type URI follows the pattern `urn:dsl:error:<error-name>`.

Issue Errors

Type URI	Category	Title	HTTP Status
<code>urn:dsl:error:entitlement-not-found</code>	<code>not_found</code>	Entitlement Not Found	404
<code>urn:dsl:error:entitlement-exhausted</code>	<code>quota</code>	Entitlement Exhausted	429
<code>urn:dsl:error:dimensions-exceeded</code>	<code>policy</code>	Dimensions Exceeded	403
<code>urn:dsl:error:modality-not-permitted</code>	<code>policy</code>	Modality Not Permitted	403
<code>urn:dsl:error:environment-not-permitted</code>	<code>policy</code>	Environment Not Permitted	403
<code>urn:dsl:error:identity-verification-failed</code>	<code>identity</code>	Identity Verification Failed	403

Renew Errors

Type URI	Category	Title	HTTP Status
urn:dsl:error:lease-not-found	not_found	Lease Not Found	404
urn:dsl:error:lease-revoked	conflict	Lease Revoked	409
urn:dsl:error:lease-expired	expired	Lease Expired	410
urn:dsl:error:identity-mismatch	identity	Identity Mismatch	403
urn:dsl:error:entitlement-revoked	conflict	Entitlement Revoked	409

Reissue Errors

Type URI	Category	Title	HTTP Status
urn:dsl:error:lease-not-found	not_found	Lease Not Found	404
urn:dsl:error:lease-not-active	conflict	Lease Not Active	409
urn:dsl:error:reissue-already-pending	conflict	Reissue Already Pending	409
urn:dsl:error:identity-verification-failed	identity	Identity Verification Failed	403
urn:dsl:error:reissue-not-found	not_found	Reissue Not Found	404
urn:dsl:error:cutover-token-invalid	authentication	Cutover Token Invalid	401
urn:dsl:error:reissue-deadline-expired	expired	Reissue Deadline Expired	410

Revoke Errors

Type URI	Category	Title	HTTP Status
urn:dsl:error:lease-not-found	not_found	Lease Not Found	404
urn:dsl:error:lease-already-revoked	conflict	Lease Already Revoked	409
urn:dsl:error:revoke-not-authorized	authorization	Revoke Not Authorized	403

Usage Record Errors

Type URI	Category	Title	HTTP Status
urn:dsl:error:lease-not-found	not_found	Lease Not Found	404
urn:dsl:error:invalid-dimension	validation	Invalid Dimension	400
urn:dsl:error:invalid-timestamp	validation	Invalid Timestamp	400

License Agent Behavior on Errors

The license agent SHOULD handle errors based on the `category` and `retryable` extension members:

Scenario	Agent Behavior
<code>retryable: true</code>	Retry with exponential backoff and jitter (per <code>renewBackoff</code> policy)
<code>retryable: false, category validation</code>	Log the error; do not retry; alert the operator
<code>retryable: false, category identity</code>	Log the error; do not retry; the workload identity configuration needs attention
<code>retryable: false, category conflict (lease revoked)</code>	Treat as revocation; apply exhaustion action
<code>retryable: false, category quota</code>	Wait for capacity; optionally alert the operator

Scenario	Agent Behavior
retryable: <code>false</code> , category expired	Treat as terminal; a new Issue is required
Category internal or unavailable	Retry with backoff; if retries are exhausted, enter grace period logic

The **instance** field, when present, SHOULD be included in support tickets or log entries to enable the control plane operator to correlate the error with server-side diagnostics.

Security Considerations

Error responses MUST NOT expose:

- Internal infrastructure details (e.g., database names, Lambda function ARNs, stack traces).
- Information about other tenants or their entitlements.
- Detailed cryptographic failure reasons that could aid an attacker.

The **detail** field SHOULD be informative enough for debugging but generic enough to avoid information leakage. The **instance** field SHOULD use opaque identifiers (not internal resource paths).

Relationship to Other Chapters

- Each API operation (Issue, Renew, Reissue, Revoke, Usage Record) references this error model for its failure cases.
- The grace policy (defined in the reference model) governs how the license agent behaves when errors prevent renewal.
- The audit log records both successful and failed operations, including the error **type**.
- The **application/problem+json** content type is defined by RFC 9457 and supersedes the earlier RFC 7807.

11. Canonical Data Model

Purpose

This chapter defines the canonical data model structure of the Dynamic Software Licensing best practices.

Its purpose is to organize the main objects and data concepts used across the best practices in a consistent and implementation-oriented way.

Role Within the Best Practices

The DSL best practices depend on a stable vocabulary not only for architecture and behavior, but also for the objects that move through the system.

This chapter provides the structured home for those objects and supports consistency across:

- the reference model,
- API chapters,
- requirement chapters,
- security and operational behavior,
- schemas and annexes.

Basis for This Chapter

The current DSL drafts already refer to recurring objects and structured concepts such as:

- lease token,
- entitlement,
- policy,
- usage events,
- identity evidence,
- token claims,
- license envelope.

This chapter organizes those objects coherently.

Scope of This Chapter

This chapter defines the major canonical objects used by the best practices. It focuses on structure and interpretation rather than vendor-specific payload detail.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Lease Object Defines the main lease-related object structure.

02. Entitlement Object Defines the object structure representing entitlement state and granted rights.

03. Policy Object Defines the object structure representing policy-driven behavior.

04. Usage Event Object Defines the object structure used for metering or usage reporting.

05. Token Claims Object Defines the structure of claims carried in signed tokens.

06. Identity Evidence Object Defines the object structure used to express workload identity evidence.

07. License Envelope Object Defines the object structure used to carry vendor-specific licensing information in a standardized container.

Relationship to Other Chapters

This chapter is closely related to:

- terminology,
- the reference model,
- normative requirements,
- the canonical API model,
- the security and trust framework,
- schemas and related annexes.

It is the main reference point for shared object definitions in the best practices.

Chapter Positioning

This chapter serves as the shared data-model anchor for the published baseline.

It is reviewed for consistency of object boundaries, naming, and alignment with the canonical API model, schemas, and security assumptions.

11.01. Lease Object

Purpose

This section defines the canonical lease object used by the Dynamic Software Licensing (DSL) best practices. The lease object represents the active or historical grant of licensed capability from the control plane to a specific workload identity.

The lease object is the runtime state anchor of the DSL model. It links entitlement, identity, policy, token issuance, lifecycle operations, and usage attribution.

Role of the Lease Object

The lease object records:

- which tenant and application the grant belongs to,
- which identity the grant is bound to,
- which dimensions and license modality were granted,
- which policy governs runtime behavior,
- what lifecycle state the lease is currently in,
- which token material and audit-relevant timestamps are associated with it.

The lease object exists independently of the token string that carries the runtime grant. A token is a portable signed artifact derived from the lease; the lease remains the control plane's authoritative record.

Canonical Structure

Field	Type	Description
<code>leaseId</code>	string	Unique identifier of the lease
<code>tenantId</code>	string	Tenant to which the lease belongs
<code>app</code>	string	Licensed application or product identifier
<code>entitlementId</code>	string	Source entitlement from which the lease was derived
<code>status</code>	string	Current lifecycle status of the lease
<code>licenseModality</code>	string	Granted modality governing lifecycle and metering behavior
<code>dimensions</code>	array	Granted dimensions and limits
<code>boundIdentity</code>	object	Verified identity to which the lease is bound
<code>policy</code>	object	Effective policy governing renewal, grace, and exhaustion
<code>issuedAt</code>	string	Time at which the lease was issued
<code>expiresAt</code>	string	End of current validity window

Field	Type	Description
<code>renewedAt</code>	string	Most recent successful renewal time, if any
<code>revokedAt</code>	string	Revocation time, if revoked
<code>workId</code>	string	Optional higher-level correlation identifier
<code>tokenRef</code>	object	Reference to the current signed token or token metadata

Required Semantics

Lifecycle status The `status` field expresses the control-plane view of the lease lifecycle. Canonical values should include at least:

- **ISSUED** — created and granted,
- **ACTIVE** — currently valid for runtime use,
- **RENEWED** — active after one or more successful renewals,
- **REISSUING** — in controlled transfer to a new identity,
- **REVOKED** — no longer permitted,
- **EXPIRED** — validity window ended without further continuation,
- **TERMINATED** — closed after controlled end of use.

Implementations may map these values to their internal state machine, but the externally visible semantics should remain consistent.

Granted dimensions The `dimensions` field contains the actual granted limits, not merely the originally requested values. Each entry should identify what is being limited and the amount granted.

```
{
  "name": "CHANNELS",
  "quantity": 4
}
```

Bound identity The `boundIdentity` field is the verified identity summary that the control plane accepted for this lease. It should be sufficient to explain binding and audit the lease, but it should not require raw identity evidence to remain embedded in the lease record.

Policy snapshot The `policy` field is the effective policy snapshot applied to this lease. It is the policy that must be enforced by the workspace plane during local validation and interruption.

Example Shape

```
{
  "leaseId": "L-01HY6G1Q3Z5W6E3Z2N7X0",
  "tenantId": "tenant-a",
  "app": "app-A",
  "entitlementId": "ENT-001",
  "status": "ACTIVE",
  "licenseModality": "hourly",
  "dimensions": [
    {
      "name": "CHANNELS",
      "quantity": 4
    }
  ],
  "boundIdentity": {
    "platform": "aws",
    "subject": "i-0abc123def456"
  },
  "policy": {
    "renewBeforeSeconds": 300,
    "offlineGraceSeconds": 1800,
    "onGraceExhausted": "DRAIN"
  },
  "issuedAt": "2026-08-12T10:00:00Z",
  "expiresAt": "2026-08-12T11:00:00Z",
  "workId": "deploy-i-0abc123def456"
}
```

Relationship to Other Objects

- The entitlement object defines the rights from which the lease is derived.
- The policy object defines the behavior snapshot carried by the lease.
- The token claims object describes the signed runtime representation of key lease data.
- The usage event object records consumption associated with the lease.

Section Positioning

This object is the canonical runtime grant record of the DSL model. It should be read as the stateful counterpart to the signed token used for local enforcement.

11.02. Entitlement Object

Purpose

This section defines the canonical entitlement object. An entitlement expresses the rights a tenant holds to use a licensed application or capability under defined limits, modalities, and policy conditions.

The entitlement object is the upstream authorization record from which leases are issued.

Role of the Entitlement Object

The entitlement object records:

- which tenant holds the right,
- which product or application is covered,
- which dimensions and limits may be granted,
- which modalities are permitted,
- which default policy applies,
- which scope restrictions or segmentation rules apply.

An entitlement is not itself a runtime grant. It authorizes the control plane to create one or more leases within its defined bounds.

Canonical Structure

Field	Type	Description
<code>entitlementId</code>	string	Unique identifier of the entitlement
<code>tenantId</code>	string	Tenant that owns the entitlement
<code>app</code>	string	Product or application identifier
<code>status</code>	string	Entitlement status such as ACTIVE , SUSPENDED , or EXPIRED
<code>allowedModalities</code>	array	Modalities that may be granted from this entitlement
<code>dimensions</code>	array	Maximum allowed dimensions and limits
<code>defaultPolicy</code>	object	Policy applied unless overridden by an allowed tenant rule

Field	Type	Description
<code>scope</code>	object	Optional project, environment, or operational scope restrictions
<code>quota</code>	object	Limits on simultaneous or aggregate grant behavior
<code>validFrom</code>	string	Start of entitlement validity
<code>validUntil</code>	string	End of entitlement validity, if bounded

Required Semantics

Product scope The `entitlement` must identify which licensed capability it governs. In simple cases this is a single application; in richer cases it may point to a product family, feature set, or structured catalog entry.

Dimensions and limits The `dimensions` field defines the maximum grantable usage scope, for example channels, frame rate, bandwidth, sessions, or process units. These are the upper bounds from which an issued lease draws its granted values.

Allowed modalities The `allowedModalities` field defines which modality or modalities are permitted for leases issued from this entitlement. If a requested modality is not present here, the control plane must not grant it.

Policy ownership The `defaultPolicy` field is the baseline vendor-defined policy attached to the entitlement. Tenant-level overrides may exist only where the vendor permits them and where such overrides remain auditable and governed.

Example Shape

```
{
  "entitlementId": "ENT-001",
  "tenantId": "tenant-a",
  "app": "app-A",
  "status": "ACTIVE",
  "allowedModalities": ["hourly", "daily"],
  "dimensions": [
    {
      "name": "CHANNELS",
      "maxQuantity": 4
    }
  ]
}
```

```
    },  
    {  
      "name": "FPS",  
      "maxQuantity": 60  
    }  
  ],  
  "defaultPolicy": {  
    "renewBeforeSeconds": 300,  
    "offlineGraceSeconds": 1800,  
    "onGraceExhausted": "DRAIN"  
  },  
  "quota": {  
    "maxActiveLeases": 2  
  }  
}
```

Relationship to Other Objects

- The lease object is derived from the entitlement object.
- The policy object supplies the behavioral policy referenced by the entitlement.
- The usage event object may later reconcile consumption against the granted rights derived from the entitlement.

Section Positioning

This object defines what a tenant is allowed to receive before any runtime lease exists. It is the commercial and operational source object for lease issuance.

11.03. Policy Object

Purpose

This section defines the canonical policy object used to govern runtime behavior of leases in the DSL model. The policy object carries the parameters that determine renewal timing, grace behavior, exhaustion handling, alerting, and related operational controls.

Role of the Policy Object

The policy object provides the behavioral rules that connect entitlement intent to runtime enforcement. It should make the operational meaning of a grant explicit and locally enforceable.

The machine-readable companion for this object is `api/policy.schema.json`. That schema mirrors the canonical structure below; individual policy instances may

omit optional fields when a profile or implementation does not use them.

Canonical Structure

Field	Type	Description
<code>renewBeforeSeconds</code>	integer	How early renewal should start before expiry
<code>offlineGraceSeconds</code>	integer	How long operation may continue after expiry without renewal
<code>onGraceExhausted</code>	string	Standardized exhaustion action
<code>heartbeatSeconds</code>	integer	Expected runtime or reporting heartbeat interval, if used
<code>renewBackoff</code>	object	Retry behavior for renewal attempts
<code>alertThresholds</code>	array	Thresholds at which operator alerts should fire
<code>revocationEndpoint</code>	string	Push endpoint information, if applicable
<code>tenantOverridesAllowed</code>	boolean	Whether tenant override of selected settings is permitted

Required Semantics

Explicit local enforceability The effective policy must be expressible in a way that the workspace plane can enforce locally while disconnected. Policy semantics must not depend on hidden control-plane logic only.

Standard exhaustion actions The canonical model recognizes three controlled exhaustion outcomes:

- **DRAIN** — finish current work and refuse new work,
- **DEGRADE** — continue with reduced capability,
- **STOP** — terminate licensed operation.

Implementations may extend these internally, but the meaning of the canonical set should remain stable for interoperability and operator understanding.

Retry behavior The `renewBackoff` object should define bounded retry semantics, for example:

```
{  
  "initialMs": 2000,  
  "maxMs": 30000,  
  "jitter": true  
}
```

Policy ownership and override The policy object is vendor-defined by default. Tenant override is not inherent; it is a governed capability that exists only where explicitly permitted and audited.

Example Shape

```
{  
  "renewBeforeSeconds": 300,  
  "offlineGraceSeconds": 1800,  
  "onGraceExhausted": "DRAIN",  
  "heartbeatSeconds": 60,  
  "renewBackoff": {  
    "initialMs": 2000,  
    "maxMs": 30000,  
    "jitter": true  
  },  
  "alertThresholds": [900, 300, 60],  
  "revocationEndpoint": "wss://lic.example.com/revocations",  
  "tenantOverridesAllowed": true  
}
```

Relationship to Other Objects

- The entitlement object carries the default policy source.
- The lease object carries the effective policy snapshot.
- The token claims object carries the runtime policy data required for local enforcement.

Section Positioning

This object is the canonical home for renewal, grace, exhaustion, and alert semantics. It provides the behavioral data model behind the operational and security expectations in the DSL best practices.

11.04. Usage Event Object

Purpose

This section defines the canonical usage event object used for metering and reporting in the DSL model. The usage event object records a unit of consumption

associated with a granted lease or equivalent runtime context.

Role of the Usage Event Object

The usage event object provides the canonical structure for:

- usage-based billing and reconciliation,
- operational visibility into consumed dimensions,
- post-outage usage replay,
- analytics and audit correlation.

Usage events are reporting artifacts. They support accounting and visibility, but they are not the primary runtime grant themselves.

Canonical Structure

Field	Type	Description
leaseId	string	Lease to which the usage applies
tenantId	string	Tenant context
productId	string	Application or product identifier
dimension	string	Metered dimension name
quantity	number	Amount consumed
granularity	string	Measurement granularity or unit
timestamp	string	Time at which the usage occurred
workId	string	Correlation identifier for deployment or workflow
submittedAt	string	Time the usage event reached the control plane

Required Semantics

Attribution Each usage event must be attributable to a tenant, product, and lease or equivalent grant context. The event should be sufficiently correlated to support operational and financial reconciliation.

Deduplication Usage event semantics must support idempotent replay. Retried or delayed events must not create double counting if the same logical event is submitted more than once.

Separation from enforcement Usage events should describe consumption, not directly carry authority. The authoritative grant remains the lease and its signed token.

Example Shape

```
{
  "leaseId": "L-01HY6G1Q3Z5W6E3Z2N7X0",
  "tenantId": "tenant-a",
  "productId": "app-A",
  "dimension": "HOURS",
  "quantity": 1,
  "granularity": "minutes",
  "timestamp": "2026-08-12T10:15:00Z",
  "workId": "deploy-i-0abc123def456"
}
```

Relationship to Other Objects

- The lease object provides the grant context for the usage event.
- The entitlement object defines which dimensions may be reported meaningfully.
- The policy object may govern heartbeat or submission cadence.

Section Positioning

This object is the canonical reporting unit for consumption-based DSL behavior. It provides the data-model bridge between runtime operation and billing, analytics, or audit workflows.

11.05. Token Claims Object

Purpose

This section defines the canonical claims carried in a signed runtime token. The token claims object is the portable, cryptographically protected representation of the lease data required for local validation and enforcement in the workspace plane.

Role of the Token Claims Object

The token claims object carries the subset of lease and policy data that the workspace plane needs at runtime without requiring a live round-trip to the control plane.

The canonical claims model should work whether the token is represented as a JWT, PASETO, or another established signed token format with equivalent semantics.

Canonical Structure

Field	Type	Description
<code>iss</code>	string	Issuer identifier of the control plane
<code>sub</code>	string	Subject of the grant, typically the bound workload or lease subject
<code>aud</code>	string	Intended application or service audience
<code>jti</code>	string	Unique token identifier
<code>iat</code>	string	Issued-at timestamp in RFC 3339 date-time form
<code>nbf</code>	string	Not-before timestamp in RFC 3339 date-time form
<code>exp</code>	string	Expiry timestamp in RFC 3339 date-time form
<code>tenantId</code>	string	Tenant context
<code>leaseId</code>	string	Lease identifier
<code>app</code>	string	Licensed application identifier
<code>dimensions</code>	array	Granted dimension values
<code>licenseModality</code>	string	Granted modality
<code>boundIdentity</code>	object	Binding summary needed for validation semantics
<code>policy</code>	object	Effective runtime policy snapshot

Required Semantics

Standard and DSL-specific claims The token claims object should combine widely understood standard claims such as issuer, audience, token identity, and validity window with DSL-specific claims describing the grant.

When a JWT representation is used, `iat`, `nbf`, `exp`, and `jti` map directly to the corresponding RFC 7519 registered claims. In the canonical DSL model, those

time values are represented as RFC 3339 date-time strings for portability across the textual model, JSON Schema artifacts, and examples.

Runtime sufficiency The claims carried in the token must be sufficient for local validation and runtime enforcement of the currently granted capability. That includes validity time, audience, dimensions, binding, and effective policy.

Bounded claim scope Only the claims needed for local validation and enforcement should be carried in the runtime token. Sensitive back-office or entitlement-management detail that is not required at runtime should remain in control-plane data stores rather than being pushed into every token.

Example Shape

```
{
  "iss": "https://lic.example.com",
  "sub": "workload:i-0abc123def456",
  "aud": "app-A",
  "jti": "tok-001",
  "iat": "2026-08-12T10:00:00Z",
  "nbf": "2026-08-12T10:00:00Z",
  "exp": "2026-08-12T11:00:00Z",
  "tenantId": "tenant-a",
  "leaseId": "L-01HY6G1Q3Z5W6E3Z2N7X0",
  "app": "app-A",
  "dimensions": [
    {
      "name": "CHANNELS",
      "quantity": 4
    }
  ],
  "licenseModality": "hourly",
  "boundIdentity": {
    "platform": "aws",
    "subject": "i-0abc123def456"
  },
  "policy": {
    "offlineGraceSeconds": 1800,
    "onGraceExhausted": "DRAIN"
  }
}
```

Relationship to Other Objects

- The lease object is the authoritative source from which the token claims are derived.

- The policy object contributes the locally enforceable policy snapshot.
- The identity evidence object informs the verified binding summarized in the claims.

Section Positioning

This object defines the portable runtime representation of a DSL grant. It is the canonical bridge between control-plane state and workspace-plane enforcement.

11.06. Identity Evidence Object

Purpose

This section defines the canonical identity evidence object used to present workload identity to the control plane. The identity evidence object carries the platform-specific proof that a requesting workload is entitled to receive or renew a bound lease.

Role of the Identity Evidence Object

The identity evidence object is the input-side counterpart to the `boundIdentity` summary that later appears in lease and token data. It provides the raw or near-raw evidence needed for the control plane to validate trust in the requester.

Canonical Structure

The canonical model uses a nested object keyed by evidence source or platform so that multiple runtime environments can be represented without changing the surrounding request structure.

Field	Type	Description
<code>aws</code>	object	AWS-specific identity evidence
<code>kubernetes</code>	object	Kubernetes or workload-identity evidence
<code>x509</code>	object	Certificate-based identity evidence
<code>custom</code>	object	Other governed platform evidence

The object does not require all fields at once. The relevant platform-specific sub-object is populated according to the deployment environment.

Example Shapes

AWS example

```
{
  "aws": {
    "iidDocument": "<base64 json>",
    "iidSignature": "<base64 pkcs7>"
  }
}
```

Kubernetes example

```
{
  "kubernetes": {
    "serviceAccountToken": "<jwt>",
    "cluster": "cluster-a",
    "namespace": "production"
  }
}
```

Required Semantics

Verification source The object must contain enough information for the control plane to verify the presented identity against a trusted authority or attestation source.

Separation from binding summary Identity evidence is request-time proof, not the long-term canonical representation of the binding. The control plane should validate the evidence and then retain a verified summary in the lease and token rather than embedding all raw evidence everywhere.

Platform variation, stable interpretation Evidence format may vary by platform, but successful validation must mean the same thing across implementations: the control plane has verified the requester's claimed workload identity.

Relationship to Other Objects

- The lease object stores the verified binding summary derived from the evidence.
- The token claims object carries the runtime binding summary, not the full raw evidence.
- The policy and security model govern what happens if evidence is absent, invalid, or stale.

Section Positioning

This object defines the canonical request-side trust input for issue, renew, and reissue. It is the data-model home for the identity-based binding principle.

11.07. License Envelope Object

Purpose

This section defines the canonical license envelope object. The license envelope is an optional container used when canonical DSL grant data must carry or accompany vendor-specific licensing artifacts in a structured, interoperable way.

No standalone JSON schema is defined for this object in the api directory. That is intentional: the envelope is a bridge object whose `payload` may remain vendor-specific and may therefore be carried either as structured JSON or as an opaque string, depending on `format`.

Role of the License Envelope Object

The DSL model seeks common lifecycle semantics without forcing every vendor to abandon its existing product-specific license representation immediately. The license envelope object provides a bridge for those cases by separating:

- canonical DSL metadata that all participants can understand, and
- vendor-specific payloads that remain necessary for a particular implementation.

Canonical Structure

Field	Type	Description
<code>leaseId</code>	string	Lease to which the envelope applies
<code>tenantId</code>	string	Tenant context
<code>app</code>	string	Licensed application identifier
<code>issuedAt</code>	string	Envelope creation time
<code>format</code>	string	Vendor payload format identifier
<code>payload</code>	object or string	Vendor-specific license content
<code>metadata</code>	object	Canonical metadata describing how to interpret the payload

Required Semantics

Canonical wrapper, vendor-specific body The envelope should make it clear which parts are canonically interpretable by DSL tooling and which parts remain vendor-specific. Canonical lifecycle semantics must not be hidden only in the vendor-specific payload.

Optional bridging mechanism The license envelope is not required for every implementation. If the canonical lease and token model are sufficient, no additional envelope is needed. The envelope exists for interoperability bridging and migration scenarios.

Explicit format declaration Where a vendor-specific payload is present, the envelope should declare its format explicitly so that clients know whether they can parse, pass through, or simply store the payload.

For the same reason, implementations should treat the field definitions in this section as the canonical baseline for interoperability rather than expecting a closed standalone schema for all possible envelope payload variants.

Example Shape

```
{
  "leaseId": "L-01HY6G1Q3Z5W6E3Z2N7X0",
  "tenantId": "tenant-a",
  "app": "app-A",
  "issuedAt": "2026-08-12T10:00:00Z",
  "format": "vendor-x-license-v2",
  "metadata": {
    "carrier": "embedded",
    "purpose": "feature-enablement"
  },
  "payload": {
    "vendorFeatureFlags": ["HDR", "MULTIVIEW"]
  }
}
```

Relationship to Other Objects

- The lease object provides the canonical grant context for the envelope.
- The token claims object provides the interoperable runtime enforcement data.
- The envelope may carry vendor-specific feature detail that is not part of the canonical token claims set.

Section Positioning

This object is the canonical bridge between the DSL common model and vendor-specific license payloads. It supports migration and interoperability without redefining the DSL core lifecycle or trust semantics.

12. Security and Trust Framework

Purpose

This chapter defines the security and trust framework of the Dynamic Software Licensing best practices.

Its purpose is to describe the security objectives, trust assumptions, and control areas that shape a trustworthy and production-ready licensing model.

Role Within the Best Practices

Security is a core principle of the DSL initiative, not a secondary appendix topic. Existing draft material repeatedly emphasizes cryptographic protection, auditability, least privilege, secure binding, and alignment with regulatory expectations.

This chapter provides the structured home for those topics.

Scope of This Chapter

This chapter defines:

- security objectives,
- threat considerations,
- trust assumptions,
- key control areas,
- audit and compliance expectations,
- offline and grace-related trust boundaries.

It explains how the best practices expects licensing to remain secure, auditable, and operationally usable.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Security Objectives Defines the high-level security goals of the best practices.

02. Threat Model Defines the main categories of security or misuse scenarios that should be considered.

03. Trust Assumptions Defines the assumptions that the system makes about identities, tokens, components, and control boundaries.

04. Security Controls Defines the main categories of security controls expected by the best practices.

05. Audit and Compliance Defines the role of auditability, traceability, and compliance alignment.

06. Offline and Grace Behavior Defines how the best practices treats continuity under partial disconnection or control-plane unavailability.

Relationship to Other Chapters

This chapter is closely related to:

- terminology,
- the reference model,
- normative requirements,
- the canonical API and data model,
- operational profiles,
- security-related annexes and diagrams.

It acts as the main security interpretation layer of the best practices.

Chapter Positioning

This chapter provides the security interpretation layer for the DSL best practices. It connects the reference model, lifecycle operations, and operational profiles to a coherent set of objectives, trust assumptions, controls, and audit expectations.

The chapter remains platform-agnostic. It defines the security posture and required behavior while leaving API payloads, schema details, and deployment-specific realizations to the canonical model, operational profiles, and informative annexes.

12.01. Security Objectives

Purpose

This section defines the high-level security objectives of the Dynamic Software Licensing (DSL) best practices. It states what the model is trying to protect, for whom, and why, and establishes the objectives against which the threat model (Section 12.02), trust assumptions (Section 12.03), and security controls (Section 12.04) are evaluated.

Security is a core design principle of the DSL initiative, not a secondary concern. The objectives below carry that principle into specific, testable goals.

Normative Language and Requirement Identifiers

This chapter uses the normative keywords **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** as defined in the conventions chapter (Chapter 03), which adopts the meanings of RFC 2119 as clarified by RFC 8174.

Where a requirement concerns a mechanism that an established standard already specifies, this chapter forwards to that standard and profiles it, rather than restating or reinventing it. The principal standards are listed in the references section of each subchapter.

Requirements in this chapter are identified with chapter-local prefixes to support traceability:

- **SEC-OBJ-n** — security objectives (this section),
- **THR-n** — threats (Section 12.02, a descriptive catalogue, not requirements),
- **TRUST-n** — trust assumptions (Section 12.03),
- **SEC-n** — security controls (Section 12.04),
- **AUD-n** — audit and compliance requirements (Section 12.05),
- **OFG-n** — offline and grace requirements (Section 12.06).

These identifiers are local to this chapter. The normative requirements chapter (Chapter 09, in particular its security requirements section) is expected to reference or consolidate them; where a conflict arises, Chapter 09 is authoritative for formal conformance.

Assets to Protect

The objectives protect the following assets:

- **Entitlements and leases** — the authoritative record of what a tenant may use and the active grant of that right.
- **License tokens** — the portable, signed artifacts that carry usage rights to the workspace plane.
- **Signing keys** — the cryptographic material whose compromise would undermine the entire trust model.
- **Usage and metering records** — the basis for charging, forecasting, and reconciliation.
- **Audit records** — the basis for non-repudiation and governance.
- **Commercial and policy data** — pricing, dimensions, and policy that are commercially sensitive.
- **Customer infrastructure and continuity** — the production environments that must keep running and that vendors must not be required to access.

Security Objectives

SEC-OBJ-1 — Integrity of entitlements and leases The integrity and authenticity of entitlements, leases, and the tokens that express them **MUST** be cryptographically protected end to end, such that they cannot be forged or altered without detection.

SEC-OBJ-2 — No vendor access to customer infrastructure The licensing model **MUST NOT** require a vendor to obtain access to, or credentials for, customer infrastructure in order to issue, validate, or enforce a license. Trust **MUST** be established through verifiable identity evidence and signed artifacts rather than through privileged access.

SEC-OBJ-3 — Continuity is not sacrificed to protection Security controls **MUST** balance intellectual-property protection against production continuity. Licensing **MUST NOT** become a single point of failure for live operation; protection mechanisms **MUST** degrade in a defined, bounded way rather than failing catastrophically.

SEC-OBJ-4 — Full auditability Every license lifecycle operation (issue, renew, reissue, revoke, usage reporting, and policy change) **MUST** be auditable, producing records sufficient to reconstruct who did what, to what, and when.

SEC-OBJ-5 — Tenant isolation Tenants **MUST** be isolated from one another. A tenant **MUST NOT** be able to read or affect another tenant's entitlements, leases, usage, or audit data.

SEC-OBJ-6 — Least privilege Every component and actor in the system **MUST** operate with the least privilege necessary for its function. Broad or standing privilege **MUST NOT** be the default.

SEC-OBJ-7 — Confidentiality of sensitive data Commercially sensitive and personal data (including usage records, pricing, and policy) **SHOULD** be protected in transit and at rest, and exposure **SHOULD** be minimized to what each party needs.

SEC-OBJ-8 — Zero-trust posture The model **MUST** assume no implicit trust between planes or components. Identity and authorization **MUST** be verified continuously rather than assumed from network location or prior interaction. This objective aligns with the Zero Trust Architecture tenets of NIST SP 800-207.

SEC-OBJ-9 — Bounded misuse and revenue protection The model **MUST** make license misuse (for example replay, over-use beyond entitled dimensions, or evasion of revocation) detectable and bounded, so that vendor revenue is protected without imposing constant connectivity on the customer.

Traceability

The objectives realize the security-related requirement themes from Chapter 05 and the design principles from Chapter 06:

Objective	Chapter 05 theme	Chapter 06 principle
SEC-OBJ-1 Integrity	T8 Security, trust, auditability	4 Security by Design
SEC-OBJ-2 No vendor access	T8	4 Security by Design
SEC-OBJ-3 Continuity vs protection	T9 Continuity under failure	5 Resilience and Continuity
SEC-OBJ-4 Auditability	T8	4, 6 Observability
SEC-OBJ-5 Tenant isolation	T8, T7 Access control	4
SEC-OBJ-6 Least privilege	T8, T7	4
SEC-OBJ-7 Confidentiality	T8, T5 Metering transparency	4
SEC-OBJ-8 Zero-trust	T8	4
SEC-OBJ-9 Bounded misuse	T8, T9	4, 5

References

- RFC 2119; RFC 8174 — interpretation of the normative keywords used in this chapter.
- NIST SP 800-207 — Zero Trust Architecture — SEC-OBJ-8.

Relationship to Other Chapters

- The threat model (Section 12.02) identifies what endangers these objectives.
- The trust assumptions (Section 12.03) state the conditions under which the objectives can be met.
- The security controls (Section 12.04) are the mechanisms that achieve the objectives.
- The reference model (Chapter 07) defines the planes and components these objectives apply to; the canonical API and data model define the artifacts; Chapter 09 carries the formal conformance requirements.

This section is normative in intent but states goals rather than mechanisms; the mechanisms are specified as controls in Section 12.04. It remains platform-agnostic: specific key services, identity providers, and transports are reserved for the informative annexes.

12.02. Threat Model

Purpose

This section defines the threat model for the Dynamic Software Licensing (DSL) best practices. It catalogues the principal security and misuse scenarios that the model must withstand, and provides the structured basis for the trust assumptions (Section 12.03) and the security controls (Section 12.04).

This section is descriptive: it identifies threats but does not itself state control requirements. Each threat is given an identifier (**THR-n**) so that controls in Section 12.04 can cite the threats they mitigate.

Method

The threat model is expressed in four parts:

1. the assets at risk (carried forward from Section 12.01),
2. the actors and adversary classes,
3. the trust boundaries across which threats act,
4. the catalogue of threats, each mapped to the objectives it endangers.

The model is deliberately platform-agnostic. It addresses threats to the licensing model itself, not the general security of the underlying platform (for example host hardening, network perimeter, or physical security), which is the responsibility of the platform and infrastructure providers.

The threat categories follow established threat-modelling practice (for example the STRIDE categorization, and the OWASP Application Security Verification Standard as a control checklist). Where adopters add a likelihood/impact rating, an established scoring method such as CVSS, applied within a risk process such as NIST SP 800-30, **SHOULD** be used rather than an ad-hoc scale.

Adversary Classes

The model considers the following adversary classes:

- **External attacker** — has no legitimate entitlement and seeks to obtain, forge, or prolong licensed capability.
- **Compromised workload** — a workspace-plane component under adversary control that attempts to abuse a legitimate position.
- **Malicious or careless insider (customer side)** — a user with some legitimate access who attempts to exceed entitlements or evade controls, deliberately or accidentally.
- **Over-reaching vendor or control-plane operator** — a party who attempts, or is wrongly required, to obtain more access or data than the model intends.
- **Network adversary** — able to observe or modify traffic between planes.

- **Over-using customer** — not malicious, but consuming beyond entitled dimensions in ways that, if unmetered, cause revenue leakage.
- **Supply-chain adversary** — able to tamper with the license agent, SDK, or its distribution.

Trust Boundaries

Threats act across the trust boundaries defined by the reference model (Chapter 07):

- the boundary between the **control plane** (the trust authority for entitlements, issuance, keys, and policy) and the **workspace plane** (lower-trust execution environments),
- the boundary between **tenants**,
- the boundary between the **customer environment** and the **vendor / control-plane operator**,
- the **network** path connecting planes.

The license token is the primary trust-carrying artifact that crosses the control-plane to workspace-plane boundary, and is therefore a focal point of the threat model.

Threat Catalogue

THR-1 — Token forgery An adversary fabricates or alters a license token to grant capability that was never issued, or to widen granted dimensions. Endangers SEC-OBJ-1, SEC-OBJ-9.

THR-2 — Token replay or reuse A valid token is captured and replayed after expiry, or reused on a workload other than the one it was issued for, to obtain unentitled capability. Endangers SEC-OBJ-1, SEC-OBJ-9.

THR-3 — Identity spoofing An adversary presents falsified identity evidence to obtain a lease for a workload it does not control or is not entitled to. Endangers SEC-OBJ-1, SEC-OBJ-2, SEC-OBJ-8.

THR-4 — Tampering in transit or at rest Lease, policy, dimension, or usage data is modified in transit between planes or in storage, to alter entitlements or to misreport usage. Endangers SEC-OBJ-1, SEC-OBJ-7.

THR-5 — Over-use and limit evasion A workload operates beyond its entitled dimensions (for example more channels or higher throughput than granted) or suppresses usage reporting, causing revenue leakage. Endangers SEC-OBJ-9.

THR-6 — Revocation evasion A revoked lease continues to be used because the revocation is ignored, the workload is kept offline to avoid receiving it, or the exposure window is unbounded. Endangers SEC-OBJ-9, SEC-OBJ-3.

THR-7 — Abuse of offline and grace behavior Grace and offline operation are exploited to extend usage indefinitely, for example by deliberately severing connectivity or tampering with the local clock to defeat expiry. Endangers SEC-OBJ-9, SEC-OBJ-3.

THR-8 — Signing key compromise or weak key management A signing key is exposed, stolen, or not rotated, allowing wholesale forgery of tokens. Endangers SEC-OBJ-1 (highest-impact threat in the model).

THR-9 — Cross-tenant access A tenant gains read or write access to another tenant's entitlements, leases, usage, or audit data through an isolation failure. Endangers SEC-OBJ-5, SEC-OBJ-7.

THR-10 — Exfiltration of sensitive data Usage, pricing, policy, or personal data is disclosed to a party that should not see it, whether across tenants, to an over-reaching operator, or to a network adversary. Endangers SEC-OBJ-7.

THR-11 — Denial of service against control functions The control plane or its interfaces are overwhelmed or made unavailable, preventing issue and renewal and thereby threatening production continuity. Endangers SEC-OBJ-3.

THR-12 — Repudiation A party disputes a usage record, a lifecycle action, or a policy change because adequate, tamper-evident audit evidence is absent. Endangers SEC-OBJ-4.

THR-13 — Excessive privilege into the customer environment The model or an implementation requires the vendor or control plane to hold standing, privileged access into the customer environment, expanding the attack surface and violating least privilege. Endangers SEC-OBJ-2, SEC-OBJ-6.

THR-14 — Supply-chain compromise of the agent or SDK The license agent, SDK, or its distribution channel is tampered with, subverting local validation and enforcement at the source. Endangers SEC-OBJ-1, SEC-OBJ-8.

Threat-to-Objective Summary

Threat	Primary objectives endangered
THR-1 Token forgery	SEC-OBJ-1, SEC-OBJ-9
THR-2 Replay / reuse	SEC-OBJ-1, SEC-OBJ-9

Threat	Primary objectives endangered
THR-3 Identity spoofing	SEC-OBJ-1, SEC-OBJ-2, SEC-OBJ-8
THR-4 Tampering	SEC-OBJ-1, SEC-OBJ-7
THR-5 Over-use / evasion	SEC-OBJ-9
THR-6 Revocation evasion	SEC-OBJ-9, SEC-OBJ-3
THR-7 Grace / offline abuse	SEC-OBJ-9, SEC-OBJ-3
THR-8 Key compromise	SEC-OBJ-1
THR-9 Cross-tenant access	SEC-OBJ-5, SEC-OBJ-7
THR-10 Data exfiltration	SEC-OBJ-7
THR-11 Denial of service	SEC-OBJ-3
THR-12 Repudiation	SEC-OBJ-4
THR-13 Excessive privilege	SEC-OBJ-2, SEC-OBJ-6
THR-14 Supply-chain compromise	SEC-OBJ-1, SEC-OBJ-8

Out of Scope

The following are acknowledged but are not addressed by this chapter, being the responsibility of platform and infrastructure providers or of other chapters:

- general hardening of hosts, containers, and orchestration platforms,
- network perimeter and physical security,
- vulnerabilities in the licensed application itself unrelated to licensing,
- availability of the underlying platform beyond the licensing control functions.

References

- STRIDE — threat categorization methodology — informs the threat catalogue structure.
- OWASP Application Security Verification Standard (ASVS) — control checklist for verification.
- CVSS — Common Vulnerability Scoring System — optional likelihood/impact rating.
- NIST SP 800-30 — Guide for Conducting Risk Assessments — risk process.

Relationship to Other Chapters

- The trust assumptions (Section 12.03) state which of these threats are addressed by design assumption versus by active control.
- The security controls (Section 12.04) cite these threat identifiers in their mitigations.
- The reference model (Chapter 07) defines the planes and boundaries referenced here.

This section is descriptive. The threat catalogue is intended to be stable and citable; it does not use normative keywords. Controls and their normative strength are defined in Section 12.04.

12.03. Trust Assumptions

Purpose

This section states the trust assumptions of the Dynamic Software Licensing (DSL) model: what the model treats as trustworthy, what it treats as untrusted, and the external dependencies on which its security rests. These assumptions explain *how* trust is established and bound, and they delimit the conditions under which the security objectives (Section 12.01) can be met against the threats (Section 12.02).

Assumptions are identified as **TRUST-n**. Where an assumption imposes an obligation it uses normative language; where it merely records a precondition it is stated declaratively.

Trust Boundary Model

The DSL trust model centres on a clear asymmetry between the two planes defined in the reference model (Chapter 07):

- The **control plane** is the trust authority. It is the source of truth for entitlements, performs issuance, holds signing keys, defines and enforces policy, and records audit.
- The **workspace plane** is treated as a lower-trust execution environment. It is where applications run and where tokens are validated and enforced locally.
- The **license agent** mediates between the application and the control plane. It is trusted to perform local validation and enforcement, but it is not trusted with secrets that would let it forge entitlement.
- The **token** is the trust-carrying artifact. Because it is signed by the control plane and validated locally, trust can cross the plane boundary without the workspace plane needing privileged access to the control plane.

Trust Assumptions

TRUST-1 — The control plane is the entitlement authority The control plane is assumed to be the sole authority for entitlement decisions, lease issuance, and policy. The workspace plane **MUST NOT** be able to grant itself entitlement or widen its own dimensions; it can only validate and enforce what the control plane has signed.

TRUST-2 — Signing keys remain in protected custody Private signing keys are assumed to be held in a managed key service or hardware security module. Private key material **MUST NOT** be distributed to, or be recoverable by, the workspace plane or any client. The security of the entire model rests on this assumption (see THR-8).

TRUST-3 — The workspace plane is potentially hostile The workspace plane is assumed to be potentially compromised or adversarial. Enforcement therefore relies on locally verifiable signed tokens with bounded validity, not on trusting the workload’s self-report of its own state. Controls are designed so that compromise of a single workload yields bounded, time-limited gain.

TRUST-4 — Identity evidence is verifiable and rooted in a trusted authority Workload identity evidence is assumed to be verifiable against a trusted identity provider or attestation source — for example an OpenID Connect provider or a workload identity framework such as SPIFFE/SVID. The strength of a binding is only as strong as the identity evidence behind it; the model assumes the chosen identity source is itself trustworthy and that its evidence cannot be trivially forged (see THR-3).

TRUST-5 — Local time is reasonably accurate Local validation of expiry and not-before times assumes a reasonably accurate local clock. Time synchronization **SHOULD** use a standard protocol — Network Time Protocol (RFC 5905), preferably with Network Time Security (RFC 8915) where integrity of time matters. Because clocks can still drift or be tampered with, validity windows **MUST** be kept short and offline operation **MUST** be bounded, so that the value of clock tampering is limited (see THR-7).

TRUST-6 — Public keys are discoverable and authenticated Consumers are assumed to be able to discover the control plane’s current public signing keys through an authenticated, integrity-protected channel — for example a JSON Web Key Set (RFC 7517) retrieved over TLS (RFC 8446). A consumer **MUST** authenticate the origin of public keys so that an adversary cannot substitute its own.

TRUST-7 — Tenant isolation is enforced by the control plane The control plane is assumed to enforce tenant isolation for all entitlement, lease, usage, and audit data. The model does not rely on workspace-plane cooperation to maintain isolation (see THR-9).

TRUST-8 — Mutual minimization of trust between vendor and customer The model assumes neither party must over-trust the other. The customer is not required to grant the vendor access to its infrastructure (addressed by identity evidence and signed tokens), and the vendor is not required to trust the customer’s unverified self-report (addressed by signed enforcement

and metered, audited usage). This mutual minimization is a defining assumption of the model and directly serves SEC-OBJ-2 and SEC-OBJ-9.

TRUST-9 — Transport is authenticated and confidential Communication between planes is assumed to occur over an authenticated, confidential channel. The model assumes transport-level protection is in place; it does not assume the network itself is trustworthy (see THR-4, THR-10).

External Dependencies

The model's security depends on the correctness and availability of:

- a key management service or hardware security module for key custody and rotation,
- one or more trusted identity providers or attestation sources for identity evidence,
- a public-key distribution mechanism,
- transport security (for example mutually authenticated TLS),
- reasonably synchronized time.

A weakness in any of these dependencies weakens the corresponding assumption above. Adopters **SHOULD** evaluate these dependencies as part of their own threat assessment.

What the Model Does Not Assume

- It does **not** assume continuous connectivity between planes; bounded disconnection is handled by offline and grace behavior (Section 12.06).
- It does **not** assume the integrity of the workspace plane; it assumes the opposite (TRUST-3).
- It does **not** assume that the customer trusts the vendor with infrastructure access, nor the reverse (TRUST-8).

References

- FIPS 140-3; NIST SP 800-57 — key custody and management — TRUST-2.
- OpenID Connect; SPIFFE/SVID — identity providers and workload identity — TRUST-4.
- RFC 5905 (NTP); RFC 8915 (NTS) — time synchronization — TRUST-5.
- RFC 7517 (JWK Set); RFC 8446 (TLS 1.3) — authenticated public-key discovery — TRUST-6, TRUST-9.

Relationship to Other Chapters

- The security controls (Section 12.04) implement the obligations implied by these assumptions and compensate where an assumption is weak.

- The offline and grace behavior (Section 12.06) defines the bounded relaxation of the connectivity assumption.
- The reference model (Chapter 07) and the security framework objectives (Section 12.01) provide the structural and goal context.

This section records assumptions and the obligations they imply. It is intentionally explicit about what is *not* assumed, because over-stated trust is itself a security risk. It remains platform-agnostic; concrete identity providers and key services are reserved for the informative annexes.

12.04. Security Controls

Purpose

This section defines the security controls of the Dynamic Software Licensing (DSL) best practices. The controls are the normative mechanisms that achieve the security objectives (Section 12.01) and mitigate the threats (Section 12.02), within the trust assumptions (Section 12.03).

Controls are identified as **SEC-n** and use the normative keywords defined in Chapter 03. Each control records the threats it mitigates. Detailed payloads, claim names, endpoints, and algorithms are out of scope here; they belong to the canonical API and data model and to the informative annexes.

Control Families

The controls are organized into families:

- A. Cryptographic integrity and key custody
- B. Token validity and replay resistance
- C. Identity and binding
- D. Secure transport
- E. Least privilege and access control
- F. Revocation and enforcement
- G. Tenant isolation
- H. Agent and supply-chain integrity

A. Cryptographic Integrity and Key Custody

SEC-1 — Signed artifacts Every license token **MUST** be cryptographically signed by the control plane such that any alteration is detectable. Implementations **SHOULD** use an established signature container rather than a bespoke scheme — for example JSON Web Signature (RFC 7515) over a JSON Web Token (RFC 7519), or PASETO — with algorithms drawn from a standard registry (RFC 7518). *Mitigates THR-1, THR-4.*

SEC-2 — Local signature validation A workspace-plane component (application or agent) **MUST** validate a token’s signature, issuer, audience, and validity times before granting or continuing licensed capability, and **MUST** follow the JSON Web Token Best Current Practices (RFC 8725) — in particular explicit algorithm allow-listing to prevent algorithm-confusion attacks. Validation **MUST** be performable locally without a per-use round-trip to the control plane. *Mitigates THR-1, THR-2.*

SEC-3 — Protected key custody Private signing keys **MUST** be generated and held in a managed key service or hardware security module (for example a module validated to FIPS 140-3), and managed in line with established key-management guidance (NIST SP 800-57). Private key material **MUST NOT** be distributed to the workspace plane or embedded in any client. *Mitigates THR-8.*

SEC-4 — Key rotation The control plane **MUST** support signing-key rotation without service interruption. Consumers **MUST** resolve the current public keys through an authenticated discovery mechanism — for example a JSON Web Key Set (RFC 7517) referenced by key identifier (the `kid` header of RFC 7515) — and **MUST** support multiple concurrently valid keys so that rotation does not prematurely invalidate tokens already issued. Rotation cadence **SHOULD** follow NIST SP 800-57. Compromised keys **MUST** be revocable. *Mitigates THR-8.*

SEC-5 — Encryption of sensitive data at rest Commercially sensitive and personal data held by the control plane (including usage, pricing, and policy) **SHOULD** be encrypted at rest. *Mitigates THR-10.*

B. Token Validity and Replay Resistance

SEC-6 — Short-lived tokens License tokens **MUST** be short-lived, carrying explicit issued-at, expiry, and not-before times (the `iat`, `exp`, and `nbf` claims of RFC 7519 where JWT is used). Token lifetime **SHOULD** be the minimum compatible with the operational continuity needs of the use case. *Mitigates THR-2, THR-6, THR-7.*

SEC-7 — Unique token identity Each token **MUST** carry a unique identifier sufficient to support replay detection and correlation in audit (the `jti` claim of RFC 7519 where JWT is used). *Mitigates THR-2, THR-12.*

SEC-8 — Workload-bound tokens Each token **MUST** be bound to the specific workload identity it was issued for, so that a captured token cannot be used on a different workload. Sender-constrained (proof-of-possession) token mechanisms **SHOULD** be used rather than a bespoke binding — for example mutual-TLS-bound tokens (RFC 8705) or DPoP (RFC 9449), following the proof-of-possession semantics of RFC 7800. *Mitigates THR-2, THR-3.*

C. Identity and Binding

SEC-9 — Verified identity evidence Issue and renew operations **MUST** require verifiable workload identity evidence, and the control plane **MUST** verify it against a trusted identity source before issuing or extending a lease. Standard machine-to-machine authentication and identity mechanisms **SHOULD** be used rather than proprietary ones — for example the OAuth 2.0 client-credentials grant (RFC 6749), OpenID Connect, or workload identity frameworks such as SPIFFE/SVID. *Mitigates THR-3.*

SEC-10 — Identity-based binding Binding **MUST** use verifiable workload identity. Binding **MUST NOT** rely solely on static host attributes such as hostname or MAC address. *Mitigates THR-3.* (See also the design principle on identity-based binding in Chapter 06.)

SEC-11 — Fail-closed on invalid identity If identity evidence is absent, invalid, or unverifiable, issuance or renewal **MUST** be denied. The system **MUST NOT** fall back to host-based binding or to an unauthenticated grant. *Mitigates THR-3.*

D. Secure Transport

SEC-12 — Authenticated, confidential transport All communication between the workspace plane and the control plane, including the push channel used for revocation and policy updates, **MUST** use authenticated, confidential transport: TLS 1.3 (RFC 8446) or later, with mutually authenticated TLS where feasible. *Mitigates THR-4, THR-10.*

SEC-13 — Private connectivity options For environments with stricter requirements, private network connectivity options **SHOULD** be supported in addition to public endpoints. *Mitigates THR-10, THR-11.*

E. Least Privilege and Access Control

SEC-14 — Least-privilege components Every control-plane function and workspace-plane component **MUST** operate with the least privilege necessary for its role. *Mitigates THR-13, THR-9.*

SEC-15 — No privileged vendor access The model **MUST NOT** require standing privileged access by the vendor or control-plane operator into the customer environment. The customer presents identity evidence; the vendor does not reach into customer infrastructure. *Mitigates THR-13.* (Realizes SEC-OBJ-2.)

SEC-16 — Per-tenant authorization Every request **MUST** be authorized in the context of its authenticated tenant. A request **MUST NOT** be able to read or modify another tenant’s entitlements, leases, usage, or audit data. *Mitigates THR-9.*

SEC-17 — Role- and scope-based entitlement access Access to entitlements **SHOULD** be governable by user, role, and scope (for example project- or production-scoped pools and time-boxed external access), with least-privilege defaults. Established access-control models **SHOULD** be used rather than ad-hoc schemes — for example role-based access control (ANSI/INCITS 359), attribute-based access control (NIST SP 800-162), or OAuth 2.0 scopes (RFC 6749). Emergency overrides **SHOULD** be permitted but **MUST** be audited. *Mitigates THR-5, THR-9. (Realizes Chapter 05 theme T7.)*

F. Revocation and Enforcement

SEC-18 — Revocation capability The control plane **MUST** support revocation of an active lease (conceptually analogous to OAuth 2.0 token revocation, RFC 7009). When the affected workload is reachable, revocation **SHOULD** be delivered promptly through the push channel; when it is not reachable, revocation **MUST** take effect no later than the next renewal. *Mitigates THR-6.*

SEC-19 — Bounded exposure window The maximum time a revoked or expired entitlement can remain in use **MUST** be bounded by the token lifetime and the offline grace bound (Section 12.06). *Mitigates THR-6, THR-7.*

SEC-20 — Enforcement of granted dimensions The workspace plane **MUST** enforce the dimensions and limits carried in the validated token, and **MUST NOT** permit operation beyond them. *Mitigates THR-5.*

G. Tenant Isolation

SEC-21 — Logical tenant separation Tenant data **MUST** be logically separated, and isolation **MUST** be enforced by the control plane rather than relying on workspace-plane cooperation. Cryptographic separation of per-tenant data **SHOULD** be applied where feasible. *Mitigates THR-9, THR-10.*

H. Agent and Supply-Chain Integrity

SEC-22 — Integrity-verified distribution The license agent and SDK **SHOULD** be distributed as integrity-verified, signed artifacts through trustworthy channels, so that consumers can verify authenticity before deployment. Established secure-development and supply-chain practices **SHOULD** be applied — for example the NIST Secure Software Development Framework (NIST SP 800-218), SLSA provenance, and artifact signing (such as Sigstore or in-toto attestations). *Mitigates THR-14.*

SEC-23 — Minimal client secrets Clients **MUST NOT** hold secrets that would allow them to forge entitlement or bypass validation. Any client credentials **MUST** be limited to authenticating requests, not to granting capability. *Mitigates THR-8, THR-14.*

Control-to-Threat Matrix

Threat	Mitigating controls
THR-1 Token forgery	SEC-1, SEC-2
THR-2 Replay / reuse	SEC-2, SEC-6, SEC-7, SEC-8
THR-3 Identity spoofing	SEC-8, SEC-9, SEC-10, SEC-11
THR-4 Tampering	SEC-1, SEC-12
THR-5 Over-use / evasion	SEC-17, SEC-20
THR-6 Revocation evasion	SEC-6, SEC-18, SEC-19
THR-7 Grace / offline abuse	SEC-6, SEC-19 (with Section 12.06)
THR-8 Key compromise	SEC-3, SEC-4, SEC-23
THR-9 Cross-tenant access	SEC-14, SEC-16, SEC-17, SEC-21
THR-10 Data exfiltration	SEC-5, SEC-12, SEC-13, SEC-21
THR-11 Denial of service	SEC-13 (with availability design in Chapter 13)
THR-12 Repudiation	SEC-7 (with audit controls in Section 12.05)
THR-13 Excessive privilege	SEC-14, SEC-15
THR-14 Supply-chain compromise	SEC-22, SEC-23

Two threats are mitigated primarily outside this section: THR-11 (denial of service) depends on availability and scaling design in the operational and deployment profiles (Chapter 13), and THR-12 (repudiation) depends on the audit controls in Section 12.05. These are noted here so the matrix does not appear to leave them uncovered.

References

This section profiles, rather than restates, established standards. The following are the principal references; specific algorithm and parameter choices are deferred to the canonical model and informative annexes.

Normative references (mechanisms a conforming implementation builds on):

- RFC 7515 — JSON Web Signature (JWS) — SEC-1, SEC-4.
- RFC 7517 — JSON Web Key (JWK / JWK Set) — SEC-4.
- RFC 7518 — JSON Web Algorithms (JWA) — SEC-1.
- RFC 7519 — JSON Web Token (JWT), including the `iat`, `exp`, `nbf`, and `jti` claims — SEC-1, SEC-6, SEC-7.

- RFC 8725 — JSON Web Token Best Current Practices — SEC-2.
- RFC 8705 — OAuth 2.0 Mutual-TLS client authentication and certificate-bound access tokens — SEC-8, SEC-12.
- RFC 9449 — OAuth 2.0 Demonstrating Proof of Possession (DPoP) — SEC-8.
- RFC 7800 — Proof-of-Possession key semantics for JWTs — SEC-8.
- RFC 6749 — OAuth 2.0 Authorization Framework (client-credentials grant; scopes) — SEC-9, SEC-17.
- RFC 8446 — TLS 1.3 — SEC-12.
- RFC 7009 — OAuth 2.0 Token Revocation (conceptual analogue) — SEC-18.

Informative references (guidance and good practice):

- NIST SP 800-57 — Recommendation for Key Management — SEC-3, SEC-4.
- FIPS 140-3 — Security requirements for cryptographic modules — SEC-3.
- NIST SP 800-162 — Attribute-Based Access Control — SEC-17.
- ANSI/INCITS 359 — Role-Based Access Control — SEC-17.
- NIST SP 800-218 — Secure Software Development Framework (SSDF) — SEC-22.
- SLSA, Sigstore, in-toto — supply-chain provenance and artifact signing — SEC-22.
- OpenID Connect; SPIFFE/SVID — workload and federated identity — SEC-9.
- PASETO — alternative signed-token format — SEC-1.

Relationship to Other Chapters

- The objectives (Section 12.01) state the goals these controls achieve; the threat model (Section 12.02) defines what they defend against.
- The canonical API model (Chapter 10) defines how the lifecycle operations referenced here are invoked; the canonical data model defines the artifacts; the security and trust controls are the constraints on both.
- The normative requirements chapter (Chapter 09) is expected to reference these controls in its security requirements section.

This section is normative. It states mechanisms and their required strength while remaining platform-agnostic and free of API- and schema-level detail. Algorithm choices, claim names, and endpoint definitions are deliberately deferred to the canonical model and annexes.

12.05. Audit and Compliance

Purpose

This section defines the audit and compliance requirements of the Dynamic Software Licensing (DSL) best practices. Auditability underpins non-repudiation, dispute resolution, and operation in regulated environments. This section realizes objective SEC-OBJ-4 and mitigates the repudiation threat (THR-12).

Requirements are identified as **AUD-n** and use the normative keywords defined in Chapter 03.

Audit Requirements

AUD-1 — Comprehensive lifecycle auditing All license lifecycle operations — issue, renew, reissue, revoke, usage reporting, and policy change — **MUST** be recorded in an audit log. *Realizes SEC-OBJ-4.*

AUD-2 — Sufficient audit content Each audit record **MUST** capture enough to reconstruct the event: the actor or component, the action, the affected entitlement or lease, the relevant correlation identifiers (for example the work and lease identifiers), the outcome, and an accurate timestamp. *Mitigates THR-12.*

AUD-3 — Append-only and tamper-evident Audit records **MUST** be append-only and tamper-evident, so that records cannot be silently altered or deleted after the fact. Log management **SHOULD** follow established guidance (NIST SP 800-92), and tamper-evidence **SHOULD** use a recognized mechanism such as cryptographically signed log entries (for example RFC 5848) or hash-chaining rather than a bespoke scheme. *Mitigates THR-12, THR-4.*

AUD-4 — Policy-change auditing Creation and modification of policy and entitlements — including vendor defaults and any permitted tenant overrides — **MUST** be audited, recording who changed what and when. *Mitigates THR-12.* (See the policy governance discussion in Section 12.06.)

AUD-5 — Trustworthy usage records Usage records **MUST** be recorded in a manner that is tamper-evident and idempotent, so that buffered or retried submissions cannot cause double counting and so that usage cannot be silently understated. *Mitigates THR-5, THR-12.* (See also the metering theme T5 in Chapter 05.)

AUD-6 — Retention and protection Audit and usage records **SHOULD** be retained according to a defined retention policy and **MUST** be protected against unauthorized access, modification, and deletion for their retention period. *Mitigates THR-10, THR-12.*

AUD-7 — Exportability for governance The system **SHOULD** support export of audit and usage data in a consumable form, so that finance, operations, security, and governance functions can reconcile, report, and investigate. (Supports Chapter 05 themes T5 and T6.)

Compliance Alignment

AUD-8 — Alignment with applicable frameworks The model **SHOULD** be implementable in a way that aligns with the compliance frameworks applicable to the adopter’s environment. Representative frameworks relevant to media and broadcast operations include:

- **ISO/IEC 27001** and **ISO/IEC 27002** — information security management and controls, with **ISO/IEC 27017** (cloud security) and **ISO/IEC 27018** (PII in public cloud) where cloud deployment applies;
- **GDPR** (Regulation (EU) 2016/679) and comparable data-protection regimes — protection of personal data;
- **TPN (Trusted Partner Network)** — content security for media and entertainment;
- **SOC 2** (AICPA Trust Services Criteria) — where customers require service-organization assurance.

These are illustrative, not exhaustive or mandatory. The specific obligations that apply depend on the adopter’s jurisdiction, customers, and content. This best practice does not certify compliance; it aims not to obstruct it.

AUD-9 — Data minimization Personal and commercially sensitive data **SHOULD** be minimized to what is necessary for licensing, metering, and audit, and **SHOULD NOT** be collected or retained beyond that need. *Supports GDPR-style obligations; mitigates THR-10.*

AUD-10 — Separation of duties for sensitive actions Where an adopter’s compliance regime requires it, sensitive administrative actions (for example key management or bulk revocation) **SHOULD** support separation of duties and be individually attributable in the audit record. *Mitigates THR-12, THR-13.*

Audit and the Trust Model

Audit is the mechanism by which the mutual minimization of trust (TRUST-8) is made accountable. Because neither party fully trusts the other, both rely on tamper-evident records to resolve disputes:

- the vendor relies on trustworthy usage and lifecycle records to substantiate charging,
- the customer relies on the same records to verify that charging reflects actual, entitled use,

- both rely on policy-change auditing to confirm that grace and entitlement changes were authorized.

Auditability therefore is not only a control; it is a precondition for the commercial trust the model depends on.

References

- NIST SP 800-92 — Guide to Computer Security Log Management — AUD-1 to AUD-3.
- RFC 5848 — Signed Syslog Messages — tamper-evident logging — AUD-3.
- NIST SP 800-53 (control AC-5, Separation of Duties) — AUD-10.
- ISO/IEC 27001, 27002, 27017, 27018 — information security management — AUD-8.
- GDPR (Regulation (EU) 2016/679) — personal-data protection and minimization — AUD-8, AUD-9.
- TPN (Trusted Partner Network) — media content security — AUD-8.
- SOC 2 (AICPA Trust Services Criteria) — service-organization assurance — AUD-8.

These compliance frameworks are referenced as informative alignment targets; this best practice does not assert or confer certification against any of them.

Relationship to Other Chapters

- The security controls (Section 12.04) protect the integrity of the records described here (notably SEC-1, SEC-7, SEC-21).
- The metering and observability themes in Chapter 05 (T5, T6) and the operational profiles (Chapter 13) consume the exported data described in AUD-7.
- The canonical data model defines the structure of audit and usage records; this section defines their required properties, not their schema.

This section is normative for auditability and advisory for compliance alignment. It names representative compliance frameworks as examples only and avoids asserting certification. It remains platform-agnostic: specific log stores, retention services, and reporting tools are reserved for the informative annexes.

12.06. Offline and Grace Behavior

Purpose

This section defines how the Dynamic Software Licensing (DSL) model treats continuity under partial disconnection or control-plane unavailability. It establishes the trust and risk boundaries of offline validation, grace policies, renewal retries, and bounded continuity.

This is where the model's central tension is resolved in concrete terms: the protection objectives (Section 12.01) versus the requirement that licensing never become a single point of failure for live production (SEC-OBJ-3, Chapter 05 theme T9). Requirements are identified as **OFG-n** and use the normative keywords defined in Chapter 03.

Principle

The model is designed so that an application does not need a constant connection to a licensing service. It operates on a short-lived, locally validated token it already holds, and needs connectivity only to renew that token or to report usage. Offline and grace behavior governs what happens between successful renewals when connectivity is degraded or lost.

The design intent is that disconnection produces **defined, bounded, observable** behavior — never an abrupt, unexplained production failure, and never unbounded unlicensed operation.

Offline and Grace Requirements

OFG-1 — Local validation without round-trips A workspace-plane component **MUST** be able to validate its current token locally, without a per-use round-trip to the control plane, so that transient disconnection does not interrupt operation. Local validation relies on the signed-token mechanisms profiled in Section 12.04 (JWS/JWT per RFC 7515/RFC 7519, validated per RFC 8725). *Realizes SEC-OBJ-3.*

OFG-2 — Defined grace policy A grace policy **MUST** define, at minimum: the renewal window (how early renewal begins), the offline grace duration (how long operation may continue after expiry without renewal), the action on grace exhaustion, the alert thresholds, and the renewal retry/backoff behavior. *Realizes SEC-OBJ-3; mitigates THR-7.*

OFG-3 — Governed policy ownership The grace policy **MUST** be defined by the vendor as a product default and **MAY** be overridden by the tenant only where the vendor permits it. The control plane **MUST** enforce the resulting effective policy, and all policy definitions and overrides **MUST** be audited (see AUD-4). *Mitigates THR-7.*

OFG-4 — Policy conveyed to the enforcement point The effective policy **MUST** be conveyed to the workspace plane (for example as a snapshot carried in the token and in the issue/renew response) so that it can be enforced locally even while disconnected. *Realizes SEC-OBJ-3.*

OFG-5 — Renewal retries with bounded backoff While disconnected, the license agent **MUST** continue attempting renewal in the background using

bounded exponential backoff with jitter — a well-established retry pattern — so that connectivity is used promptly when it returns without overwhelming the control plane on recovery. *Mitigates THR-11.*

OFG-6 — Defined exhaustion actions When the grace window is exhausted, the configured action **MUST** be applied. The model defines three standard actions:

- **DRAIN** — finish current work and refuse new sessions or jobs;
- **DEGRADE** — continue with a reduced capability profile (for example fewer channels or disabled premium features);
- **STOP** — halt licensed operation.

The action is selected by policy according to the use case and the vendor’s protection stance. *Realizes SEC-OBJ-3; mitigates THR-5, THR-7.*

OFG-7 — Advance alerting Operators **MUST** be alerted as the grace window counts down, at the thresholds defined in the policy, so that a human can intervene before production is affected. *Realizes SEC-OBJ-3.*

OFG-8 — Bounded offline duration Offline operation **MUST** be bounded. The combination of short token lifetime and a finite offline grace duration **MUST** limit how long operation can continue without contact, so that the value of severing connectivity or tampering with the local clock is limited. *Mitigates THR-6, THR-7.*

OFG-9 — Revocation is not evadable beyond the bound Offline and grace behavior **MUST NOT** be usable to evade revocation beyond the bounded window. A revocation that could not be delivered while disconnected **MUST** take effect no later than the next renewal attempt. *Mitigates THR-6. (Consistent with SEC-18, SEC-19.)*

OFG-10 — Idempotent reconciliation of offline usage Usage accrued while offline **MUST** be buffered locally and reconciled idempotently when connectivity returns, so that no usage is lost and none is double counted. *Mitigates THR-5; supports AUD-5.*

OFG-11 — Air-gapped operation Air-gapped operation **MAY** be supported using time-boxed offline licenses delivered through transferable media. Where it is supported, the reduced functionality and the explicit trust trade-offs (notably the inability to deliver timely revocation) **MUST** be stated, and the offline period **MUST** remain bounded (OFG-8). *Mitigates THR-6, THR-7.*

Trust and Risk Trade-off

Offline and grace behavior is the explicit resolution of the protection-versus-continuity tension named in Chapter 05 (T8 versus T9):

- Longer grace and longer offline bounds favour continuity but widen the window in which a revoked or over-using workload can operate.
- Shorter bounds favour protection but increase sensitivity to connectivity problems.

The model does not prescribe a single setting. Instead it requires that the trade-off be **explicit, governed, bounded, and observable**: chosen by policy (OFG-2, OFG-3), conveyed to the enforcement point (OFG-4), bounded in time (OFG-8, OFG-9), and alerted on (OFG-7). The appropriate point on the spectrum depends on the use case class (Chapter 05, Section 5.3) and the operational profile (Chapter 13) — a continuous 24/7 production typically favours continuity, while a strict-protection product may favour STOP with a short bound.

References

- RFC 7515 (JWS); RFC 7519 (JWT); RFC 8725 (JWT BCP) — locally validatable signed tokens — OFG-1 (profiled in Section 12.04).
- RFC 7009 — OAuth 2.0 Token Revocation (conceptual analogue) — OFG-9.

The bounded-backoff-with-jitter retry behavior in OFG-5 follows widely adopted reliability practice rather than a single specification.

Relationship to Other Chapters

- The security controls (Section 12.04) provide the short-lived tokens (SEC-6), bounded exposure (SEC-19), and revocation (SEC-18) that make bounded offline behavior safe.
- The workflow definitions in Chapter 05 (notably the grace/offline continuity and offline acquisition workflows) describe the operational sequences this section secures.
- The operational and deployment profiles (Chapter 13) determine which exhaustion action and bounds are appropriate per environment.
- The canonical data model defines the policy structure; this section defines its required semantics, not its schema.

This section is normative. It defines the required *semantics* of grace and offline behavior (windows, actions, bounds, alerting, reconciliation) while leaving the concrete policy schema and default values to the canonical data model, the per-profile guidance in Chapter 13, and the informative annexes. The action

names DRAIN, DEGRADE, and STOP are treated as a controlled vocabulary for this behavior.

13. Operational and Deployment Profiles

Purpose

This chapter defines the operational and deployment profiles used by the Dynamic Software Licensing best practices.

Its purpose is to describe the practical runtime environments and operating patterns that the best practices supports.

Role Within the Best Practices

The DSL best practices are written for real production environments, not only abstract reference architectures.

This chapter provides the structure for expressing how the licensing model is understood across different operating patterns, such as:

- continuous baseline environments,
- event-driven production,
- temporary or test environments,
- hybrid or DR-focused environments,
- value-oriented or AI-enabled scenarios.

Scope of This Chapter

This chapter defines recurring operational profiles that influence:

- continuity expectations,
- deployment patterns,
- metering behavior,
- availability assumptions,
- operational controls,
- migration priorities.

It connects the DSL model to practical deployment realities.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. 24x7 Broadcast Profile Defines the profile for continuous production environments.

02. Event Production Profile Defines the profile for event-driven or burst-oriented production scenarios.

03. Test and Validation Profile Defines the profile for non-production or pre-production environments.

04. AI Outcome Services Profile Defines the profile for outcome-oriented or AI-related service models.

05. Hybrid and DR Profile Defines the profile for hybrid, failover, or disaster recovery-oriented deployment models.

Relationship to Other Chapters

This chapter is closely related to:

- stakeholders and use cases,
- conformance and profiles,
- normative requirements,
- security and trust,
- migration and transition.

It provides the operational framing for interpreting the best practices in real environments.

Chapter Positioning

This chapter provides the operational interpretation layer for the published baseline.

It is reviewed as the link between abstract architecture and real deployment modes, especially where continuity, resilience, metering, and commercial behavior differ by profile.

13.01. 24x7 Broadcast Profile

Purpose

This chapter defines the operational and deployment profile for continuous 24x7 broadcast and always-on media service environments within the Dynamic Software Licensing (DSL) best practices.

Its purpose is to describe the licensing expectations of long-running production systems where service continuity, predictability, and operational stability are more important than short-term elasticity.

Background

Many broadcast and media environments run as baseline, continuously available services rather than as short-lived event workloads. Typical examples include channel playout, master control, core contribution or distribution services, and operationally stable media processing chains.

In these environments, licensing must support long-lived workloads, controlled change windows, and a strong expectation that licensing should not become the trigger for service interruption. Operators usually prefer predictable continuity and early warning over frequent activation or highly granular consumption shifts.

These environments may still span on-premises, cloud, or hybrid infrastructure, but the licensing posture is shaped by continuity first.

Profile Characteristics

- Workloads are long-running and comparatively stable.
- Renewal and continuity behavior must be predictable and low-noise.
- Short token lifetimes remain acceptable only when paired with reliable background renewal and bounded grace behavior.
- Failover and maintenance scenarios matter more than rapid burst scale-out.
- Operators require clear visibility into expiry horizon, grace state, and capacity headroom.

Licensing Interpretation

Within this profile, the DSL model should be understood as follows:

- Leases are typically granted for workloads that remain active for extended periods.
- Renewal is a normal continuous background activity, not an exceptional event.
- Grace policy should favor controlled continuity, typically with alerting and drain or degrade behavior before hard stop where justified.
- License mobility must support failover, maintenance replacement, and controlled cutover without manual vendor intervention.
- Usage metering may still apply, but operational continuity requirements generally constrain how aggressively protection windows can be shortened.

Operational Expectations

The following expectations are typical for this profile:

- Advance warning before any licensing condition could affect service.
- Strong observability of renewal health and current license state.
- Clear self-service inspection and controlled administrative action.
- Consistent handling of maintenance windows, node replacement, and planned failover.
- Bounded but continuity-oriented offline and grace parameters.

Typical Deployment Patterns

- On-premises broadcast plants with redundant application tiers.
- Private or public cloud media backends running continuously.
- Hybrid deployments where core services remain active across more than one environment.
- Managed services that provide sustained production capacity to one or more tenants.

Relationship to Other Chapters

- Chapter 05 defines the continuous-operations use case class whose pressures this profile expresses operationally.
- Chapter 09 defines the operational, policy, and integration requirements that this profile interprets in practice.
- Chapter 12 defines the bounded continuity and trust constraints that apply even in always-on environments.

Chapter Positioning

This chapter provides the operational interpretation layer for stable, long-running production workloads in the DSL best practices.

It is the profile that emphasizes continuity, observability, and controlled failover over short-lived elasticity.

13.02. Event Production Profile

Purpose

This chapter defines the operational and deployment profile for Event Productions used by the Dynamic Software Licensing best practices.

Its purpose is to describe the practical runtime environments and operating patterns that the best practices supports.

Background

This section describes a reference implementation of the DSL for live event production using software media functions deployed as containerized services in a Kubernetes cluster environment, hosted either on-premises or in a public cloud.

No single licensing model applies across all event scenarios. Requirements vary and continue to evolve as the industry transitions from hardware appliance-based solutions to software-defined infrastructure. Licensing terms may cover a seasonal

production, a major international event spanning several weeks, or shorter, more granular periods of use.

Demand is increasing for consumption-based licensing models that enable costs to be forecast and aligned directly with production events and budgets. Accordingly, hourly and minute-based usage models are gaining interest.

Live production spans multiple tiers, from Tier 1 and Tier 1+ international sporting events to Tier 2 and Tier 3 lower leagues and minority sports. Production budgets, creative requirements, audience expectations, and availability requirements vary by tier. Continuity requirements in the event of system failure also vary accordingly.

Higher-tier productions typically require redundant services, which must also be licensed. Lower-tier productions may not justify a full 1+1 redundancy model and may instead require lease transfer or reissue from a failed instance to a replacement or standby instance.

Live Production Workload Example

The diagram below illustrates a simplified live production workload intended to highlight the key concepts.

Event Production

Figure 1: Event Production

The workload represents an event-based production deployment. Resources are ephemeral and are deployed only for the duration of the production, including pre-event setup and post-event tear-down.

Media functions used within the workload must be licensed appropriately. The licensing scheme must support ephemeral deployment. Each media function must obtain the required lease and associated license dimensions before operation and must release the lease when the workload terminates. The model must also support schedule overruns, such as extra time in sports events, through lease extension or renewal.

In the scenario described, the workload supports a small studio operation comprising:

- **Studio Cameras** deployed on the studio floor. These may require feature licenses, such as HDR processing. Cameras may be network-connected or air-gapped.
- **Input Processing** functions used to normalize and convert studio camera output. These may require frame synchronization, facility time alignment, or other licensed advanced features.
- A **Switcher** providing vision mixing for compositing video elements. The switcher may require licenses for supported input types, input counts, or advanced processing functions such as keyers.

- An **Audio Mixer** providing program audio mixing from studio cameras and external microphones. The audio mixer may require licenses for specific filters and effects.
- **Output Processing** used to combine switcher and audio mixer outputs for onward distribution. This may require licenses for signal encoding.

How DSL addresses Event Production

The industry is transitioning toward software-defined infrastructure and new licensing models. The DSL addresses event-based licensing requirements and supports the following scenarios:

- Multivendor ecosystems and best-of-breed deployments, with vendor-specific license models for required features and dimensions
- Platform-neutral licensing for both public cloud and on-premises deployments
- Leasing of licenses and associated dimensions for a defined validity period
- Usage metering to support consumption-based models
- Lease extension for overrunning events and schedule changes
- High-availability deployments, including licensing of redundant instances and lease transfer from failed instances to replacement instances

Adoption of the DSL may not be uniform across the vendor community in the early stages. However, the best practices defined in this repository are intended to provide a vendor-neutral solution developed by the industry for the needs of end users, system integrators, and ISVs.

Chapter Positioning

This chapter provides the operational interpretation layer for event-driven deployments within the DSL best practices.

It connects the abstract lifecycle, entitlement, metering, and continuity concepts from the earlier chapters to a concrete deployment profile with short-lived workloads, schedule-driven runtime, and tier-dependent availability expectations.

13.03. Test and Validation Profile

Purpose

This chapter defines the operational and deployment profile for test, validation, staging, and pre-production environments within the Dynamic Software Licensing (DSL) best practices.

Its purpose is to describe how the DSL model supports short-lived, experimental, and non-production workloads without forcing production-style licensing commitments.

Background

Test and validation environments are created to prove workflow design, validate integration, exercise failure scenarios, and support engineering change before production rollout. These environments are often intermittent, highly automated, and more numerous than production environments.

Because their lifetime and timing are variable, licensing in this profile must support non-interactive activation, short-duration use, and clear separation from production rights. The main challenge is enabling experimentation and delivery velocity without blurring the governance boundaries of production entitlements.

Profile Characteristics

- Environments are short-lived, bursty, and frequently created or destroyed.
- Automation and headless operation are first-order requirements.
- Fine-grained time modalities or scoped sandbox rights are common.
- Test usage must remain distinguishable from production usage.
- Operators and engineers need rapid self-service rather than ticket-driven workflows.

Licensing Interpretation

Within this profile, the DSL model should be understood as follows:

- License issue and revoke behavior should integrate cleanly into CI/CD, staging, and infrastructure automation.
- Entitlement scope should be able to distinguish test, training, validation, and production use.
- Short-duration leases and granular modalities are often preferred so that temporary environments do not consume long-lived production rights unnecessarily.
- Inspection must make clear which features, dimensions, and restrictions apply in the test context.
- Usage reporting should support cost attribution and governance without turning testing into a manual administrative exercise.

Operational Expectations

The following expectations are typical for this profile:

- Non-interactive issue and renew behavior for unattended pipelines.
- Fast environment bring-up and tear-down, including licensing.
- Clear separation between sandbox or demo rights and production rights.
- Visibility into current allocation to avoid hidden buildup of stale test leases.
- Simple inspection and controlled release of short-lived grants.

Typical Deployment Patterns

- CI/CD-triggered ephemeral environments.
- Staging clusters used for end-to-end workflow validation.
- Engineering sandboxes and demo environments.
- Periodic batch or quality-control jobs that do not justify permanent licensing commitments.

Relationship to Other Chapters

- Chapter 05 defines the test and validation use case class whose needs this profile expresses.
- Chapter 09 defines the integration and operational requirements that are especially important in this profile.
- Chapter 14 later describes how these environments can support phased migration and adoption.

Chapter Positioning

This chapter provides the operational interpretation layer for non-production and pre- production DSL usage.

It connects the DSL model to engineering workflows that prioritize speed, automation, and safe separation from production entitlements.

13.04. AI Outcome Services Profile

Purpose

This chapter defines the operational and deployment profile for AI-enabled, process-oriented, and outcome-based media services within the Dynamic Software Licensing (DSL) best practices.

Its purpose is to describe environments where the licensed value is more naturally expressed in results, processed units, or service outcomes than in static instance time alone.

Background

AI-oriented and other value-added services often scale according to work performed rather than according to a fixed always-on footprint. Examples include automated content analysis, compliance processing, subtitling, highlight generation, metadata enrichment, and other software-defined workflows where value is produced in discrete or measurable outcomes.

In these environments, licensing must support expressive dimensions, clear usage attribution, and transparent connection between technical consumption and

commercially meaningful units. The profile also needs to accommodate variable workload intensity and queue-driven or batch- oriented execution patterns.

Profile Characteristics

- Workload intensity can vary widely over time.
- Metering often relies on process units, outcomes, or normalized work metrics.
- Time-based leasing may still exist, but it is often insufficient on its own to describe value.
- Forecasting and transparency depend on clear linkage between reported dimensions and delivered outcomes.
- Integration with analytics, billing, or external reporting systems is typically important.

Licensing Interpretation

Within this profile, the DSL model should be understood as follows:

- Entitlements and leases must support dimensions that describe work performed, not only elapsed runtime.
- Usage records should be able to distinguish the unit of value being consumed, for example generated clips, processed minutes, or normalized process units.
- Modality and metering semantics must remain explicit so that customers and integrators can understand how reported usage maps to granted rights.
- Local validation and bounded continuity remain necessary, but operator transparency around cost and usage meaning is especially important.
- External integration with analytics, billing, or marketplace-style metering is often part of the deployment model.

Operational Expectations

The following expectations are typical for this profile:

- Rich, clearly documented usage dimensions.
- Reliable idempotent usage reporting and reconciliation.
- Visibility into outcome-oriented consumption and forecast drivers.
- Support for variable demand patterns without redefining core lifecycle semantics.
- Consistent treatment of AI and non-AI services under the same canonical lifecycle model.

Typical Deployment Patterns

- Queue-driven processing services.
- Elastic media analytics or inference workloads.
- Outcome-oriented SaaS services layered on top of media platforms.
- Hybrid execution patterns where burst processing supplements stable baseline capacity.

Relationship to Other Chapters

- Chapter 05 defines the AI-powered and outcome-based use case class whose pressures this profile expresses operationally.
- Chapter 09 defines the modality, metering, and integration requirements that are especially important in this profile.
- Chapter 11 defines the usage event and token or lease data structures that carry the relevant dimensions.

Chapter Positioning

This chapter provides the operational interpretation layer for outcome-oriented and AI-enabled DSL deployments.

It is the profile where expressive metering, transparent attribution, and modality clarity become especially important.

13.05. Hybrid and DR Profile

Purpose

This chapter defines the operational and deployment profile for hybrid, failover, and disaster-recovery-oriented environments within the Dynamic Software Licensing (DSL) best practices.

Its purpose is to describe how the DSL model supports portability, controlled cutover, and resilience across more than one environment or runtime boundary.

Background

Many media systems are not confined to a single deployment environment. They span on-premises and cloud infrastructure, maintain standby capacity in a second location, or rely on a disaster recovery design that must take over quickly under failure conditions.

In these scenarios, licensing becomes part of the resilience architecture. The core challenge is to support movement, failover, and readiness without reverting to static host binding or manual vendor intervention during time-critical incidents.

Profile Characteristics

- More than one environment or location participates in the operational design.
- Standby, failover, or alternate execution contexts may need rapid access to licensed capability.
- Identity-based binding and controlled reissue are critical.
- Portability and auditability matter as much as normal steady-state operation.
- Hybrid trust and connectivity assumptions may differ across participating environments.

Licensing Interpretation

Within this profile, the DSL model should be understood as follows:

- Binding must support movement between primary and secondary environments through controlled reissue or equivalent lifecycle behavior.
- The model should allow readiness and resilience without requiring continuous duplicate consumption beyond what entitlement and policy explicitly allow.
- Operational visibility must make it clear which workload currently holds the active grant and what failover or cutover state applies.
- Continuity behavior must remain bounded even when connectivity or reachability differs across environments.
- Hybrid deployment should not change the canonical lifecycle semantics, only the environment in which they execute.

Operational Expectations

The following expectations are typical for this profile:

- Rapid reissue or cutover during failure or planned migration.
- Clear audit trail for transfer and activation events.
- Operator visibility into primary, standby, and replacement states.
- Predictable handling of network partitions and partial control-plane reachability.
- Support for staged migration where both legacy and DSL-aligned environments may coexist.

Typical Deployment Patterns

- Primary on-premises production with cloud-based failover.
- Dual-site broadcast operations with active and standby environments.

- Blue or green cutover between two deployment stacks.
- Hybrid media platforms where some functions remain local and others burst to cloud.

Relationship to Other Chapters

- Chapter 05 defines the resilience and continuity pressures that this profile operationalizes.
- Chapter 09 defines the identity, policy, security, and operational requirements that govern cutover and bounded continuity.
- Chapter 14 defines the migration sequencing that often leads into this profile.

Chapter Positioning

This chapter provides the operational interpretation layer for DSL deployments that span more than one environment or that must preserve continuity through controlled failover.

It is the profile where identity-based mobility, auditability, and resilience come together most directly.

14. Migration and Transition

Purpose

This chapter defines the migration and transition structure of the Dynamic Software Licensing best practices.

Its purpose is to describe how organizations can move from legacy licensing approaches toward the DSL target model in a staged and practical way.

Role Within the Best Practices

The DSL initiative is not only about defining a future target model. It must also support the path from current reality to that model.

The existing best practices material already describes migration as a phased transformation rather than a one-step replacement, including analysis, pilot operation, technical transformation, business logic expansion, decommissioning, and scaling.

This chapter provides the structural home for that transition logic.

Scope of This Chapter

This chapter defines:

- migration phases,
- transition logic,
- adoption sequencing,
- relationship between technical and business transformation,
- practical considerations for moving away from legacy models.

It provides a structured migration view without becoming a vendor-specific project plan.

Structure of This Chapter

This chapter is organized into the following subchapters:

01. Assessment and Preparation Defines the initial analysis and preparation phase.

02. Pilot and Parallel Operation Defines the initial transition phase in which legacy and new approaches may coexist.

03. Technical Transformation Defines the phase in which the foundational technical building blocks of the new model are established.

04. Business Logic Expansion Defines the phase in which the commercial and business logic of the model becomes more capable.

05. Full Migration and Decommissioning Defines the phase in which legacy mechanisms are retired.

06. Scaling and Industry Establishment Defines the phase in which the new model becomes broader, repeatable, and industry-established.

Relationship to Other Chapters

This chapter is closely related to:

- the introduction,
- stakeholders and use cases,
- conformance and profiles,
- normative requirements,
- operational profiles,
- governance and future work.

It provides the transition logic between the current industry state and the DSL target model.

Chapter Positioning

This chapter provides the transformation roadmap perspective for the published baseline.

Its role in review is to test whether the DSL target model is not only desirable, but also adoptable through realistic staged transition.

14.01. Assessment and Preparation

Purpose

This section defines the first migration phase of the DSL transition path: understanding the current state and preparing the organization for controlled adoption.

Role of This Phase

Before any technical migration begins, organizations need a clear picture of their existing licensing estate, operational dependencies, entitlement models, and deployment patterns. This phase establishes that baseline.

Its purpose is not to redesign the future state in detail, but to make later pilot and transformation work realistic, scoped, and traceable.

Typical Activities

- Inventory current licensed products, license servers, dongles, portals, and manual workflows.
- Identify which workloads are continuous, event-driven, test-oriented, hybrid, or outcome-based.
- Map current entitlement boundaries, modality assumptions, and usage-reporting dependencies.
- Identify high-friction areas such as manual activation, hardware binding, poor observability, or weak failover behavior.
- Determine which stakeholders must participate in migration decisions, including operations, engineering, finance, security, procurement, and vendors.

Key Questions

- Which current licensing mechanisms create the highest operational risk?
- Which products or workflows would benefit most from API-first or identity-based licensing?
- Where are current renewal, failover, or reporting practices too manual?
- Which environments are suitable for early low-risk adoption?

Outputs of This Phase

The phase should produce:

- a current-state inventory,
- a prioritized set of migration candidates,
- an initial target-state hypothesis,
- identified blockers, dependencies, and cross-team ownership.

Relationship to Later Phases

This phase feeds directly into pilot selection and transition planning. Poor preparation here typically results in over-scoped pilots, unclear ownership, or false assumptions about legacy dependency.

Section Positioning

This section represents the readiness and discovery phase of DSL adoption. It provides the fact base on which all later migration decisions depend.

14.02. Pilot and Parallel Operation

Purpose

This section defines the phase in which DSL-aligned licensing is first exercised in controlled real-world conditions while legacy approaches still remain available.

Role of This Phase

The goal of this phase is to validate the target model without forcing immediate cutover of all workloads. It allows organizations to test lifecycle behavior, policy handling, identity- based binding, observability, and operator workflows in a limited scope.

Parallel operation is often necessary because legacy and DSL-aligned mechanisms will coexist for some time across different products, environments, or vendors.

Typical Activities

- Select a low-risk but meaningful candidate workload or service.
- Run the DSL lifecycle in a bounded environment such as staging, test, or a non-critical production slice.
- Validate issue, renew, revoke, reissue, usage reporting, and inspection workflows.
- Exercise failure paths such as temporary control-plane outage, renewal retry, grace, and recovery.
- Confirm that operations, finance, and engineering teams can understand the new visibility and governance model.

Pilot Selection Characteristics

Good pilot candidates often have these properties:

- limited blast radius,
- clear ownership,
- measurable lifecycle and usage behavior,
- realistic automation opportunities,
- enough business relevance to produce actionable feedback.

Expected Outcomes

This phase should confirm:

- that the DSL model works for at least one meaningful workload,
- that the new lifecycle semantics are operationally understandable,
- that policy and continuity behavior are acceptable,
- that legacy fallback is no longer needed for the pilot slice.

Relationship to Later Phases

Pilot experience informs the technical transformation phase by revealing which integration, data-model, operational, or governance capabilities need to mature before broader rollout.

Section Positioning

This section represents the first practical adoption phase of DSL. It proves the model in a controlled scope while reducing the risk of broad production rollout.

14.03. Technical Transformation**Purpose**

This section defines the phase in which the foundational technical building blocks of the DSL model are established beyond the initial pilot.

Role of This Phase

Once pilot viability is demonstrated, organizations need to move from isolated proof to repeatable technical capability. This phase focuses on the structural changes required to make DSL adoption sustainable across environments and products.

Typical Activities

- Introduce or stabilize the control-plane interfaces used for issue, renew, reissue, revoke, usage reporting, and inspection.
- Establish identity-based binding mechanisms and trusted identity evidence flows.
- Adopt signed, locally validatable runtime artifacts and key-discovery behavior.
- Integrate renewal, observability, and usage reporting into operational tooling.
- Align deployment automation and licensing workflows so that licensing becomes part of normal provisioning and recovery behavior.

Transformation Themes

This phase typically includes movement toward:

- API-first operations instead of console-only or ticket-driven workflows,
- identity-based binding instead of hardware-bound or host-bound licensing,
- explicit policy and grace handling instead of opaque client logic,

- observability and inspectability instead of hidden runtime state,
- automation-ready integration rather than manual licensing side paths.

Technical Risks to Manage

- coexistence with legacy identity or activation approaches,
- inconsistent lifecycle semantics across vendors,
- insufficient operator tooling for inspection and alerting,
- weak integration of usage data into financial or operational systems,
- incomplete trust assumptions around keys, clocks, and identity sources.

Expected Outcomes

By the end of this phase, the organization should have a reusable technical DSL foundation that can support more than one workload and more than one deployment profile without rethinking the core model each time.

Relationship to Later Phases

Technical transformation enables the later expansion of modality, cost, and business logic. It also reduces dependency on legacy mechanisms, making decommissioning feasible.

Section Positioning

This section represents the point at which DSL stops being a pilot pattern and becomes a real technical operating model.

14.04. Business Logic Expansion

Purpose

This section defines the phase in which the commercial and business-logic capabilities of the DSL model expand beyond simple technical replacement of legacy licensing.

Role of This Phase

Early DSL adoption may focus primarily on operational improvement. Over time, however, organizations and vendors typically want to use the model to support richer entitlement segmentation, more flexible modalities, and more transparent usage-to-cost relationships.

This phase is where the model becomes commercially more capable without abandoning the canonical lifecycle and trust semantics.

Typical Activities

- Expand from a single dominant modality to multiple modality types where justified.
- Introduce clearer differentiation between production, test, event, or partner usage scope.
- Improve usage attribution and forecasting using richer dimensions and work correlation.
- Support more explicit self-service or delegated control for customers and integrators.
- Integrate licensing data into finance, procurement, analytics, or marketplace-style systems.

Transition Considerations

This phase should not be treated as a purely commercial exercise. Business-logic growth must stay aligned with:

- the canonical data model,
- the operational profiles,
- security and tenant-isolation constraints,
- the requirement that lifecycle semantics remain stable across modalities.

Expected Outcomes

By the end of this phase, the DSL model should support a broader set of real business cases, including more granular event licensing, richer usage-driven services, and clearer governance of multi-product or multi-scope entitlement landscapes.

Relationship to Later Phases

Business-logic expansion often exposes the final legacy constraints that still block full cutover. It therefore provides the basis for eventual decommissioning and broader scaling.

Section Positioning

This section represents the phase where DSL becomes not only technically viable but also commercially and operationally expressive enough for broader adoption.

14.05. Full Migration and Decommissioning**Purpose**

This section defines the phase in which legacy licensing mechanisms are retired and the DSL model becomes the primary operational baseline.

Role of This Phase

The purpose of this phase is to complete the transition from coexistence to controlled primary use of DSL-aligned licensing. It is where organizations deliberately remove dependency on legacy servers, dongles, host-bound activation paths, or opaque manual processes.

Typical Activities

- Identify which legacy components remain in use and why.
- Confirm that equivalent DSL-aligned lifecycle and operational capabilities are in place.
- Cut over the remaining workloads, products, or environments.
- Retire obsolete operational runbooks and update governance, support, and audit processes.
- Decommission or isolate legacy infrastructure that no longer has an operational role.

Readiness Indicators

Full migration is usually credible only when:

- pilot and broader technical rollout have proven stable,
- lifecycle operations are operationally well understood,
- observability and audit expectations are satisfied,
- critical failover and continuity scenarios have been exercised,
- remaining legacy use is exceptional rather than baseline.

Decommissioning Concerns

Decommissioning should account for:

- retained legal or contractual dependency on older licensing forms,
- archival or audit obligations,
- interoperability with vendors that have not yet completed their own transition,
- rollback boundaries if a remaining critical issue is discovered.

Expected Outcomes

By the end of this phase, DSL-aligned licensing is the normal operating model and legacy licensing paths have been reduced to explicitly managed exceptions or removed entirely.

Section Positioning

This section represents the point at which DSL becomes the accepted baseline rather than a transition program.

14.06. Scaling and Industry Establishment

Purpose

This section defines the phase in which DSL adoption moves beyond individual migrations and becomes repeatable, scalable, and more broadly established across teams, products, and industry participants.

Role of This Phase

The final transition phase is not only about internal rollout. It is also about making the DSL approach understandable, repeatable, and interoperable across a broader vendor and customer ecosystem.

Typical Activities

- Reuse the migration approach across additional products, services, and deployment profiles.
- Standardize internal implementation patterns, operational playbooks, and integration models.
- Align partner, reseller, integrator, and customer expectations around common lifecycle semantics and terminology.
- Improve supporting annexes, schemas, examples, and governance artifacts as adoption grows.
- Feed operational and commercial lessons back into future best-practices revisions.

Scaling Characteristics

At this stage, the DSL model should support:

- repeated adoption with lower marginal effort,
- more predictable onboarding of new products or participants,
- stronger cross-vendor interoperability,
- clearer shared vocabulary and expectation setting,
- more stable review and publication cycles for the best practices themselves.

Industry Establishment Considerations

Broader establishment requires more than technical reuse. It also depends on:

- controlled terminology,
- stable lifecycle semantics,
- credible migration examples,
- governance that can handle future extensions without fragmenting the model,

- enough implementation evidence to make the approach trustworthy to new adopters.

Expected Outcomes

By the end of this phase, DSL is no longer treated as a one-off migration exercise. It has become a repeatable operational and industry pattern that can continue to evolve through versioned best-practices updates.

Section Positioning

This section represents the transition from project-level adoption to ecosystem-level normalization.

15. Governance and Future Work

Purpose

This chapter defines the governance-related and forward-looking aspects of the Dynamic Software Licensing best practices.

Its purpose is to describe how the best practices evolves over time and which future development areas remain dependent on broader industry collaboration.

Role Within the Best Practices

The DSL best practices are more than a static one-off document.

As an IAMT initiative, it evolves through collaboration, refinement, and clarification of terminology and requirements toward a more mature industry reference point.

This chapter provides the structural home for those topics.

Scope of This Chapter

This chapter captures:

- governance-related considerations for maintaining the best practices,
- future development themes that are already visible from the current draft,
- areas where additional industry alignment is expected,
- next-step priorities for post-review refinement and publication.

It does not duplicate repository governance files, but explains how governance and evolution relate to the best practices itself.

Governance of the Best Practices

The DSL best practices should be governed as a controlled, reviewable industry document. That governance should ensure that future revisions remain coherent across terminology, architecture, requirements, annexes, and schemas.

The governance model for the document should follow these principles:

- editorial decisions remain explicit and traceable,
- terminology changes are treated as controlled vocabulary changes,
- normative changes are reviewed for cross-chapter impact before acceptance,
- annex growth remains aligned with the main document rather than becoming a parallel specification,
- publication updates are versioned and documented.

This chapter complements the operational governance files in the repository root by describing the governance expectations of the best-practices text itself.

Post-Review Priorities

The current published baseline already establishes the main document structure and a substantial content baseline. The next priorities after group review should be:

- resolve cross-chapter terminology adjustments identified during review,
- tighten requirement wording where informative text must become more disciplined,
- confirm traceability between use cases, requirement themes, workflows, and normative requirements,
- align conformance, API, data model, and security chapters on shared interpretation points,
- verify that annex content remains supportive and does not conflict with the main text.

These priorities convert review feedback into a stable publication baseline rather than reopening the document scope.

Expected Areas of Future Development

Several themes are likely to require further development after the review cycle:

Conformance and Validation Evolution The current best practices establishes conformance structure and profiles, but future iterations may expand practical validation approaches, implementation evidence, and profile-oriented assessment guidance.

Interoperability and Ecosystem Alignment As more stakeholders review and apply the DSL model, additional interoperability questions are likely to emerge around API conventions, identity evidence, token handling, policy expression, and operational handoff between vendors and operators.

Migration and Adoption Guidance Migration will remain an area of ongoing refinement. Later revisions may provide more explicit transition patterns, readiness indicators, and practical adoption sequencing for different organizational starting points.

Annex and Schema Expansion Future revisions may expand supporting annexes and machine-readable artifacts where that improves consistency, testability, or implementation clarity. Such additions should remain subordinate to the main document and should only be introduced where they reduce ambiguity.

Relationship to Other Chapters

This chapter is closely related to:

- status of the document,

- conventions and normative language,
- conformance and profiles,
- migration and transition,
- repository governance material in the root of the repository.

It provides the long-term orientation layer of the best practices.

Review and Publication Path

The current document state supports formal group review. That review is used to:

- confirm that the document structure is complete,
- identify wording improvements and consistency fixes,
- surface remaining editorial ambiguities,
- validate whether the maturity claim should advance toward publication readiness.

Following review, the publication path should proceed in a controlled sequence:

- incorporate accepted comments,
- confirm status updates at chapter and repository level,
- publish a release candidate baseline for final editorial approval,
- publish the agreed best-practices release with versioned change documentation.

Maintenance Expectations

Once published, the best practices remain maintainable through versioned updates rather than ad hoc edits. Proposed changes are evaluated according to their impact on:

- terminology stability,
- normative interpretation,
- interoperability expectations,
- annex and schema consistency,
- migration guidance and operational applicability.

This allows the DSL initiative to evolve responsibly while preserving a stable baseline for industry adoption.

Chapter Positioning

This chapter closes the main body of the DSL best practices by linking the current review baseline to its future maintenance and publication lifecycle. It makes clear that the document becomes stable enough for adoption while remaining governed, versioned, and improvable over time.

Annex A. Conformance Checklist

Purpose

This annex provides a compact checklist structure for assessing whether an implementation can credibly claim alignment with the DSL best practices and with one of the conformance profiles defined in Chapter 08.

How to Use This Annex

This checklist is intended as a review aid, not as a certification instrument. Each statement should be assessed against observable implementation behavior for the relevant conformance target.

Possible assessment states include:

- yes,
- partial,
- no,
- not applicable.

Foundational Checklist

Area	Checklist statement	Primary requirement anchors
General architecture	The implementation exposes documented lifecycle behavior rather than manual-only procedures.	GEN-1, GEN-2, API-1
Identity	The grant binds to verifiable workload identity rather than static hardware attributes.	IDB-1, IDB-2
Lease behavior	Lease lifecycle state, renewal, and expiry behavior are explicit and reviewable.	LPR-1, LPR-2, LPR-3
Policy handling	Runtime policy includes bounded grace and defined exhaustion behavior.	LPR-4, LPR-5, OPS-1
Security	Runtime grants are cryptographically protected and validated locally.	SREQ-1, SREQ-2

Area	Checklist statement	Primary requirement anchors
Observability	Operators can inspect active grants, status, and relevant expiry conditions.	OPS-3, OPS-4

Cloud Native Checklist

Area	Checklist statement	Primary requirement anchors
Automation	Licensing can be integrated into CI/CD, IaC, or orchestrated deployment flows.	INT-1, INT-2
Dynamic identity	Identity evidence supports dynamic or cloud-native workload semantics.	IDB-3, IDB-4
Continuity	Renewal retry and continuity behavior remain predictable under transient outage.	OPS-1, OPS-2, LPR-4
API behavior	Lifecycle operations are consistently exposed through documented APIs.	API-1, API-2, API-3
Metering	Usage reporting and attribution support operational and financial interpretation.	MTR-1, MTR-2, MTR-5
Auditability	Operational events are traceable across issue, renew, reissue, and revoke paths.	SREQ-6, OPS-6

Advanced Interoperable Checklist

Area	Checklist statement	Primary requirement anchors
Semantic consistency	Canonical API, runtime token, and data model remain aligned across implementations.	API-2, GEN-9, INT-7
Portability	Controlled transfer, reissue, and failover behavior work across environments.	GEN-8, LPR-7, INT-5
Tenant isolation	Multi-tenant controls preserve separation, auditability, and least privilege.	SREQ-4, GEN-6
Modality support	Supported license modalities remain explicit and do not alter core lifecycle semantics.	MOD-1, MOD-2, MOD-5
Integration maturity	External automation and partner-facing integrations preserve documented semantics.	INT-3, INT-5, INT-7
Governance readiness	Terminology, schemas, and lifecycle states are stable enough for repeatable cross-team use.	GEN-1, API-7, LPR-1

Annex Positioning

This annex provides a review-oriented conformance aid tied to Chapter 08 profiles and Chapter 09 requirement groups.

Annex B. API Contracts

Purpose

This annex consolidates the canonical API surface referenced throughout Chapter 10 into a compact contract summary for review and cross-reference.

Contract Scope

The canonical API defines lifecycle and reporting operations between the workspace plane and the control plane. The annex does not replace the detailed

explanatory text in Chapter 10, but it provides a stable summary of the baseline contract surface.

Baseline Endpoints

Operation	Endpoint	Method	Purpose
Issue	<code>/v1/leases</code>	POST	Request a new lease for a workload
Renew	<code>/v1/leases/{leaseId\}:renew</code>	POST	Extend an active lease
Reissue	<code>/v1/leases/{leaseId\}:reissue</code>	POST	Initiate transfer to a new workload identity
Reissue finalize	<code>/v1/leases/{leaseId\}:finalizeReissue</code>	POST	Complete a controlled lease transfer
Inspect	<code>/v1/leases/{leaseId\}</code>	GET	Retrieve current lease state without changing lifecycle state
Revoke	<code>/v1/leases/{leaseId\}</code>	DELETE	Terminate an active lease
Usage record	<code>/v1/usage-records</code>	POST	Submit usage or metering events
JWKS	<code>/.well-known/jwks.json</code>	GET	Retrieve public signing keys

Common Request Expectations

Baseline requests should include:

- `app`,
- `tenantId`,
- `identityEvidence`,
- `client`,
- trace context where supported,
- `workId` when idempotent correlation is required.

Common Response Expectations

Successful lifecycle responses should include at least:

- `leaseId`,

- `token`,
- `expiresAt`,
- `policy`.

The exact response shape may vary by operation, but these elements define the core runtime contract.

Error Model

Errors should be expressed using the Problem Details structure defined by RFC 9457 and should preserve enough detail for automation, diagnostics, and tenant-safe troubleshooting.

Contract Notes

- Custom methods are used where lifecycle behavior has domain-specific side effects.
- The push channel for revocation or policy updates complements, but does not replace, the REST API.
- Implementations may add vendor-specific fields only where they do not break canonical semantics.

Machine-Readable OpenAPI Companion

A GitHub-friendly overview of the machine-readable API artifacts is provided at [api/README.md](#).

The underlying OpenAPI companion contract for the baseline REST surface is provided at [api/dsl-api.openapi.yaml](#).

The OpenAPI artifact captures the canonical endpoints, common request and response structures, security expectations, and the RFC 9457 problem-details error envelope in a form that can be used for review, mock generation, client scaffolding, and contract testing.

For the current published baseline, Annex B and Chapter 10 remain the primary textual authorities if a wording ambiguity exists. The OpenAPI file is intended to stay aligned with that baseline contract surface, not to redefine it independently.

Annex Positioning

This annex is the compact normative reference surface for the baseline API operations of the published DSL best practices.

Annex C. Token Claims

Purpose

This annex defines the baseline claim set carried in the signed runtime token used by the DSL model.

Claim Set Overview

The token must carry enough information for local validation and runtime enforcement without requiring continuous reachability to the control plane.

Baseline Claims

Claim	Type	Purpose
iss	string	Identifies the issuing control plane
sub	string	Identifies the lease subject or bound workload
aud	string	Identifies the intended application or service audience
jti	string	Uniquely identifies the token
iat	string	States when the token was issued
nbf	string	States when the token becomes valid
exp	string	States when the token expires
tenantId	string	Carries tenant context
leaseId	string	Correlates the token to the lease record
app	string	Identifies the licensed application
dimensions	array	Carries granted dimensions for runtime enforcement
licenseModality	string	Carries the granted license modality
boundIdentity	object	Summarizes verified binding context
policy	object	Carries the effective runtime policy snapshot

Claim Semantics

- Standard claims and DSL-specific claims are combined in one signed object.
- The token should contain runtime-relevant information only.
- Sensitive commercial or back-office detail that is not needed for enforcement should remain outside the token.
- The claim set should remain stable across token formats that provide equivalent trust semantics.

Relationship to Other Artifacts

- The lease object is the authoritative source for token derivation.
- The policy object contributes the effective runtime behavior controls.
- Key distribution relies on the JWKS endpoint defined in Annex B.

Annex Positioning

This annex is the compact normative reference for the baseline DSL runtime claim set.

Annex D. Lifecycle and State Model

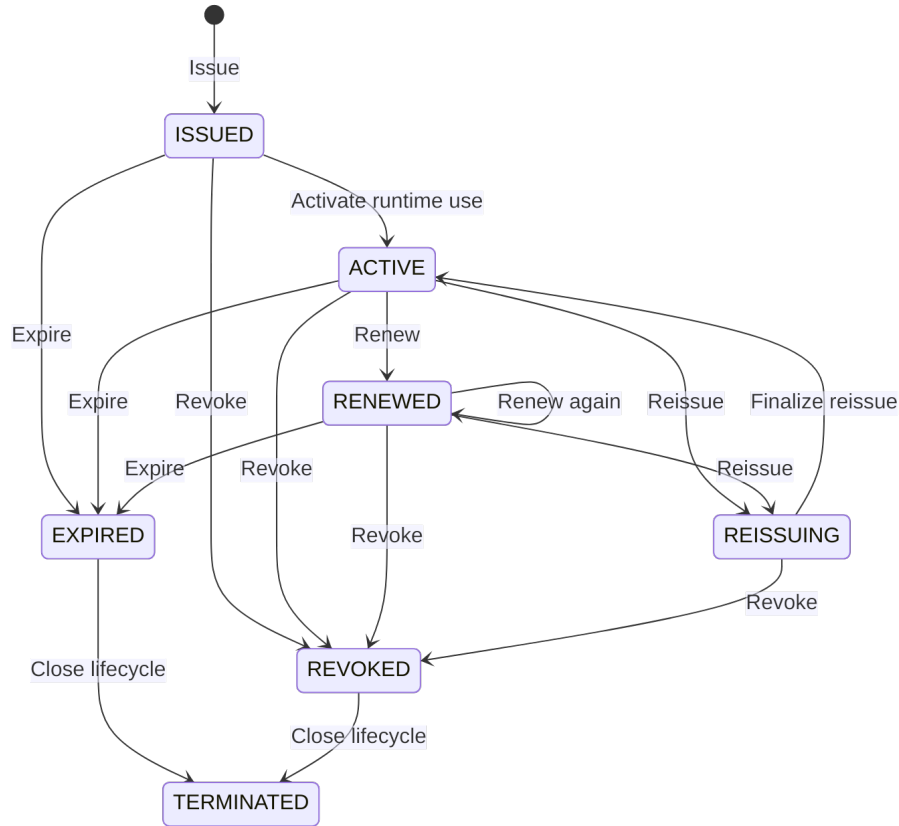
Purpose

This annex consolidates the baseline lease lifecycle states and the main operations that cause state transitions in the DSL model.

Canonical Lease States

State	Meaning
ISSUED	A lease has been created and granted by the control plane
ACTIVE	A lease is currently valid for runtime use
RENEWED	A lease remains active after one or more successful renewals
REISSUING	A lease is in controlled transfer to a new identity or environment
REVOKED	A lease has been invalidated before natural expiry
EXPIRED	The validity window ended without further continuation
TERMINATED	The lease has reached a controlled end of use and is closed

Lifecycle Overview Diagram



Primary Lifecycle Operations

Operation	Typical transition intent
Issue	Create a new lease and enter ISSUED or ACTIVE
Renew	Continue an active lease and extend validity
Reissue	Move a lease into controlled transfer
Finalize reissue	Complete transfer and restore active runtime use
Revoke	End authorization before natural expiry
Expire	Reach the end of validity without successful continuation
Terminate	Close the lease after controlled completion of use

Transition Semantics

- **ISSUED** and **ACTIVE** may be collapsed internally by an implementation, but externally visible semantics should remain understandable.

- Renewal extends validity while preserving lease identity and policy semantics unless a policy update explicitly changes them.
- Reissue is the controlled mechanism for moving a grant between workload identities or environments.
- Revocation takes effect immediately where push is available and otherwise at the next renewal boundary or token expiry.
- Expiry and termination are distinct: expiry reflects elapsed validity, while termination is a deliberate lifecycle close.

Operational Notes

- Grace behavior affects runtime continuity but does not replace lifecycle state semantics.
- Audit trails should preserve the sequence of issue, renew, reissue, revoke, expiry, and termination events.
- The state model should remain stable across profiles even where implementations differ internally.

Annex Positioning

This annex is the compact normative reference for lifecycle-state interpretation in the published DSL best practices.

The canonical form in this annex is the state and transition tables together with the semantic notes above. The inline state diagram complements that normative baseline as a visual summary and does not replace the tabular interpretation.

Annex E. Terminology Reference

Purpose

This annex consolidates selected preferred terms used across the DSL best practices where terminology discipline is especially important for consistent review.

Preferred Terms

Preferred term	Use this term for	Avoid or constrain
Dynamic Software Licensing (DSL)	The initiative and architectural model described in this repository	Informal alternative labels for the same model

Preferred term	Use this term for	Avoid or constrain
Entitlement	The legal and commercial right assigned to a tenant or customer	Using lease as a synonym for entitlement
Lease	The active, time-bounded grant derived from an entitlement	Using license as the runtime object when the lease is meant
License Modality	The granted commercial and operational usage model at runtime	Ambiguous standalone use of modality where the licensing meaning matters
Policy Object	The structured runtime and lifecycle behavior controls associated with a lease	Free-form policy wording when a structured object is meant
Identity-Based Binding	Binding to verifiable workload identity	Hardware-bound terminology when workload identity is intended
Control Plane	The authoritative system managing leases, policy, and usage intake	Mixing it with runtime-side enforcement components
Workspace Plane	The runtime-side enforcement and interaction environment	Referring to it as the control plane

Terminology Guidance

- Use License Modality as the preferred term in normative, API, schema, and data-model contexts.
- Commercial Modality may still be used when discussing higher-level business framing, but it should not replace License Modality where runtime behavior is being specified.
- Modality without qualifier should be avoided where the reader could confuse business framing, metering structure, and runtime behavior.
- The same term should preserve the same meaning across requirement, API, data model, and operational profile chapters.

Annex Positioning

This annex is the compact normative terminology reference for cross-chapter consistency in the published DSL best practices.

Annex J. FAQ

Purpose

This informative annex captures short answers to recurring review questions about the DSL best practices.

Frequently Asked Questions

What is DSL at a high level?

Dynamic Software Licensing (DSL) is an open best-practices framework for managing software entitlements, runtime authorization, policy enforcement, and usage observability in dynamic broadcast and media environments.

How does DSL differ from traditional license management?

Traditional license management often assumes static hosts, manual assignment, long-lived instances, and limited workload mobility. DSL separates durable customer rights from runtime authorization and is designed for automated, policy-driven, and portable operating models.

Is DSL a formal standard?

No. The repository currently publishes best practices with standard-like structure and traceability, but it is not presented as a certification or formal standards program.

Is DSL primarily a reference model or an implementation prescription?

DSL is primarily a shared best-practices reference model. It defines common concepts, lifecycle semantics, interfaces, and expectations that implementations can adopt, rather than mandating one product architecture or one deployment blueprint.

Does DSL use proprietary technology?

No. DSL does not depend on proprietary technology. It combines established architectural and security patterns and aims to align with existing standards and broadly available implementation approaches where that supports interoperability.

Does DSL include security?

Yes. Security is treated as a first-class design concern throughout the model. DSL defines trust, identity evidence, policy control, auditable actions, and bounded runtime authorization behavior. Individual implementations still need to apply their own deployment, monitoring, hardening, and operational security controls.

What security responsibility remains with an implementation?

DSL defines the expected security model and trust boundaries, but implementations remain responsible for secure deployment, credential protection, infrastructure hardening, monitoring, tenant isolation, incident response, and compliance with their own operational environment.

Does DSL require permanent connectivity to a licensing service?

No. The model is designed around signed, locally validatable runtime artifacts with bounded renewal and grace behavior. Continuous connectivity is therefore not required, although the control plane still needs a defined renewal and recovery path.

Does DSL support failover and disaster recovery?

Yes. The model is explicitly designed to support continuity scenarios such as failover, reissue, recovery, and bounded grace behavior without reducing licensing to a single fixed host binding.

Does DSL replace every vendor-specific commercial model?

No. The best practices standardize lifecycle semantics and interoperability patterns, while allowing vendors to retain differentiated commercial offerings.

Must an existing licensing system be fully replaced to use DSL?

No. DSL can be introduced incrementally and can complement existing licensing approaches. Organizations and vendors can adopt the concepts, interfaces, and lifecycle behaviors in stages rather than replacing their entire licensing estate in a single step.

Does DSL only fit cloud-native deployments?

No. The model is intended to remain platform-agnostic and to support cloud, on-premises, hybrid, and resilience-focused environments.

Is DSL only relevant for large broadcasters or large vendors?

No. The model is relevant wherever software licensing needs to behave predictably across changing runtime contexts. That can apply to vendors, broadcasters,

service providers, integrators, and smaller teams that still need portability, auditability, and controlled entitlement handling.

How vendor-neutral is DSL in practice?

DSL is intended to be vendor-neutral at the model and interoperability layer. It defines common concepts and expected behaviors while leaving room for vendor-specific product differentiation, commercial models, and implementation choices.

Why is identity-based binding emphasized so strongly?

Because dynamic workloads, failover, and automation-first operations are difficult to support reliably with static hardware binding.

What is the difference between an entitlement, a lease, and a policy?

An entitlement expresses the durable customer right to use a capability. A lease is the time-bounded runtime grant issued to a specific workload within that right. A policy defines the runtime control behavior around that lease, including renewal timing, grace limits, and enforcement actions.

What is the runtime object in DSL?

The runtime object is the lease and its signed token representation, derived from an entitlement and governed by an effective policy object.

What does DSL intentionally not standardize?

DSL does not attempt to standardize every commercial model, every vendor-specific token payload, every administrative back-office process, or every deployment topology. Its role is to define a common interoperable baseline, not to remove all implementation choice.

How does DSL relate to IAM, OIDC, and PKI?

DSL builds on these kinds of trust mechanisms rather than replacing them. Identity evidence, token signing, verification, and key distribution can be implemented using established IAM, OIDC, PKI, and related security building blocks.

How can I try out DSL?

An example implementation is available at [Dynamic-Software-Licensing/reference-implementation](https://github.com/dynamic-software-licensing/reference-implementation). It can be deployed in an AWS account to explore the control-plane flows and validate the specification behavior in practice.

How do I get started with DSL in practice?

The best starting point is to understand the core vocabulary and lifecycle model, review the main API and data objects, and then test the reference implementation to see how entitlement, lease, policy, identity, and usage flows work together in practice.

Who is this document for?

The best practices are intended for product teams, architects, platform and operations teams, system integrators, and decision-makers who need a shared model for software licensing in modern broadcast and media workflows.

Why does the repository include annexes, schemas, and API artifacts in addition to the main text?

The main chapters define the overall best-practices model, while the annexes provide supporting detail, examples, and interpretive material. The API and schema artifacts provide a machine-readable companion baseline for implementations and technical review.

How can I contribute to DSL?

The best starting point is the IAMT DSL group page at theiamt.org/groups/dynamic-software-licensing, which explains how to join the group and contribute to the ongoing evolution of the best practices.

Annex Positioning

This annex provides concise explanatory support for reviewers and new readers of the best practices and is intentionally positioned as an early informative reference point for recurring questions.

Annex F. AWS Reference Architecture**Purpose**

This informative annex outlines one possible AWS-aligned realization of the platform-neutral DSL reference model.

Scope and Status

This annex is illustrative. It does not introduce normative requirements and it does not imply that AWS-specific services are required for DSL alignment.

The architecture described here is AWS-first rather than multi-cloud. It is intended to show how the DSL control-plane, metering, and trust concepts can be realized with established AWS services and governance patterns.

AWS-Specific Design Priorities

An AWS-oriented realization of DSL should preserve the same architectural priorities defined in the main document while expressing them through AWS-native controls:

- API-first lifecycle handling through a single protected entry layer.
- Serverless-first control-plane behavior instead of static license servers.
- Security by design using least privilege, tenant-aware isolation, and managed key services.
- Observability through logs, metrics, traces, and auditable configuration state.
- Licensing as code through declarative policy and infrastructure delivery.

Reference Architecture Zones

The AWS realization can be understood as five cooperating zones.

The current review material is also supported by a small set of local AWS diagrams that fit this annex particularly well because they visualize the same architecture at three different levels of detail:

- a high-level end-to-end AWS architecture view,
- a control-plane-focused license-management view,
- a metering and usage-pipeline data-flow view.

These views are reflected in the structure of this annex so that the text and the diagrams can be reconciled later without changing the architectural intent.

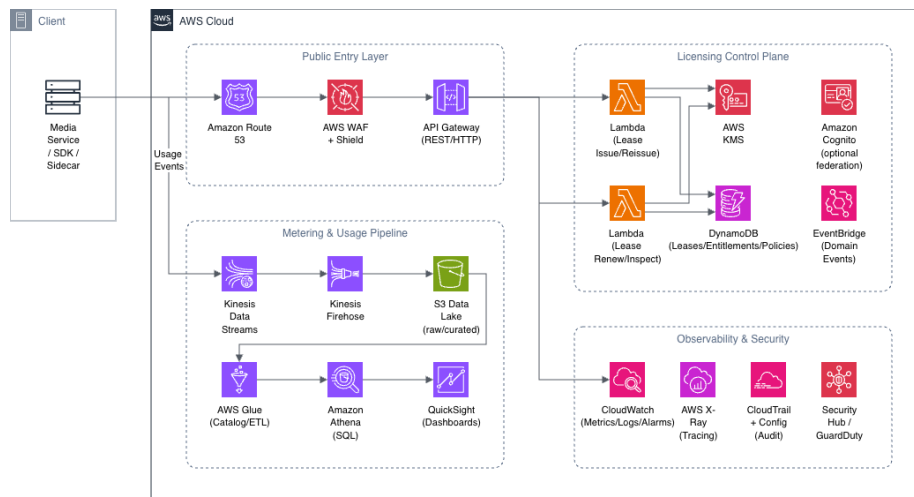


Figure 2: AWS high-level reference architecture

1. Clients and Media Services

Media workloads consume the licensing platform through SDKs, sidecars, or equivalent runtime integration. Typical workloads include transcoders, graphics engines, quality-control tools, and other media services that need issue, renew, reissue, validation, and usage reporting behavior.

2. Public Entry Layer

The public entry layer fronts the licensing platform and provides routing, protection, and authentication entry points.

Typical AWS services include:

- Amazon Route 53 for DNS entry.
- AWS WAF and AWS Shield for perimeter protection.
- Amazon API Gateway as the canonical API front door.
- Amazon Cognito or federated identity as an optional user and API authentication layer.

3. Licensing Control Plane

The control plane implements the core DSL lifecycle and trust behavior.

Typical AWS realizations include:

- AWS Lambda or container-based services for issue, renew, reissue, revoke, and inspection logic.
- Amazon DynamoDB for lease, entitlement, policy, quota, and token metadata.
- AWS KMS or CloudHSM-backed designs for signing and key-management duties.
- Amazon EventBridge for lifecycle, policy, and operational domain events.
- Amazon Aurora Serverless v2 where relational reporting or accounting views are needed.

4. Metering and Usage Pipeline

Where license modality depends on measured consumption, the metering pipeline captures raw usage, normalizes it, and prepares it for operational and commercial interpretation.

Typical AWS services include:

- Amazon Kinesis Data Streams for usage ingestion.
- Kinesis Data Firehose for managed delivery.
- Amazon S3 as the raw and curated data lake.
- AWS Glue for cataloging and normalization.

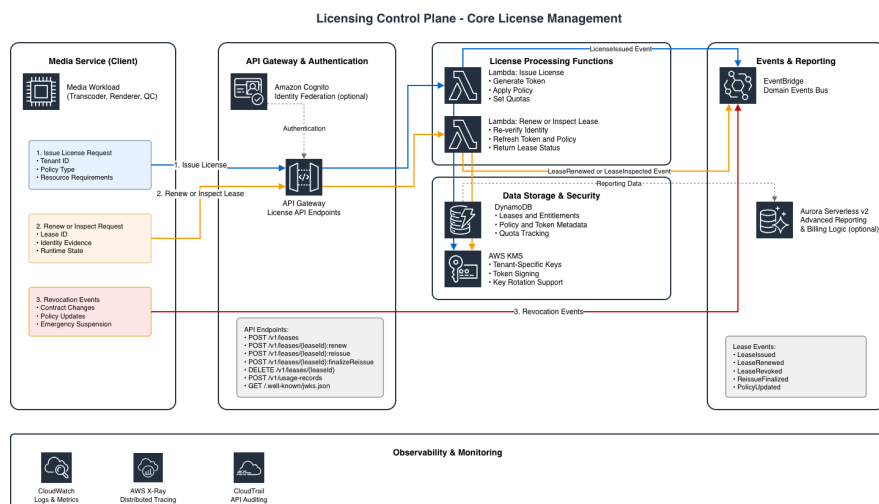


Figure 3: AWS licensing control plane detail

- Amazon Athena and Amazon QuickSight for queries, dashboards, forecasts, and consumption reporting.

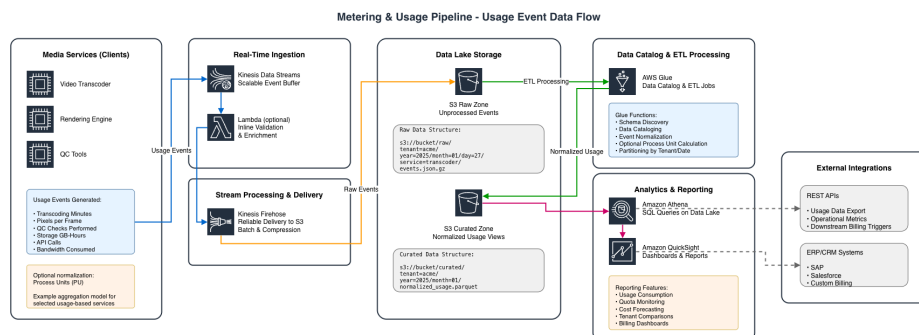


Figure 4: AWS metering and usage pipeline

5. Observability and Security Services

The platform should make license behavior transparent and auditable while maintaining secure defaults.

Typical AWS services include:

- Amazon CloudWatch for logs, metrics, and alarms.
- AWS X-Ray or OpenTelemetry-compatible tracing for end-to-end request visibility.
- AWS CloudTrail and AWS Config for audit and configuration traceability.

- AWS Security Hub, GuardDuty, Inspector, and Macie for broader security posture monitoring.

Example Mapping

DSL function	Example AWS realization
API gateway	Amazon API Gateway
Control-plane compute	AWS Lambda or container-based services
Lease and entitlement store	Amazon DynamoDB or Aurora
Key management	AWS KMS or CloudHSM-backed design
Event orchestration	Amazon EventBridge and Step Functions
Usage analytics landing zone	Amazon S3 with downstream analytics services
Workload identity evidence	EC2 instance identity, IAM role, or EKS service account federation
Observability	Amazon CloudWatch and OpenTelemetry-compatible tooling

AWS Account and Organization Structure

For multi-tenant or production-grade DSL deployments, the AWS account model is part of the security architecture rather than an administrative afterthought.

Common patterns include:

- AWS Organizations for centralized governance and billing structure.
- separation of security, logging, shared services, and workload accounts.
- distinct production, staging, and development boundaries.
- central guardrails through Service Control Policies and account vending controls.
- consistent tagging for tenant, service, environment, and cost attribution.

This structure supports least privilege, audit isolation, and cleaner separation between customer-facing runtime services and central platform operations.

Identity and Access Interpretation

AWS identity controls should align with the DSL trust model rather than replace it.

Important patterns include:

- prefer IAM roles and federated identities over long-lived users or static credentials,

- propagate tenant or project context through claims, role attributes, or session tags,
- keep secrets in AWS Secrets Manager or Systems Manager Parameter Store,
- use private connectivity and scoped service-to-service permissions where required,
- keep signing and encryption key access tightly bounded.

Example Interaction Pattern

An AWS-based implementation may follow this pattern:

1. A workload running on EC2, ECS, or EKS gathers AWS-native identity evidence.
2. A license agent calls the DSL issue or renew endpoint.
3. The control plane verifies identity evidence, entitlement, and quota.
4. The control plane signs a runtime token and returns policy and expiry data.
5. The workload validates the token locally and continues operation without a synchronous dependency on the control plane.

In metered scenarios, a parallel usage path may send normalized usage events through Kinesis or an equivalent ingestion path into the reporting and analytics layer.

Design Considerations

- IAM and workload identity should be preferred over instance-bound manual activation.
- Network design should preserve least privilege and tenant isolation.
- Short-lived tokens and JWKS distribution should be integrated with key rotation practices.
- Operational telemetry should make renewal, grace, reissue, and revoke behavior visible.
- Tenant isolation should be visible in both the data model and the AWS account or key management design.
- Billing and reporting components should remain downstream consumers of lifecycle and usage events rather than redefining the canonical licensing semantics.

Relationship to the Main Document

- Chapter 07 defines the platform-neutral reference model.
- Chapter 10 defines the canonical API behavior.

- Chapter 12 defines the trust and control expectations that still apply on AWS.

Annex Positioning

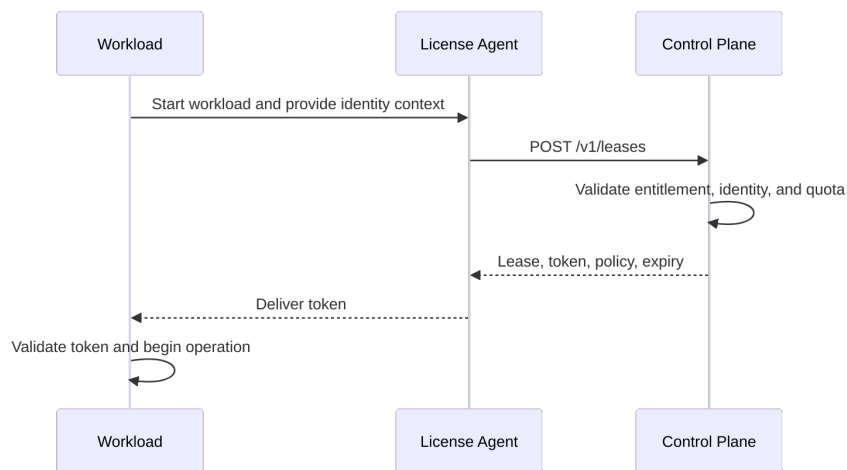
This annex gives reviewers a concrete cloud example while preserving the platform-neutral intent of the main best-practices text.

Annex G. Example Sequences

Purpose

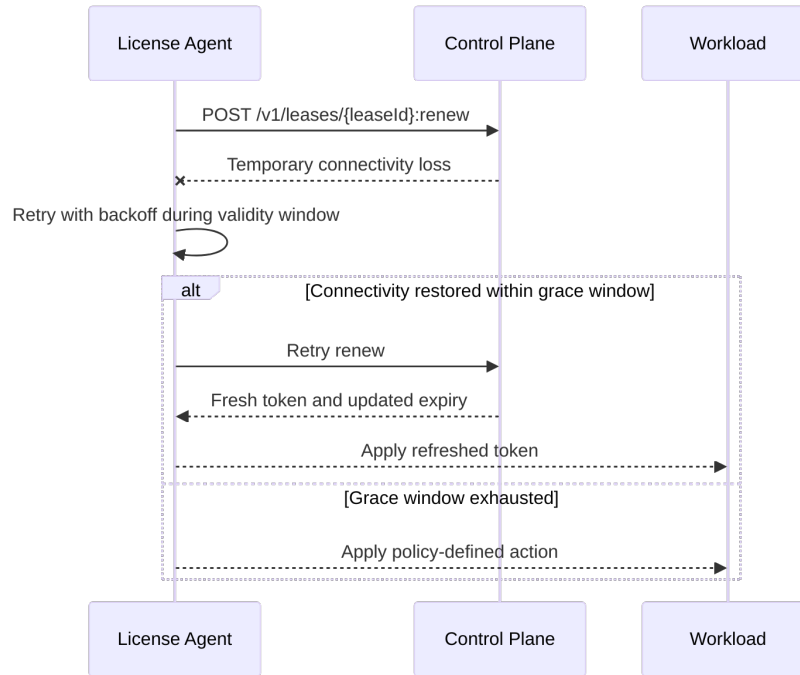
This informative annex provides compact example sequences that illustrate how the canonical lifecycle behaves in practice.

Sequence 1. Initial Issue



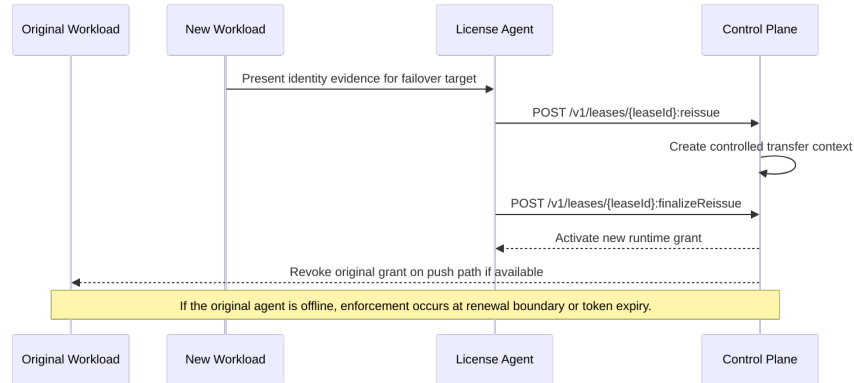
1. The workload starts and the license agent collects identity evidence.
2. The agent calls the issue endpoint with **app**, **tenantId**, requested dimensions, **licenseModality**, identity evidence, and **workId**.
3. The control plane validates entitlement, identity, and quota.
4. The control plane creates a lease, signs a token, and returns policy and expiry data.
5. The application validates the token locally and begins operation.

Sequence 2. Renewal with Temporary Connectivity Loss



1. The license agent opens the renewal window based on `renewBeforeSeconds`.
2. A renewal attempt fails because the control plane is temporarily unreachable.
3. The agent retries with backoff while the current token remains valid.
4. If connectivity returns within the grace window, renewal succeeds and a fresh token is applied.
5. If the grace window is exhausted, the workload applies the policy-defined action.

Sequence 3. Reissue for Failover



1. A standby workload becomes the intended execution target.
2. The new workload presents identity evidence and calls the reissue operation.
3. The control plane creates a controlled transfer context.
4. Finalization completes the cutover and revokes the original runtime grant.
5. If the original agent is offline, revocation is enforced at the next renewal boundary or token expiry.

Relationship to the Main Document

- Chapter 05 defines the workflow patterns that motivate these sequences.
- Chapter 10 defines the detailed API behavior.
- Annex D defines the baseline lifecycle states referenced by these examples.

Annex Positioning

This annex provides simple walkthroughs intended to make the published baseline easier to evaluate without redefining the normative model.

The numbered prose sequences remain the primary informative walkthroughs in this annex. The inline sequence diagrams complement them as visual summaries and do not supersede Annex D or Chapter 10.

Annex H. Persona Mapping

Purpose

This informative annex summarizes how the main personas introduced in Chapter 05 map to the most important DSL concern areas.

Persona-to-Concern Mapping

Persona	Primary concern areas
Director of Engineering	resilience, portability, predictable operational behavior
Operations and DevOps Engineer	automation, observability, repeatable lifecycle handling
Broadcast Engineer or Production Operator	continuity, low-friction activation, emergency flexibility
Finance and Procurement Lead	metering transparency, forecastability, modality clarity
Security and Compliance Lead	least privilege, auditability, trust boundaries
ISV Product and Commercial Owner	revenue protection, adoption friction, portfolio flexibility
ISV Licensing Architect	lifecycle correctness, token trust, interoperability
ISV Support and Customer Success Lead	diagnosability, explainable behavior, support scalability
SI Solution Architect or Integrator	semantic consistency, automation reuse, hybrid reliability
Platform and Infrastructure Stakeholders	workload identity, secure integration, operational fit

Interpretation Notes

- Multiple personas often converge on the same requirement area for different reasons.
- Operational transparency is relevant to both technical and commercial stakeholders.
- Identity-based binding is both a security concern and an enabler for mobility and failover.

Relationship to the Main Document

- Chapter 05.01 defines the stakeholder groups.
- Chapter 05.02 defines the detailed persona perspectives.
- Chapter 05.04 derives requirement themes from these concerns.

Annex Positioning

This annex provides a compact review aid for checking whether the draft remains balanced across stakeholder perspectives.

Annex I. Use Case Mapping

Purpose

This informative annex summarizes how the primary use case classes map to dominant lifecycle, metering, and continuity expectations.

Use Case Mapping Summary

Use case class	Dominant lifecycle emphasis	Typical modality emphasis	Key operational concern
Continuous operations	renew, inspect, reissue	low-granularity time commitments	uninterrupted continuity
Frequent event-driven production	issue, revoke, renew, inspect	recurring time-bounded uplift	repeatable scale-up and release
Infrequent event-driven production	issue, revoke, usage reporting	fine-grained or burst-oriented usage	short intense demand windows
Test and validation	issue, revoke, inspect	short-lived scoped usage	low-friction ephemeral access
AI-powered and outcome-based services	usage reporting, inspect, renew	process-based or outcome-based	trustworthy attribution
Resilience and continuity overlay	renew, reissue, revoke	profile-dependent	bounded failover and recovery

Interpretation Notes

- The resilience and continuity overlay is cross-cutting rather than a standalone primary use case class.
- More than one modality may apply to the same operational class.
- The mapping is intended to support traceability into Chapters 09, 13, and 14.

Relationship to the Main Document

- Chapter 05.03 defines the use case classes in detail.
- Chapter 05.05 defines the workflow patterns derived from them.
- Chapter 13 shows how those patterns surface in concrete operational profiles.

Annex Positioning

This annex is a compact review aid for cross-checking lifecycle and modality assumptions across the draft.

License

This publication is distributed under the Apache License, Version 2.0. The IAMT name, logo, and other trademarks are not licensed except as permitted by Section 6 of the Apache License, Version 2.0.

Apache License Version 2.0, January 2004 <http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

“License” shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

“Licensor” shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

“Legal Entity” shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, “control” means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

“You” (or “Your”) shall mean an individual or Legal Entity exercising permissions granted by this License.

“Source” form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

“Object” form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

“Work” shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work.

“Derivative Works” shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

“Contribution” shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on

behalf of the copyright owner. For the purposes of this definition, “submitted” means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as “Not a Contribution.”

“Contributor” shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.
3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.
4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:
 - (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
 - (b) You must cause any modified files to carry prominent notices stating that You changed the files; and
 - (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

- (d) If the Work includes a “NOTICE” text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

- 5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.
- 6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.
- 7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.
- 8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to

use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

This repository is licensed under the Apache License, Version 2.0.

The license does not grant rights to use the IAMT name, logo, or other trademarks, except as permitted by Section 6 of the Apache License, Version 2.0.